

Introduction to Operating Systems

Mythili Vutukuru
CSE, IIT Bombay

What is a computer system?

- Software + hardware to run user applications and programs, to accomplish some tasks
- Components:
 - **Hardware**: CPU, memory, I/O devices, ...
 - **System software**: Operating System (OS), ...
 - **User software**: user applications (browser, email client, games, ...)
- Real-life computer systems (e.g., an e-commerce website) have multiple interconnected computers, each running one or more applications that communicate with each other

Real life computer systems are complex

- Real life systems are **complex**
 - Multiple interacting components and sub-systems
 - Each component independently developed, but have to work together for a common purpose
 - Prone to failures, bugs, crashes
- But still, we expect:
 - The system always does what it is supposed to do ... (**functional correctness**)
 - Quickly, efficiently, for a large number of users, lots of data ... (**performance**)
 - Even when it is overloaded or when failures occur ... (**reliability**)

Why study operating systems?

- Knowledge of hardware (architecture) + system software (OS), and how user programs interact with these lower layers, is essential to writing “good” (high performance, reliable) user programs
 - What exactly happens when you run a user program?
 - How to make your program run faster and more efficiently?
 - Why do programs crash and how to avoid it?
 - How to make your programs more secure?
- OS expertise is one of the most important building blocks when building high performance, robust, complex real life systems

Beyond OS to real systems

- Architecture + OS: Basic foundation to understand how a user program runs on a single machine
- Networking: How programs talk to each other across machines
- Databases and data storage: How applications store data efficiently and reliably in one or more machines
- Performance engineering: how to make programs run faster
- Distributed systems: How multiple applications across multiple machines work together to perform a useful task reliably
- Virtualization, cloud computing, security, ...

What is an operating system?

- Middleware between user programs and system hardware
 - Not user application software but **system software**
 - Example: Linux, Windows, MacOS
- Manages computer hardware: CPU, main memory, I/O devices (hard disk, network card, mouse, keyboard etc.)
 - User applications do not have to worry about low-level hardware details
- Operating system has **kernel** + other extra useful software
 - Kernel = the core functionality of the OS
 - Other useful programs = shell, commands on shell, other utilities that help users interact with the OS

What is an operating system?

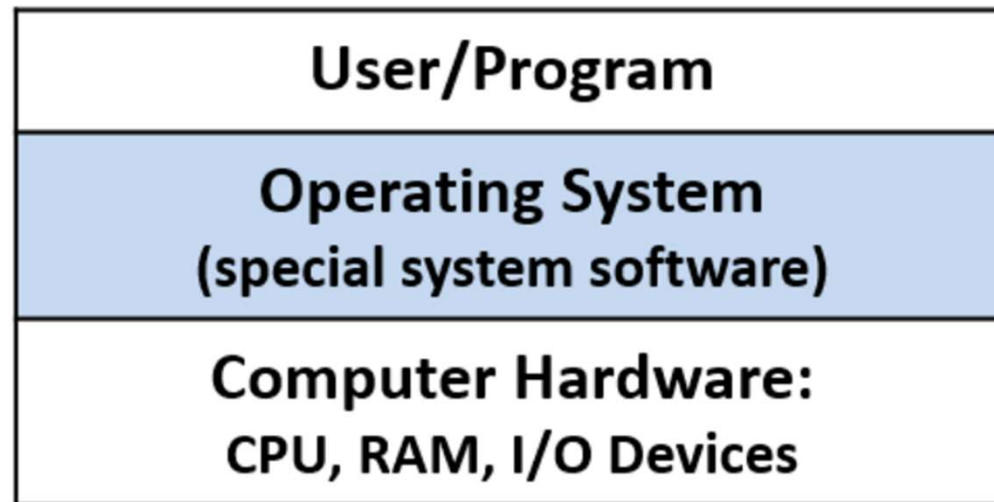


Figure 1. The OS is special system software between the user and the hardware. It manages the computer's hardware and implements abstractions to make the hardware easier to use.

History of operating systems

- Started out as a library to provide common functionality to access hardware, invoked via function calls from user program
 - **Convenient** to use OS instead of each user writing code to manage hardware
 - Centralized management of hardware resources is more **efficient**
- Later, computers evolved from running a single program to multiple processes concurrently
 - Multiple untrusted users must share same hardware
- So OS evolved to become trusted system software providing **isolation** between users, and protecting hardware
 - Multiple users are isolated and protected from each other
 - System hardware and software is protected from unauthorized access by users

Hardware organization

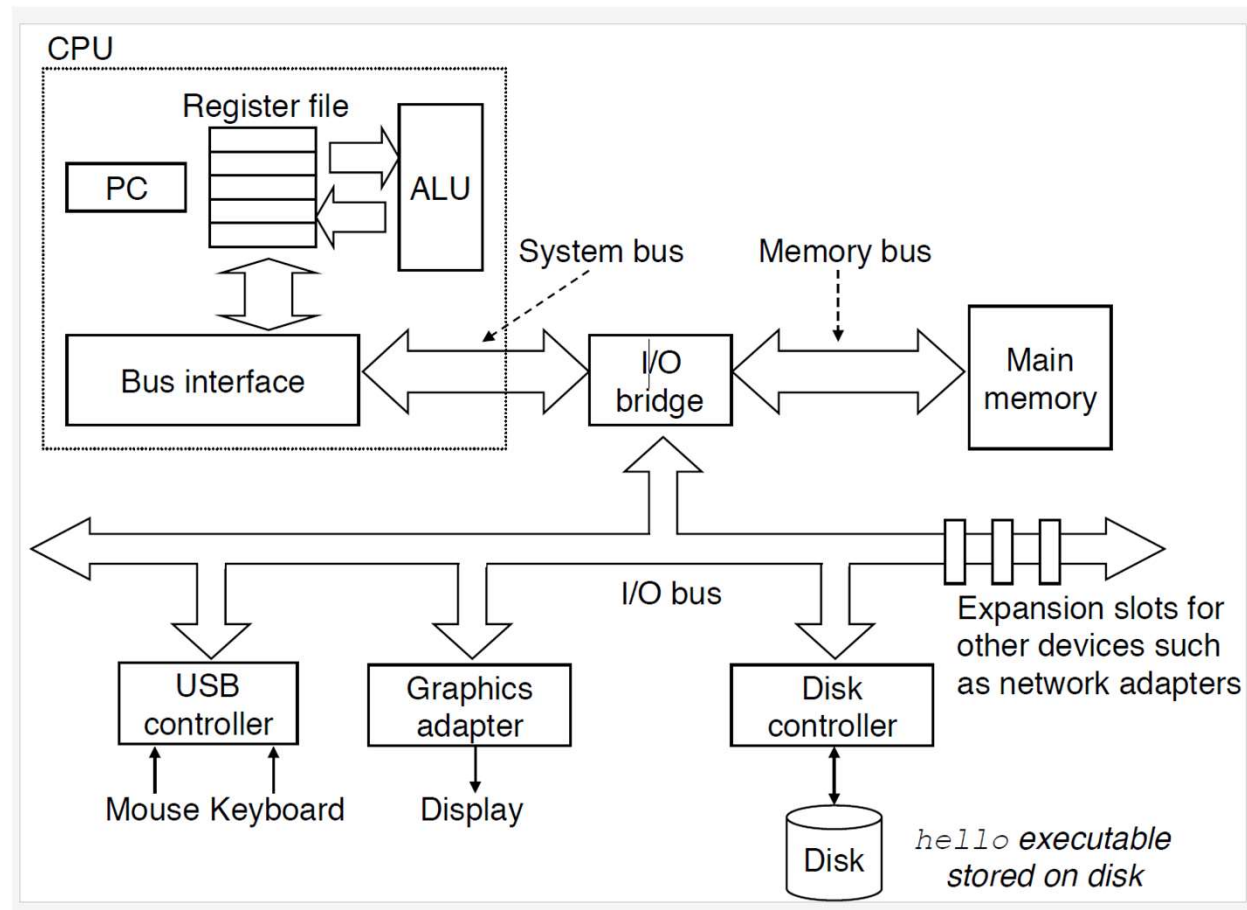


Image credit: CSAPP

What is a program?

- User **program** = code (instructions for CPU) + data
- Stored program concept
 - User programs stored in main memory or **Random Access Memory (RAM)**
 - Instructions/data occupy multiple contiguous bytes in memory
 - Memory is **byte-addressable**: data accessed via memory address / location / byte#
 - CPU fetches code/data from RAM using memory address, and executes instructions
- CPU runs **processes = running programs**
- Modern CPUs have multiple **cores** for parallel execution
 - Each core runs one process at a time each
 - Modern CPUs have hyper-threading, where each core can run more than one process also (OS treats hyper-threading cores also as multiple CPU cores)

CPU ISA

- Every CPU has
 - A set of **instructions** that the hardware can execute
 - A set of **registers** for temporary storage of data within the CPU
- High level language (C code) translated into CPU instructions by compiler
 - Can directly write machine code, but cumbersome
- Instructions and registers defined by **ISA** = Instruction Set Architecture
 - Specific to CPU manufacturer (e.g., Intel CPUs follow x86 ISA)
- Registers: special registers (specific purpose) or general purpose
 - **Program counter (PC)** is special register, has memory address of the next instruction to execute on the CPU
 - General purpose registers can be used for anything, e.g., operands in instructions
- Size of registers defined by architecture (32 bit / 64 bit)

CPU instructions

- Some common examples of CPU instructions
 - **Load**: copy content from memory location → register
 - **Store**: copy content from register → memory location
 - **Arithmetic** and **logical** operations like add: $\text{reg1} + \text{reg2} \rightarrow \text{reg3}$, compare, ..
 - **Jump**: change value of PC
- Simple model of CPU
 - Each clock cycle, **fetch** instruction at PC, **decode**, access required data, **execute**, update PC, repeat
 - PC increments to next instruction, or jumps to some other value
- Many optimizations to this simple model
 - **Pipelining**: run multiple instructions concurrently in a pipeline
 - Many more in modern CPUs to optimize #instructions executed per clock cycle

Memory hierarchy

- Hierarchy of storage elements which store instructions and data
 - CPU registers (small number, accessed in <1 nanosec)
 - Multiple levels of CPU caches (few MB, 1-10 nanosec)
 - Main memory or RAM (few GB, ~100 nanosec)
 - Hard disk (few TB, ~1 millisec)
- Hard disk is non-volatile storage, rest are volatile
 - Hard disk stores files and other data persistently
- As you go down the hierarchy, memory access technology becomes cheaper, slower, less expensive
- CPU caches **transparent** to OS, managed by hardware
 - Software only accesses memory, doesn't know if served from cache or DRAM

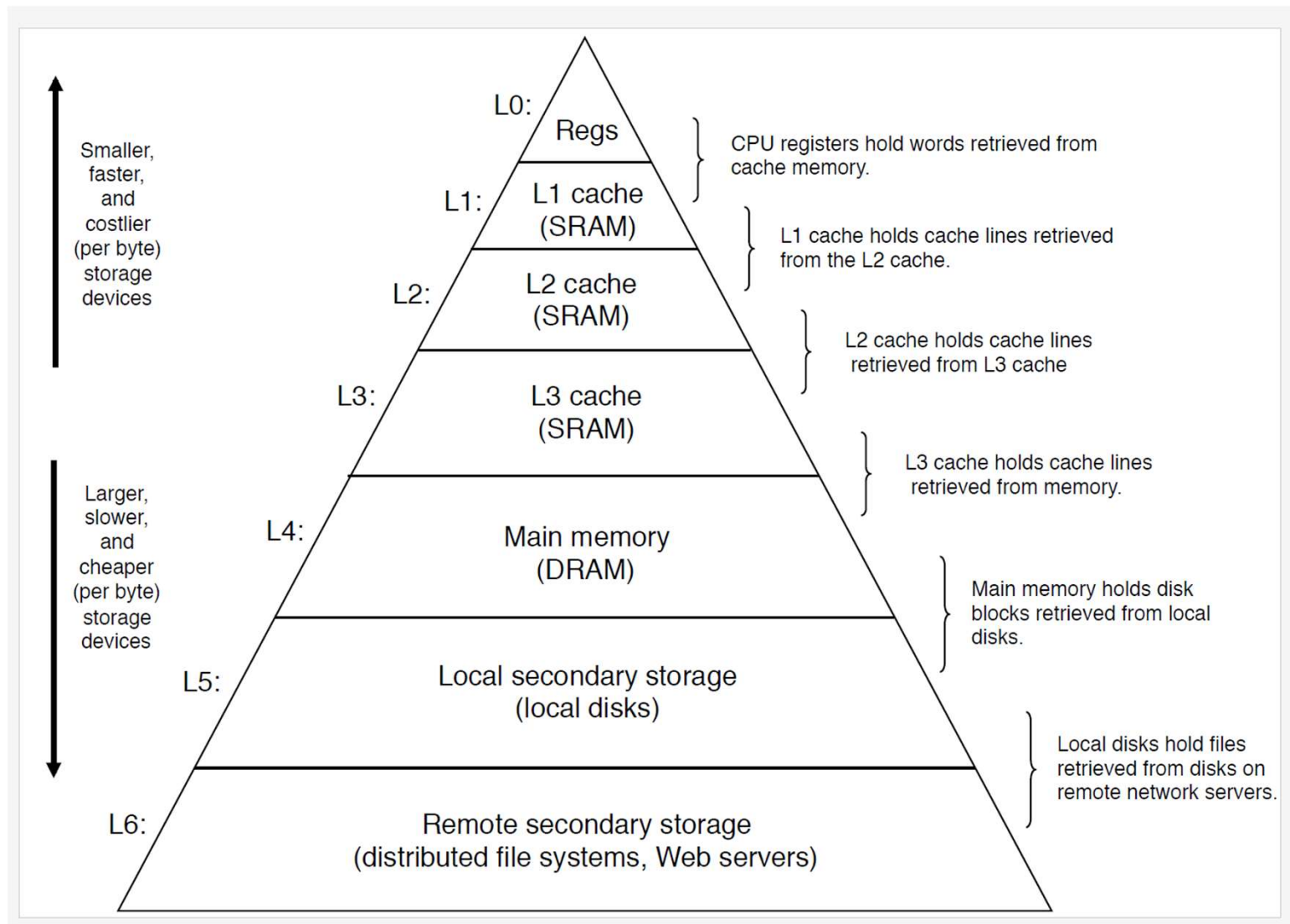
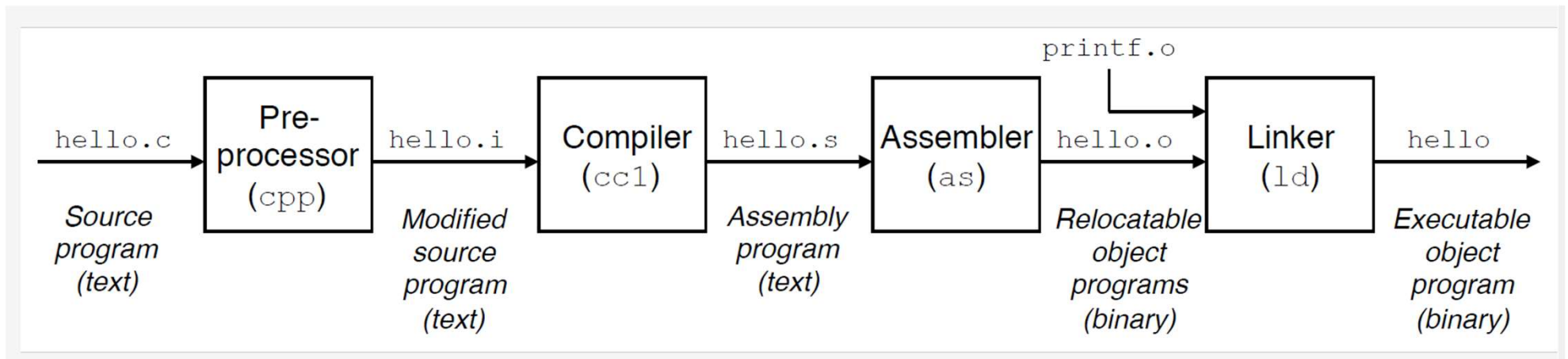


Image credit: CSAPP

Running a program

- What happens when you run a C program?
 - C code translated into executable by compiler

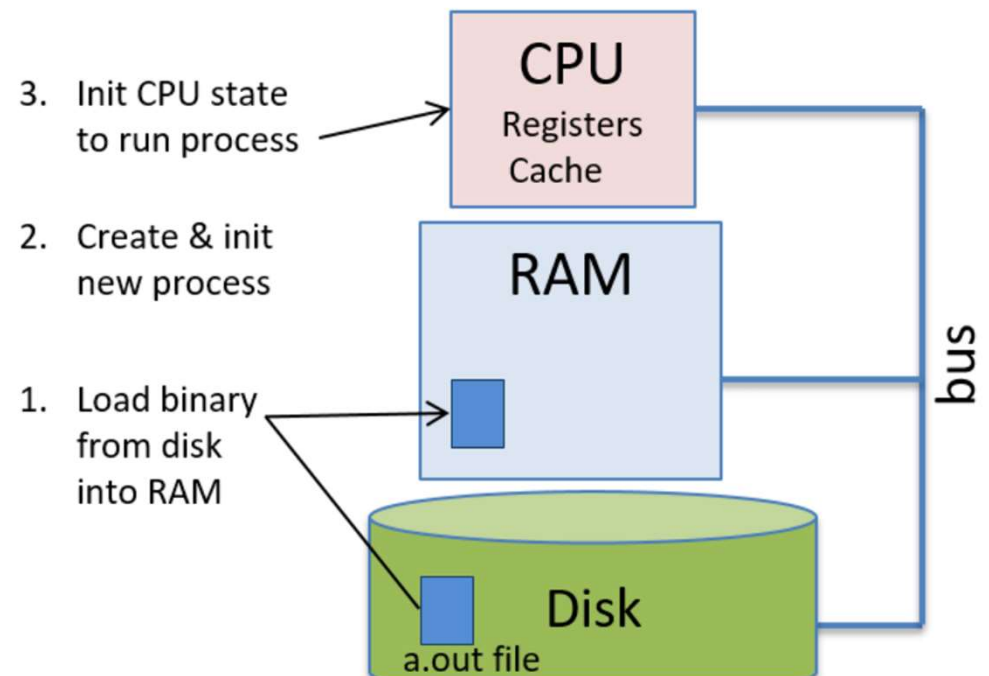


Running a program

- What happens when you run a C program?
 - C code translated into executable by compiler
 - Executable file stored on hard disk (say, “a.out”)
 - When executable is run, a new **process** is created
 - Process allocated space in RAM to store code and data
 - CPU starts executing the instructions of the program
- When CPU is running a process, CPU registers contain the execution **context** of the process
 - PC points to instruction in the program, general purpose registers store data in the program, and so on

Role of OS in running a process

- Allocates memory for new process in RAM
 - Loads code, data from disk
- Initializes CPU context
 - PC points to first instruction
- Process starts to run
 - OS steps in as needed



xv6: a simple teaching OS

- xv6 is a simple OS for easy teaching of OS concepts
 - Two versions, one for x86 hardware and one for RISC-V hardware
 - This series of lectures based on x86 version
 - <https://github.com/mit-pdos/xv6-public>
- Easy to read code, simple exercises to write OS code
- Much simpler than real OS like Linux, but basic concepts remain the same
- Runs inside QEMU emulator

OS code in C or assembly?

- OS is also like any other program run by CPU, but it is the most important program that manages other programs
 - OS code mostly written in a high-level language like C, compiled into executable, loaded at boot time
- But some parts of OS are written directly in assembly language or CPU instructions that the hardware can understand?
 - Why not write everything in C? Not possible to express certain low level actions performed by OS in high level language
- Basic understanding of x86 assembly code required for understanding xv6 OS code in this course

Learn how to use xv6

- xv6 source code is available online
 - xv6 kernel code
 - User programs to test OS functionality, e.g., simple shell programs like ls
- Instructions have been provided for you to learn how to:
 - Set up QEMU and other software needed to run xv6 (on your personal laptops; the lab machines should have all of this)
 - Compile and run xv6, for example, execute simple shell commands like “ls” in the xv6 shell
 - Add your own simple “command” to the xv6 code, make necessary changes to the code and Makefile, and rerun xv6 again

Reference: x86 registers

- General purpose registers: store data during computations (eax, ebx, ecx, edx, esi, edi)
- Pointers to stack locations: base of stack (ebp) and top of stack (esp)
- Program counter or instruction pointer (eip): next instruction to execute
- Control registers: hold control information or metadata of a process (e.g., cr3 has information related to memory of process)
- Segment registers (cs, ds, es, fs, gs, ss): information about segments (related to memory of process)

Reference: x86 instructions

- Load/store: *mov src, dst*
 - *mov %eax, %ebx* (copy contents of eax to ebx)
 - *mov (%eax), %ebx* (copy contents at the address in eax into ebx)
 - *mov 4(%eax), %ebx* (copy contents stored at offset of 4 bytes from address stored at eax into ebx)
- Push/pop on stack: changes esp
 - *push %eax* (push contents of eax onto stack, update esp)
 - *pop %eax* (pop top of stack onto eax, update esp)
- *jmp* sets eip to specified address
- *call* to invoke a function, *ret* to return from a function
- Variants of above (*movw, pushl*) for different register sizes

Reference: Mechanics of function call

- Local variables, arguments, return address stored on stack for duration of function call
- What happens in a function call?
 - Push function arguments on stack
 - *call fn* (instruction pushes return address on stack, jumps to function)
 - Allocate local variables on stack
 - Run function code
 - *ret* (instruction pops return address, eip goes back to old value)
- All of this is automatically done by the C compiler for you, and is part of the C calling convention.

Reference: Caller and callee save registers

- What about values in registers that existed before function call?
Registers can get clobbered during a function call, so how can computation resume?
 - Some registers saved on stack by caller before invoking the function (caller save registers). Function code (callee) can freely change them, caller restores them later on.
 - Some registers saved by callee function and restored after function ends (callee save registers). Caller expects them to have same value on return.
 - Return value stored in eax register by callee (one of caller save registers)
- All of this is automatically done by C compiler (C calling convention)

Reference: More details of function call

- Anatomy of a function call
 - Push caller save registers (eax, ecx, edx)
 - Push arguments in reverse order
 - Return address (old eip) pushed on stack by the call instruction
 - Push old ebp on stack
 - Set ebp to current top of stack (base of new “stack frame” of the function)
 - Push local variables and callee save registers (ebx, esi, edi)
 - Execute function code
 - Pop stack frame and restore old ebp
 - Return address popped and eip restored by the ret instruction
- Stack pointers: ebp stores address of base of current stack frame and esp stores address of current top of stack
 - Function arguments are accessible from looking under the stack base pointer

OS Mechanisms

Mythili Vutukuru
CSE, IIT Bombay

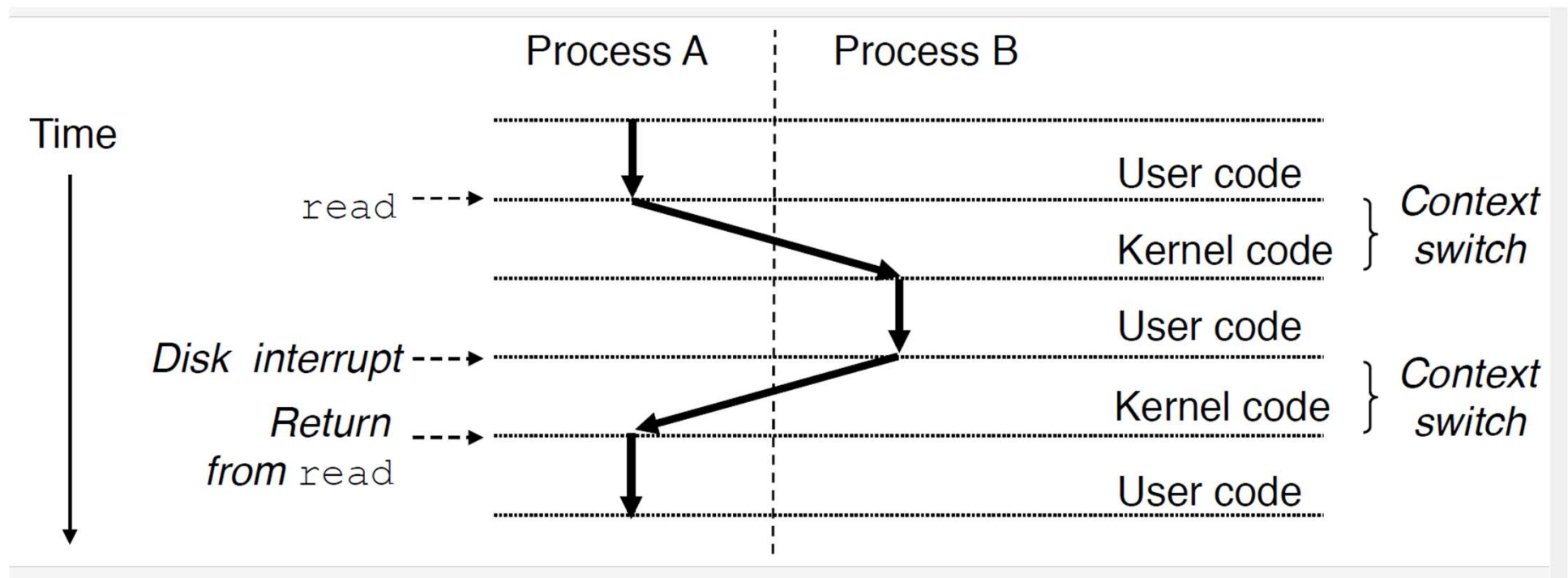
Key concepts in OS

- The OSTEP textbook identifies 3 concepts that are fundamental to OS:
- **Virtualization**: OS gives a “virtual” or logical view of the hardware to the users, hiding the messy real “physical” view
- **Concurrency**: OS runs multiple user programs at the same time, giving each user/program the illusion that it has the entire hardware to itself
- **Persistence**: OS stores user data persistently on external I/O devices
- We will now understand these concepts and other OS terminology

Concurrent execution & CPU virtualization

- CPU runs multiple programs concurrently
 - Run one process, switch to another, switch again, ...
- How does OS ensure correct concurrent execution?
 - Run user code of process A for some time
 - Pause A, **save context** of A, load context of B: **context switching**
 - Run user code of process B for some time
 - Pause B, save context of B, restore context of A, run A
- Every process thinks it is running alone on CPU
 - **Saving and restoring context** ensures process sees no disruption
 - OS takes care of this switching across processes
- In this manner, OS **virtualizes CPU** across multiple processes
- OS **scheduler** decides which process to run on which CPU at what time

Context switching



Memory allocation for a process

- When is memory allocated for code/data in RAM?
- When OS creates process, memory to store compiled executable allocated in RAM
 - Executable contains code (instructions) in the program, global/static variables in program
- Should we allocate memory for local variables, arguments of functions in executable?
 - No, since we do not now if/how many times the function will be called at runtime
- Similarly, malloc is for dynamic memory allocation at runtime, not compile time

```
int g;

int increment(int a) {
    int b;
    b = a+1;
    return b;
}

main() {
    int x, y;
    x = 1;
    y = increment(x);

    int *z = malloc(40);
}
```

Example memory allocation

- Variable “g” allocated at compile time
- Function local variables, arguments stored on “stack”
 - Example: variables “x”, “y” of main, variables “a”, “b” of function “increment”
 - During function call, arguments and local variables are “pushed” (allocated memory) on the stack, “popped” when function returns
- Dynamically memory allocated on “heap”
 - Malloc returns address of allocated chunk on heap
 - Example: memory address of 40 bytes on heap is stored in pointer variable “z” which is on the stack

```
int g;  
  
int increment(int a) {  
    int b;  
    b = a+1;  
    return b;  
}  
  
main() {  
    int x, y;  
    x = 1;  
    y = increment(x);  
  
    int *z = malloc(40);  
}
```

Memory image of a process

- Memory image of a process: code+data of process in memory
 - **Code**: CPU instructions in the program
 - **Compile-time data**: global/static variables in program executable
 - **Runtime data**: stack+heap for dynamic memory allocation at runtime
- Heap and stack can grow/shrink as process runs, with help of OS
 - Stack pointer CPU register keeps track of top of stack
- Memory image also contains other code (not directly part of the program) that the process may want to execute, e.g., programming language libraries, kernel code and data, and so on

Address space of a process

- Which memory addresses contain what part of memory image?
- OS gives every process the illusion that its memory image is laid out contiguously from memory address 0 onwards
 - This view of process memory is called the **virtual address space**
- In reality, processes are allocated free memory in small chunks all over RAM at some physical addresses, which the programmer is not aware of
 - Pointer addresses printed in a program are virtual addresses, not physical
- When a process accesses a virtual address, OS arranges to retrieve data from the actual physical address
- OS **virtualizes memory** for all processes

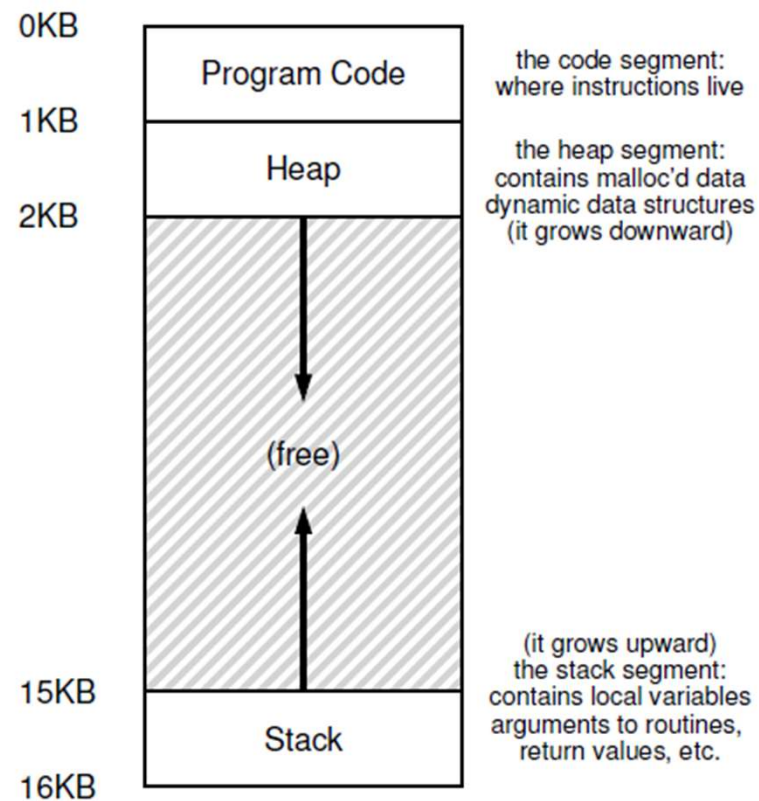


Figure 13.3: An Example Address Space

Isolation and privilege levels

- How to protect concurrent processes from one another?
 - Can one process mess up the code or data of another process?
 - When we virtualize, how do we share safely?
- Modern CPUs have mechanisms for **isolation**
- **Privileged** and **unprivileged** instructions
 - Privileged instruction access (perform) sensitive information (actions)
 - Regular instructions (e.g., add) are unprivileged
- CPU has multiple modes of operation (Intel x86 CPUs run in 4 **rings**)
 - Low privilege level (e.g., ring 3) only allows unprivileged instructions
 - High privilege level (e.g., ring 0) allows privileged instructions also

User mode and kernel mode

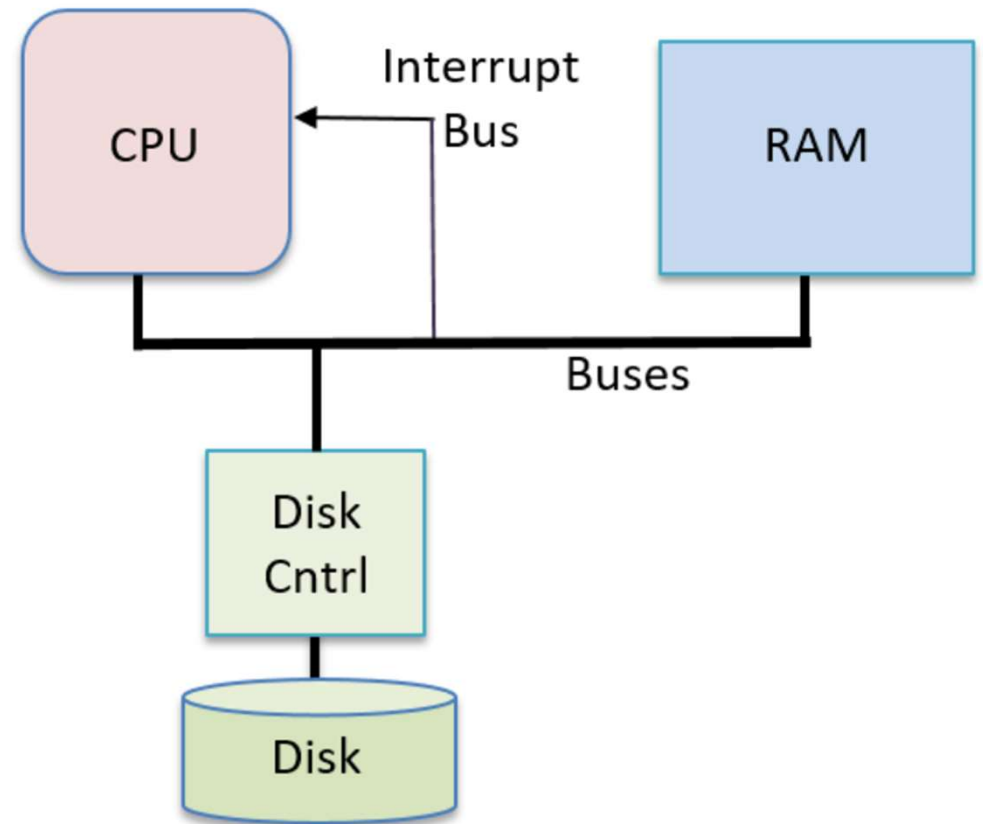
- User programs runs in user (unprivileged) mode
 - CPU is in unprivileged mode, executes only unprivileged instructions
- OS runs in kernel (privileged) mode
 - CPU is in privileged mode, can execute both privileged and unprivileged instructions
- CPU shifts from **user mode** to **kernel mode** and executes OS code when following events occur:
 - System calls: user request for OS services
 - Interrupts: external events that require attention of OS
 - Program faults: errors that need OS attention
- After performing required actions, OS returns back to user program, CPU shifts back to user mode

System calls

- When user program requires a service from OS, it makes a **system call**
 - Example: Process makes system call to read data from hard disk
 - Why? User process cannot run privileged instructions that access hardware
 - CPU jumps to OS code that implements system call, and returns back to user code
- Normally, user program does not call system call directly, but uses language library functions
 - Example: printf is a function in the C library, which in turn invokes the system call to write to screen

Interrupts

- In addition to running user programs, CPU also has to handle external events (e.g., mouse click, keyboard input)
- **Interrupt** = external signal from I/O device asking for CPU's attention
- Example: program issues request to read data from disk, and disk raises interrupt when data is available (instead of program waiting for data)



System calls vs. interrupts

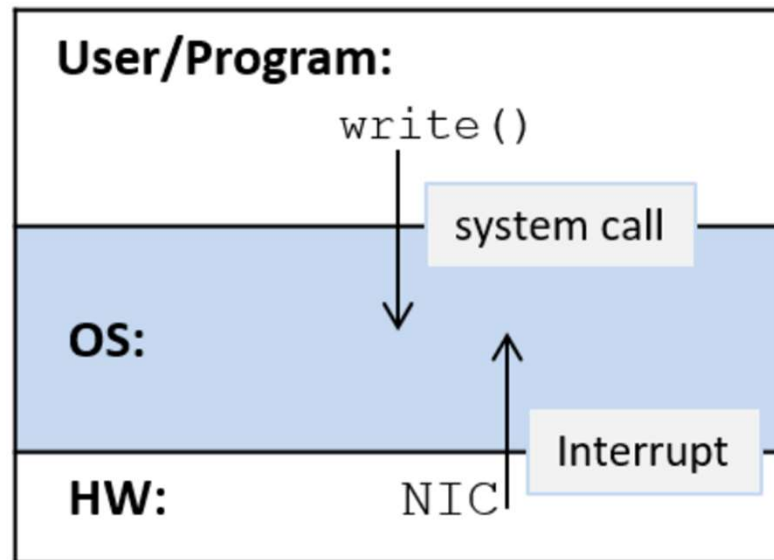
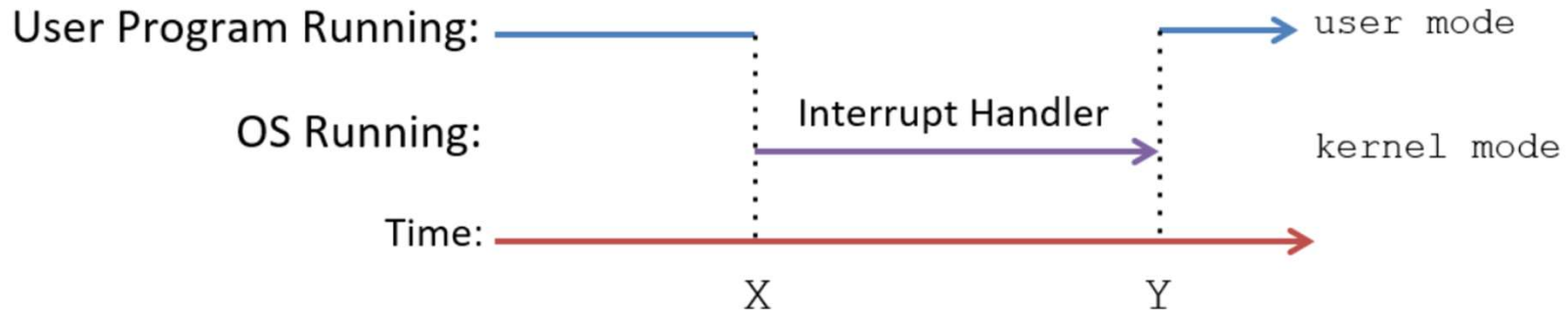


Figure 2. In an interrupt-driven system, user-level programs make system calls, and hardware devices issue interrupts to initiate OS actions.

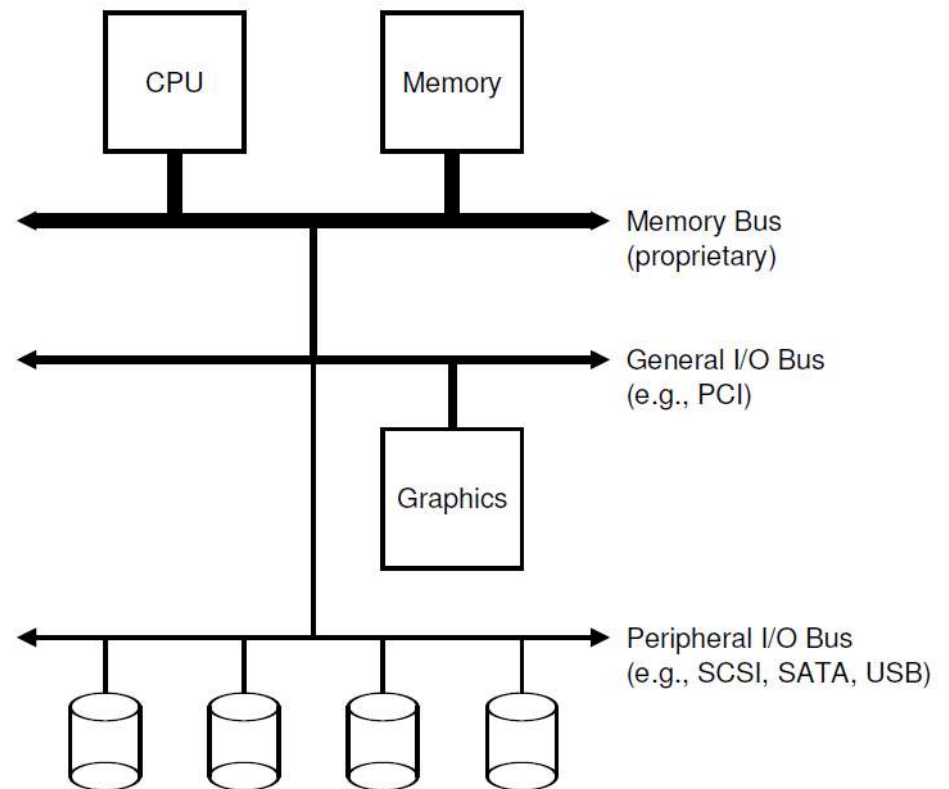
Interrupt handling

- How are interrupts handled?
 - CPU is running process P and interrupt arrives
 - CPU saves context of P, runs OS code to handle interrupt (e.g., read keyboard character) in kernel mode
 - Restore context of P, resume P in user mode
- Interrupt handling code is part of OS
 - CPU runs **interrupt handler of OS** and returns back to user code



I/O devices

- CPU and memory connected via high speed system (memory) bus
- I/O devices connect to the CPU and memory via other separate buses
 - Interface with external world
 - Store user data persistently
- OS manages I/O devices on behalf of the users



Device controller and device driver

- I/O device is managed by a **device controller**
 - Microcontroller which communicates with CPU/memory over bus
- Device specific knowledge required to correctly communicate with device controller to handle I/O operations
 - Done by special software called **device driver**
 - Part of operating system code
- Functions performed by kernel device driver
 - Initialize I/O devices
 - Start I/O operations, give commands to device (e.g., read data from hard disk)
 - Handle interrupts from device (e.g., disk raises interrupt when data is ready)

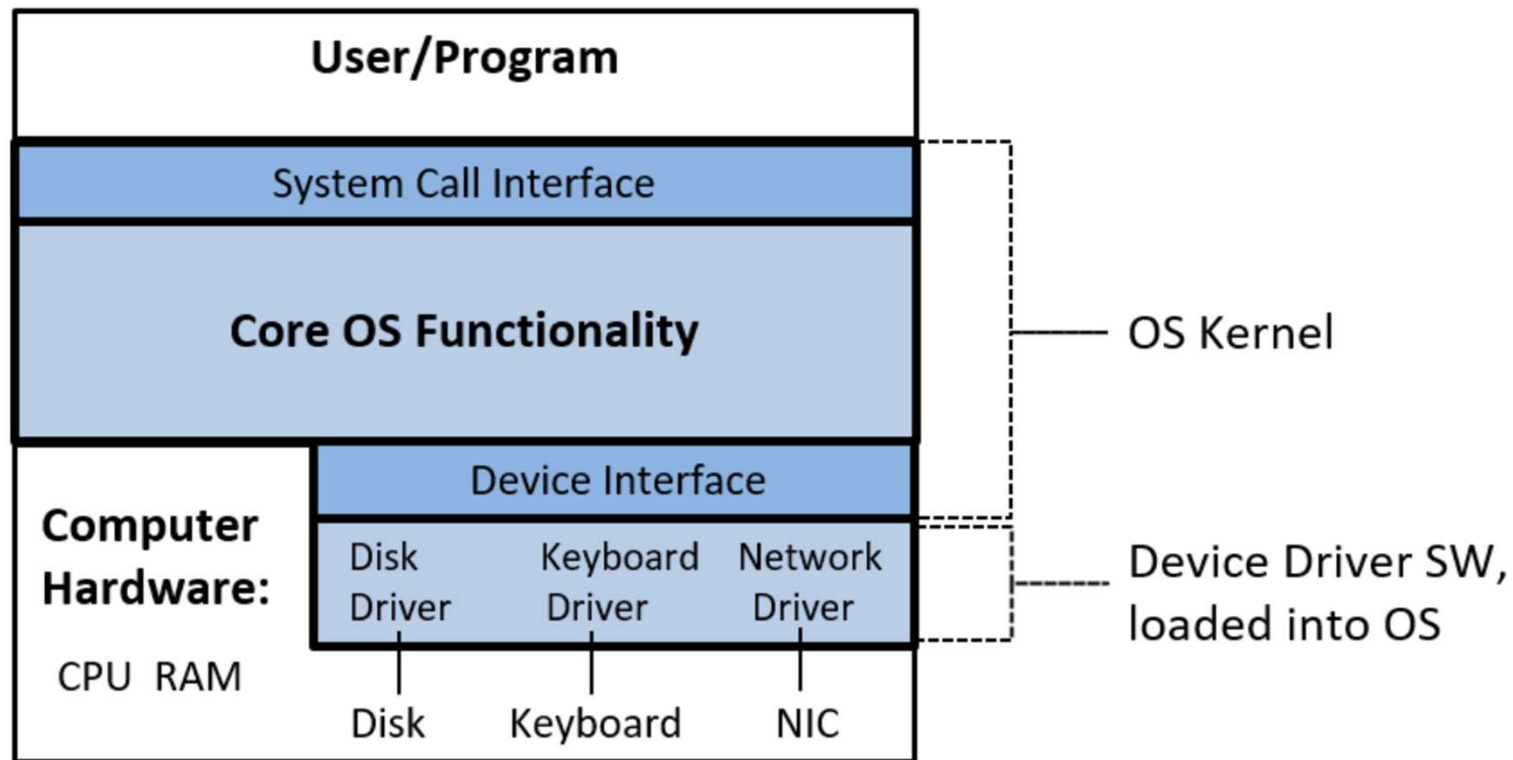


Figure 2. The OS kernel: core OS functionality necessary to use the system and facilitate cooperation between I/O devices and users of the system

Processes

Mythili Vutukuru
CSE, IIT Bombay

The process abstraction

- Process is a running program
- When program is run, OS creates new process, allocates memory, initializes CPU context, and starts process in user mode
- User program runs on CPU normally, unless OS needs to step in for system calls, interrupts, ...

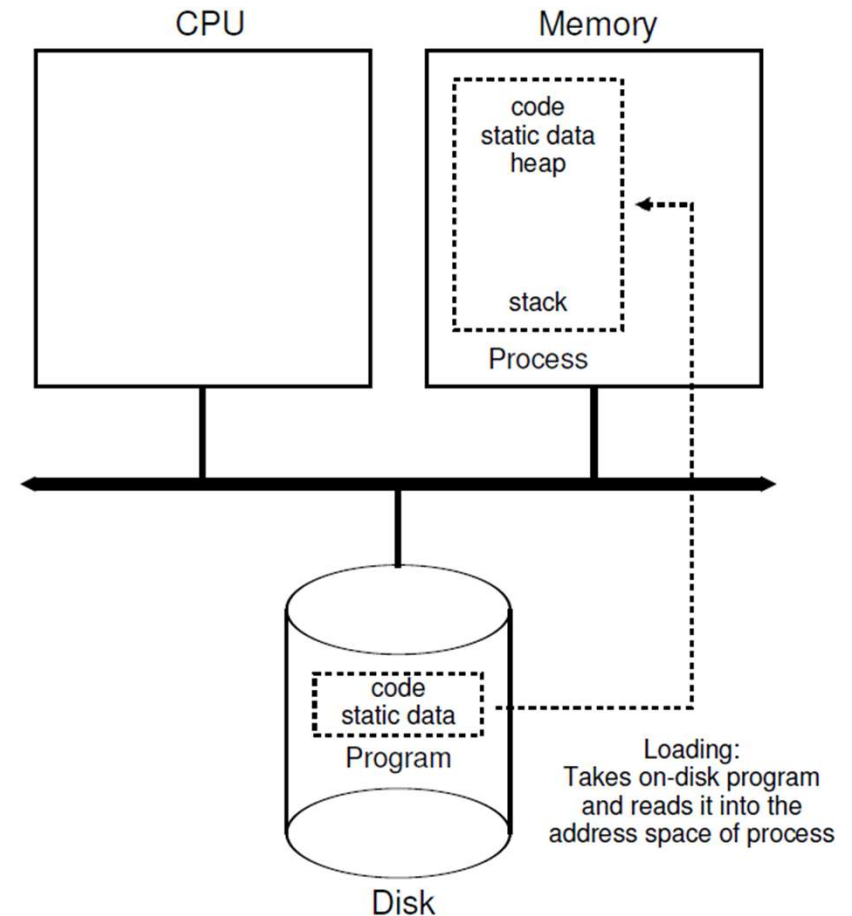


Figure 4.1: Loading: From Program To Process

What defines a process?

- Every process has a unique **process identifier (PID)**
- Process occupies some memory in RAM (**memory image**)
 - Code+data from executable
 - Stack+heap allocated for runtime use
- The execution **context** of the process (values of CPU registers)
 - PC has address of instruction of process, some registers have process data
 - Process context is in CPU registers when process is running on CPU
 - Context saved in memory when process is paused, restored when run again
- Ongoing **communication with I/O devices**
 - Information is maintained about files that are open, ongoing network connections, other active connections to I/O devices

States of a process

- OS manages multiple active processes at the same time. An active process can be in one of the following situations.
- **Running**: currently executing on CPU
 - CPU registers contain context of process
- **Blocked/suspended/sleeping**: process cannot run for some time
 - Example: process has requested data from disk, command issued, but process cannot proceed until the data from disk is available
- **Ready/runnable**: ready to run but waiting for OS scheduler to switch the process in
 - Many processes can be ready but scheduler can only run one on a CPU core
- Context of blocked and ready processes is saved in memory, so that they can continue to run later on

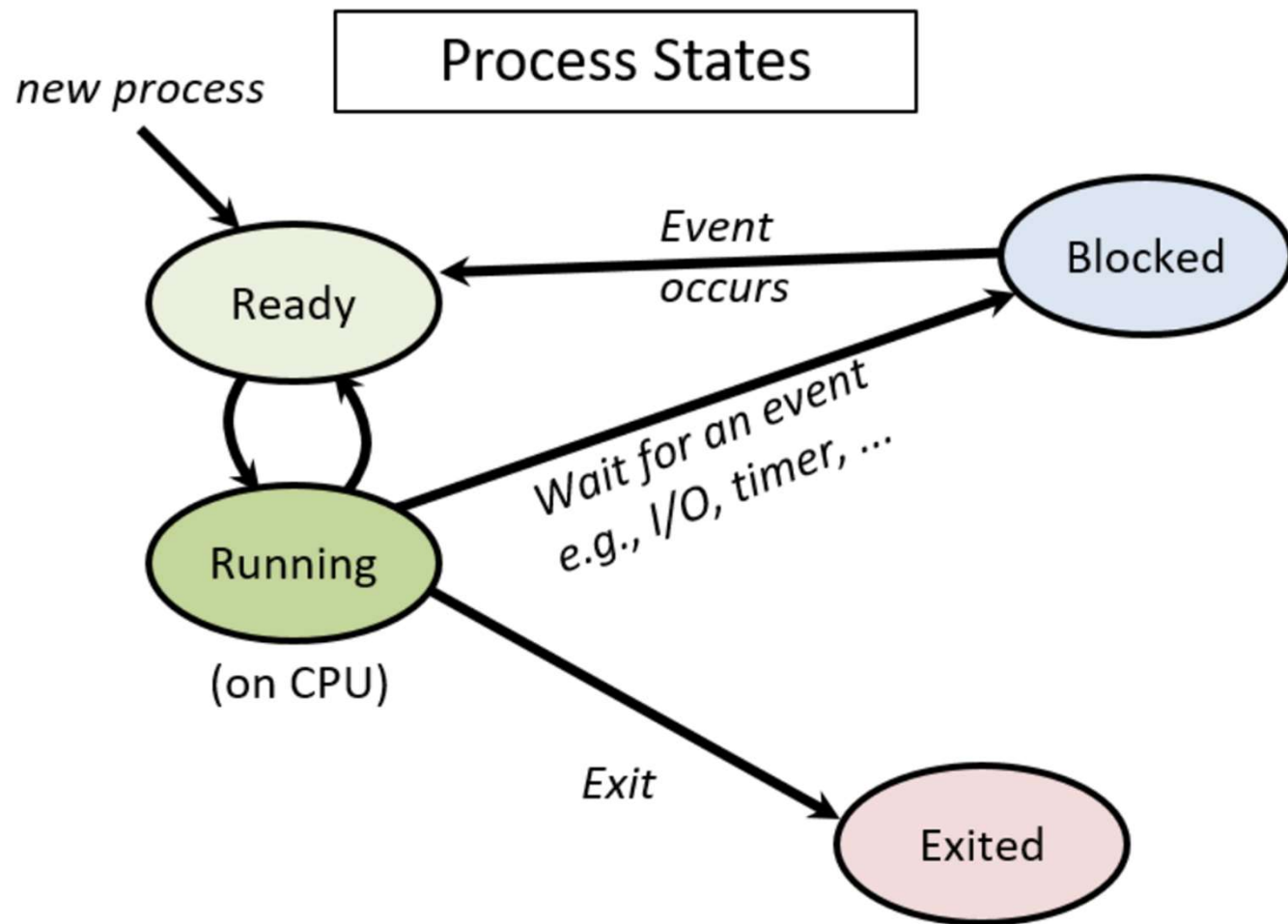


Figure 1. The states of a process during its lifetime

Example: process state transitions

- Consider a system that has two user processes P0 and P1
 - Initially P0 is running, P1 is ready and awaiting its turn
 - P0 wants to read a file from disk via a system call
 - OS handles the system call and gives command to disk, but data is not available immediately
 - Process P0 is moved to blocked state, OS switches to process P1
 - Process P1 runs for some time, and then an interrupt occurs from disk
 - CPU jumps to OS which handles interrupt, P0 is moved to ready state
 - OS can continue to run P1 again after interrupt and OS scheduler switches to ready process P0 later on after some time

Example: process state transitions

Time	Process ₀	Process ₁	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	Process ₀ initiates I/O
4	Blocked	Running	Process ₀ is blocked,
5	Blocked	Running	so Process ₁ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	Process ₁ now done
9	Running	–	
10	Running	–	Process ₀ now done

Figure 4.4: Tracing Process State: CPU and I/O

Process State Transitions

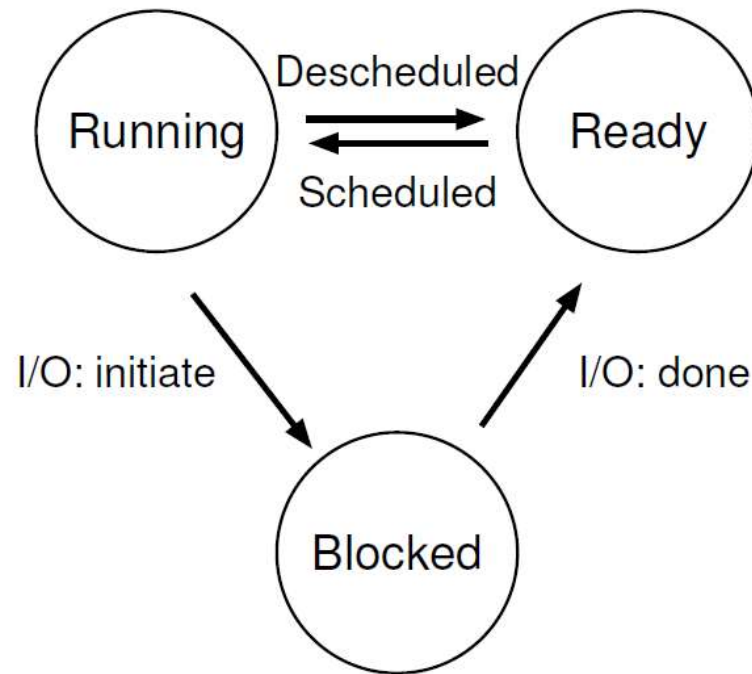


Figure 4.2: Process: State Transitions

Process control block (PCB)

- All information about a process is stored in a kernel data structure called the **process control block (PCB)**
 - Process identifier (PID)
 - Process state (running, ready, blocked, terminated, ..)
 - Pointers to other related processes (parent, children)
 - Saved CPU context of process when it is not running
 - Information related to memory locations of a process
 - Information related to ongoing I/O communication
 - ...
- PCB is known by different names in different OS
 - struct proc in xv6
 - task_struct in Linux

CPU scheduler

- Every OS maintains list of PCBs in some data structure
 - Array, linked list, heap – what is suitable when?
- Scheduler loops through list of PCBs and picks process to run, switches to process, switches to another process after some time, and this continues...
- Scheduler picks one process to run on every core, so number of running process is the number of parallel processors available

Booting

- What happens when you boot up a computer system?
- Basic Input Output System (**BIOS**) starts to run
 - Resides in non-volatile memory, sets up all other hardware
- BIOS locates the **boot loader** in the boot disk (hard disk, USB, ..)
 - Simple program whose job is to locate and load the OS
 - Present in the first sector of the boot disk
 - Combination of assembly and C code
- Boot loader sets up CPU registers suitably, loads kernel image from disk to memory, transfers control to kernel
- OS code starts to run, exposes terminal to user, user starts programs

Booting real systems

- Bootloader must fit into 512 bytes (first sector of boot disk) to be found easily by BIOS
- Bootloaders for simple/old OS could fit into one sector, but no longer the case for modern OS
- Real life bootloaders are complex, need to read a large kernel image from disk and network, do not fit into 512 bytes
- Real life booting is two step process: BIOS loads simple bootloader, which loads a more complex bootloader, which then loads the OS

System calls for process management

Mythili Vutukuru
CSE, IIT Bombay

API for process management

- What API does OS provide to user programs to manage processes?
 - How to create, run, terminate processes?
- API = Application Programming Interface
 - = functions available to write user programs
- API provided by OS is a set of “system calls”
 - System call is a function call into OS code that runs at higher CPU privilege level
 - Sensitive operations (e.g., access to hardware) are allowed only at a higher privilege level
- Some “**blocking**” system calls cause the process to be blocked and context switched out (e.g., `read()` from disk), while others (e.g., `getpid()` to get PID) can return immediately

Portability of code across OS

- **POSIX API**: standard set of system calls (and some C library functions) available to user programs, defined for portability
 - Programs written using POSIX API can run on any POSIX compliant OS
 - Most modern OSes are POSIX compliant
 - Program may still need to be recompiled for different architectures
- Program language libraries hide the details of invoking system calls
 - The `printf` function in `libc` calls the `write` system call to write to screen
 - User programs usually do not need to worry about invoking system calls
- ABI (application binary interface) is the interface between machine code and underlying hardware: ISA, calling convention, ...

Process related system calls (in Unix)

- `fork()` creates a new child process
 - All processes are created by forking from a parent
 - OS starts `init` process after boot up, which forks other processes
 - The `init` process is ancestor of all processes, including shell/terminal
- `exec()` makes a process execute a given executable
- `exit()` terminates a process
- `wait()` causes a parent to block until child terminates
- Many variants of the above system calls exist in language libraries with different arguments

Process creation: fork

- Parent process calls “fork” system call to create (spawn) a new process
- New child process created with new PID
- Memory image of parent is copied into that of child
- Parent and child run different copies of same code

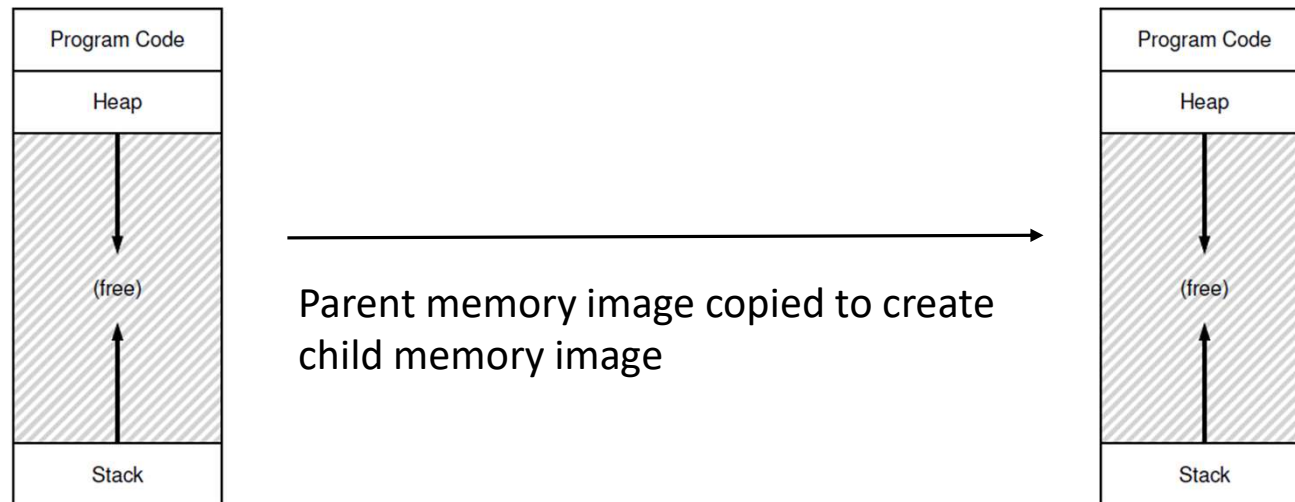


Image credit: OSTEP

What happens after fork?

- Parent and child resume execution in their copies of the code
- Child starts executing with a return value of 0 from fork
- Parent resumes executing with a return value equal to child PID
- Parent and child run independently
- Any changes in parent's data after fork does not impact child

```
int ret = fork()
if(ret == 0) {
    print "I am child"
}
else if(ret > 0) {
    print "I am parent"
}
```

Child resumes
here

Parent resumes
here

```
int ret = fork()
if(ret == 0) {
    print "I am child"
}
else if(ret > 0) {
    print "I am parent"
}
```

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int
6  main(int argc, char *argv[])
7  {
8      printf("hello world (pid:%d)\n", (int) getpid());
9      int rc = fork();
10     if (rc < 0) {          // fork failed; exit
11         fprintf(stderr, "fork failed\n");
12         exit(1);
13     } else if (rc == 0) { // child (new process)
14         printf("hello, I am child (pid:%d)\n", (int) getpid());
15     } else {               // parent goes down this path (main)
16         printf("hello, I am parent of %d (pid:%d)\n",
17               rc, (int) getpid());
18     }
19     return 0;
20 }

```

Figure 5.1: Calling `fork ()` (`p1.c`)

Example code with fork

- Parent and child run independently and print to screen
- Order of execution of parent and child can vary

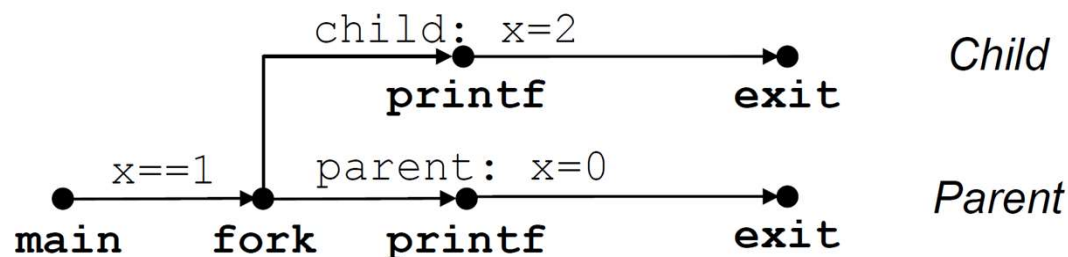
When you run this program (called `p1.c`), you'll see the following:

```
prompt> ./p1
hello world (pid:29146)
hello, I am parent of 29147 (pid:29146)
hello, I am child (pid:29147)
prompt>

prompt> ./p1
hello world (pid:29146)
hello, I am child (pid:29147)
hello, I am parent of 29147 (pid:29146)
prompt>
```

Example code with fork

- What values of x are printed?
- Parent and child both start with their own independent copies of variable x in their memory images
- Child increments its copy of x, prints 2
- Parent decrements its copy of x, prints 0

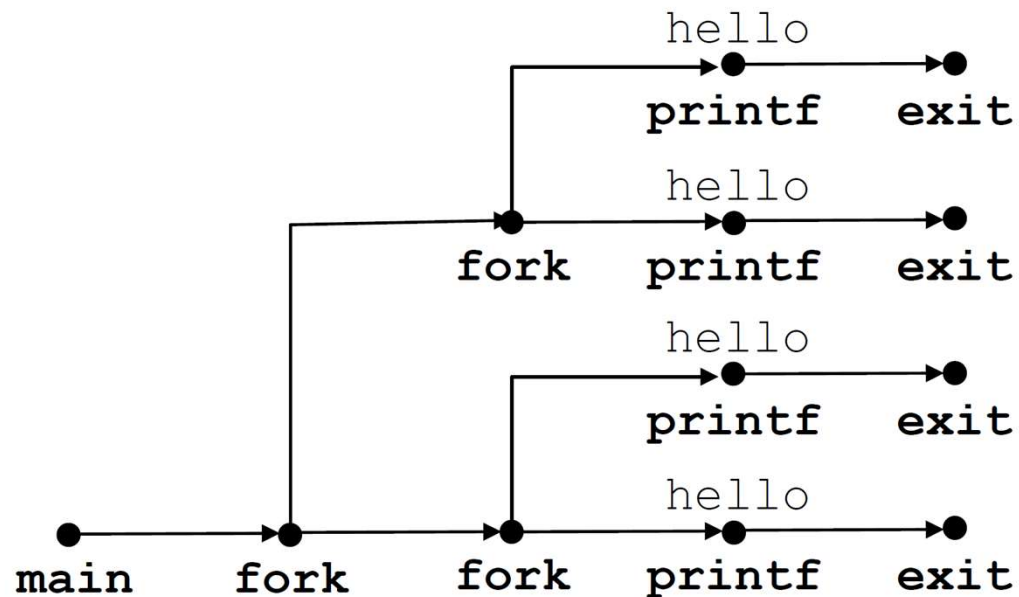


```
int ret = fork()
int x = 1
if(ret == 0) {
    print "I am child"
    x = x+1
    print x
}
else if(ret > 0) {
    print "I am parent"
    x = x -1
    print x
}
```

Example code with nested fork

- Total 4 processes (1 parent + 3 child)
- Hello printed 4 times

```
fork()  
fork()  
print hello  
exit
```



Exit system call

- When a process finishes execution, it calls `exit` system call to terminate
 - OS switches the process out and never runs it again
 - Exit is automatically called at end of main
- Exiting process cannot clean up its memory, and memory must be freed up by someone else (why? More on this later.)
- Terminated process exists in a `zombie` state
- How are zombies cleaned up?

Wait system call

- Parent calls `wait` system call to `reap` (clean up memory of) a zombie child
- `Wait` cleans up memory of one terminated child and returns in parent process
- If child still running, `wait` system call blocks parent until child exits
 - If child terminated already, `wait` reaps child and returns immediately
 - If parent with no child calls `wait`, it returns immediately without reaping anything

```
...  
int ret = fork()  
if(ret == 0) {  
    print "I am child"  
    exit()  
}  
else if(ret > 0) {  
    print "I am parent"  
    wait()  
}  
...
```

More on wait

- Wait system call variant `waitpid` reaps a specific child with a given PID, while regular `wait` reaps any terminated child
 - Read man pages for more details on arguments to `waitpid` and `wait`
- Wait system call “reaps” one dead child at a time (in any order)
 - Every fork must be followed by call to wait at some point in parent
- What if parent has exited while child is still running?
 - Child will continue to run, becomes orphan
 - Orphans adopted by init process, reaped by init when they terminate
- If parent forks children, but does not bother calling wait for long time, system memory fills up with zombies
 - Common programming error, exhausts system memory


```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int
7  main(int argc, char *argv[])
8  {
9      printf("hello world (pid:%d)\n", (int) getpid());
10     int rc = fork();
11     if (rc < 0) {          // fork failed; exit
12         fprintf(stderr, "fork failed\n");
13         exit(1);
14     } else if (rc == 0) { // child (new process)
15         printf("hello, I am child (pid:%d)\n", (int) getpid());
16     } else {              // parent goes down this path (main)
17         int wc = wait(NULL);
18         printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
19               rc, wc, (int) getpid());
20     }
21     return 0;
22 }

```

Figure 5.2: Calling `fork()` And `wait()` (p2.c)

Example code with fork and wait

- Order of printing of child and parent is deterministic now
- Why? Parent waits until child prints and exits, then prints

```
prompt> ./p2  
hello world (pid:29266)  
hello, I am child (pid:29267)  
hello, I am parent of 29267 (wc:29267) (pid:29266)  
prompt>
```

Exec system call

- Isn't it impractical to run the same code in all processes?
 - Sometimes parent creates child to do similar work..
 - .. but other times, child may want to run different code
- Child process uses “exec” system call to get a new “memory image”
 - Allows a process to switch to running different code
 - Exec system call takes another executable as argument
 - Memory image is reinitialized with new executable, new code, data, stack, heap, ...

```
...  
int ret = fork();  
if(ret == 0) {  
    exec("some_executable")  
}  
else if(ret > 0) {  
    print "I am parent"  
}  
...
```

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/wait.h>
6
7  int
8  main(int argc, char *argv[])
9  {
10     printf("hello world (pid:%d)\n", (int) getpid());
11     int rc = fork();
12     if (rc < 0) {           // fork failed; exit
13         fprintf(stderr, "fork failed\n");
14         exit(1);
15     } else if (rc == 0) { // child (new process)
16         printf("hello, I am child (pid:%d)\n", (int) getpid());
17         char *myargs[3];
18         myargs[0] = strdup("wc"); // program: "wc" (word count)
19         myargs[1] = strdup("p3.c"); // argument: file to count
20         myargs[2] = NULL;          // marks end of array
21         execvp(myargs[0], myargs); // runs word count
22         printf("this shouldn't print out");
23     } else {                // parent goes down this path (main)
24         int wc = wait(NULL);
25         printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
26             rc, wc, (int) getpid());
27     }
28     return 0;
29 }

```

Figure 5.3: Calling `fork()`, `wait()`, And `exec()` (p3.c)

Example code with exec

- Many variants of exec system call (execvp used in example), which differ in the arguments provided (read more in man pages)
- If exec successful, child gets new memory image, never comes back to the code in old memory image after exec
 - Print statement after exec doesn't run if exec successful
- If exec unsuccessful, reverts back to original memory image

```
prompt> ./p3
hello world (pid:29383)
hello, I am child (pid:29384)
      29      107      1030 p3.c
hello, I am parent of 29384 (wc:29384) (pid:29383)
prompt>
```

Shell / Terminal

- After bootup, the `init` process is first process created
- The `init` process spawns a shell like `bash`
- All future processes are created by forking from existing processes like `init` or shell
- Shell reads user command, forks a child, execs the command executable, waits for it to finish, and reads next command
- Common commands like `ls`, `echo`, `cat` are all readily available executables that are simply exec-ed by the shell

Example shell code

- How does the shell run a user command?
- Read input from user
- Shell process **forks** a child process
- Child process runs **exec** with “echo” program executable as argument, calls **exit** when done
- Parent shell calls **wait**, blocks till child terminates, reaps it, goes back for next input

```
$echo hello  
hello  
$
```

```
do forever {  
    input(command)  
  
    int ret = fork()  
  
    if(ret == 0) {  
        exec(command)  
    }  
    else {  
        wait()  
    }  
}
```

More on shell and commands

- Some commands already exist as programs written by OS developers and compiled into executables
 - Shell runs such command by simply calling `exec` in child process
- Some commands are implemented directly in shell code itself
- Think: why doesn't shell `exec` command directly? Why fork a child?
 - Do we want the shell program code to be rewritten fully?
- For “`cd`” command, “`chdir`” system call used to change directory of parent process itself, no child process is forked. Why?
 - Every process has a current working directory
 - Do we want to change directory of some child process or shell itself?


```
$sleep 10 &  
$
```

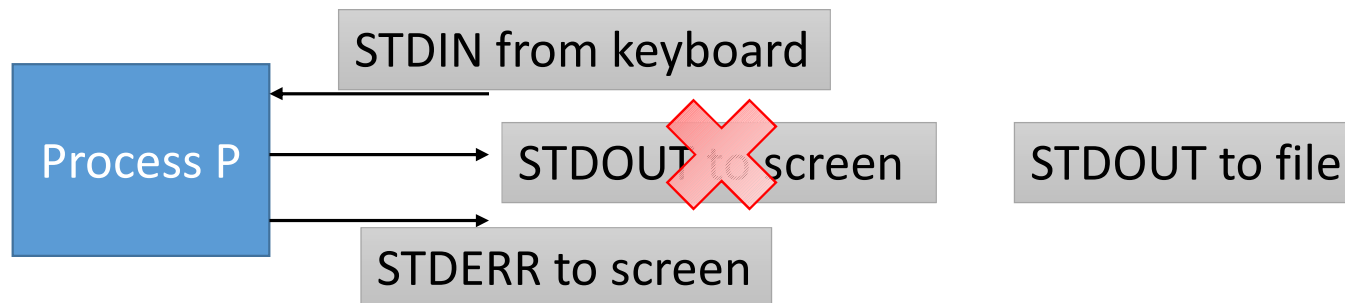
Foreground and background execution

- By default, user command runs in foreground, shell cannot accept next command until previous one finishes
- Background execution: when we type command followed by &
 - Shell starts child to run command, but does not wait for command to finish
- Background processes reaped at a later time by shell
 - When? Periodically? When next input is typed?
 - How? There is a way to invoke wait where parent is not blocked even if child has not exited (explore it on your own)
- It is also possible to run multiple commands in the foreground
 - One after the other serially (next command starts after previous finishes)
 - Or, all start at same time in parallel
 - Explore how such things can be done in the standard Linux shell

```
$ls > foo.txt  
$
```

I/O redirection

- Every process has some I/O channels (“files”) open, which can be accessed by file descriptors
 - STDIN, STDOUT, STDERR open by default for all processes
- Parent shell can manipulate these file descriptors of child before exec in order to do things like I/O redirection
- E.g., output redirection is done by closing the default STDOUT and opening a regular file in its place



```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <fcntl.h>
6  #include <sys/wait.h>
7
8  int
9  main(int argc, char *argv[])
10 {
11     int rc = fork();
12     if (rc < 0) {           // fork failed; exit
13         fprintf(stderr, "fork failed\n");
14         exit(1);
15     } else if (rc == 0) { // child: redirect standard output to a file
16         close(STDOUT_FILENO);
17         open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);
18
19         // now exec "wc"...
20         char *myargs[3];
21         myargs[0] = strdup("wc"); // program: "wc" (word count)
22         myargs[1] = strdup("p4.c"); // argument: file to count
23         myargs[2] = NULL;          // marks end of array
24         execvp(myargs[0], myargs); // runs word count
25     } else {                 // parent goes down this path (main)
26         int wc = wait(NULL);
27     }
28     return 0;
29 }

```

Here is the output of running the p4.c program:

```

prompt> ./p4
prompt> cat p4.output
      32      109      846 p4.c
prompt>

```

Open uses the first available file descriptor (STDOUT in this case)

Figure 5.4: All Of The Above With Redirection (p4.c)

Image credit: OSTEP

Signals

- Signal: a way to send notifications to processes
- Standard signals available in operating systems, each corresponding to a specific event, and with a specific signal number
 - Signal SIGINT sent to process by typing Ctrl+C, SIGSTP for Ctrl+D
 - Signal SIGCHLD sent to parent process when child terminates
 - SIGTERM and SIGKILL to terminate/kill processes
- System call `kill` can be used to send a signal from one process to other
 - Kill system call can send all signals, not just SIGKILL
 - Some restrictions on who can send to whom for isolation and security
- Signals can also be generated by OS for a process, e.g., when it handles interrupt due to Ctrl+C keyboard event
- Kill command to send signals, e.g., “kill -9 <pid>” sends SIGKILL (#9)

Process groups

- When we type Ctrl+C on keyboard, which processes get the signal?
- Processes are organized into process groups, every process belongs by default to process group of its parent
- When signal is sent to a process, it is delivered to all processes in its process group by default
- Example: when we hit Ctrl+C on keyboard, signal sent to all processes in the foreground process group
- System call `setpgid` can be used to change process group of signals, to control signal distribution

Signal handling

- Signals to a process are queued up by OS and delivered when process is running
- Default behavior defined by OS for a process receiving a signal
 - Ignore some signals (e.g., SIGCHLD)
 - Terminate when some signals are received (e.g., SIGINT)
- User processes can define their own signal handler functions to be executed when a signal is received
 - Override default behavior defined for a signal
 - Some signals (e.g., SIGKILL) cannot be overridden
- Process jumps to signal handler, executes it, resumes if still alive

Examples: sending and catching signals

- Parent sends SIGKILL to child using kill system call
- Child runs in infinite loop until killed by parent

```
int pid = fork()
if(pid == 0) {
    while(1); //infinite loop
    //terminates on SIGKILL
}
//parent
kill(pid, SIGKILL)
```

- Default SIGINT handler overridden
- Process prints message before terminating on SIGINT

```
void sigint_handler(int sig) {
    print "caught signal"
    exit()
}
int main() {
    signal(SIGINT, sigint_handler)
    ...
}
```

Process management in xv6

Mythili Vutukuru
CSE, IIT Bombay

PCB in xv6: struct proc

```
2334 enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
2335
2336 // Per-process state
2337 struct proc {
2338     uint sz;                // Size of process memory (bytes)
2339     pde_t* pgdir;           // Page table
2340     char *kstack;           // Bottom of kernel stack for this process
2341     enum procstate state;    // Process state
2342     int pid;                // Process ID
2343     struct proc *parent;     // Parent process
2344     struct trapframe *tf;    // Trap frame for current syscall
2345     struct context *context; // swtch() here to run process
2346     void *chan;              // If non-zero, sleeping on chan
2347     int killed;              // If non-zero, have been killed
2348     struct file *ofile[NOFILE]; // Open files
2349     struct inode *cwd;       // Current directory
2350     char name[16];           // Process name (debugging)
2351 };
2352
```

struct proc: page table

- Every instruction or data item in the memory image of process (code/data, stack, heap, etc.) has an address
 - Virtual addresses, starting from 0
 - Actual physical addresses in memory can be different (all processes cannot store their first instruction at address 0)
- Page table of a process maintains a mapping between the virtual addresses and physical addresses
- Page table used to translate virtual addresses to physical addresses

struct proc: kernel stack

- Stack to store CPU context when process jumps to kernel mode from user mode, or when process is context switched out
 - Why separate stack? OS does not trust user stack
 - Separate area of memory in the kernel, not accessible by regular user code
 - Linked from struct proc of a process

struct proc: list of open files

- Array of pointers to open files
 - When user opens a file, a new entry is created in this array, and the index of that entry is passed as a file descriptor to user
 - Subsequent read/write calls on a file use this file descriptor to refer to the file
 - First 3 files (array indices 0,1,2) open by default for every process: standard input, output and error
 - Subsequent files opened by a process will occupy later entries in the array

Process table (ptable) in xv6

- Ptable in xv6 is a fixed-size array of all processes
- Real kernels have dynamic-sized data structures

```
2409 struct {  
2410     struct spinlock lock;  
2411     struct proc proc[NPROC];  
2412 } ptable;
```

CPU scheduler in xv6

- The OS loops over all runnable processes in ptable, picks one, and sets it running on the CPU

```
2768    // Loop over process table looking for process to run.
2769    acquire(&ptable.lock);
2770    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
2771        if(p->state != RUNNABLE)
2772            continue;
2773
2774        // Switch to chosen process.  It is the process's job
2775        // to release ptable.lock and then reacquire it
2776        // before jumping back to us.
2777        c->proc = p;
2778        switchvm(p);
2779        p->state = RUNNING;
```

xv6 system calls

- In xv6, as in other systems, system calls are made by user library functions
 - User code invokes library function only
- System calls available to user programs are defined in user library header “user.h”
 - Equivalent to C library headers (xv6 doesn't use standard C library)

```
struct stat;
struct rtcdate;

// system calls
int fork(void);
int exit(void) __attribute__((noreturn));
int wait(void);
int pipe(int*);
int write(int, const void*, int);
int read(int, void*, int);
int close(int);
int kill(int);
int exec(char*, char**);
int open(const char*, int);
int mknod(const char*, short, short);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int getpid(void);
char* sbrk(int);
int sleep(int);
int uptime(void);
```

What happens on a system call?

- The user library makes the actual system call to invoke OS code
- NOT a regular function call to OS code as it involves CPU privilege level change
- User library invokes special “trap” instruction called “int” in x86 (see `usys.S`) to make system call
- The **trap (int) instruction** causes a jump to kernel code that handles the system call
 - More on trap instruction later

```
#include "syscall.h"
#include "traps.h"

#define SYSCALL(name) \
    .globl name; \
    name: \
        movl $SYS_ ## name, %eax; \
        int $T_SYSCALL; \
        ret

SYSCALL(fork)
SYSCALL(exit)
SYSCALL(wait)
```


xv6: fork system call implementation

```
2579 int
2580 fork(void)
2581 {
2582     int i, pid;
2583     struct proc *np;
2584     struct proc *curproc = myproc();
2585
2586     // Allocate process.
2587     if((np = allocproc()) == 0){
2588         return -1;
2589     }
2590
2591     // Copy process state from proc.
2592     if((np->pgdir = copyvm(curproc->pgdir, curproc->sz)) == 0){
2593         kfree(np->kstack);
2594         np->kstack = 0;
2595         np->state = UNUSED;
2596         return -1;
2597     }
2598     np->sz = curproc->sz;
2599     np->parent = curproc;
```

```
2600     *np->tf = *curproc->tf;
2601
2602     // Clear %eax so that fork returns 0 in the child.
2603     np->tf->eax = 0;
2604
2605     for(i = 0; i < NOFILE; i++)
2606         if(curproc->ofile[i])
2607             np->ofile[i] = filedup(curproc->ofile[i]);
2608     np->cwd = idup(curproc->cwd);
2609
2610     safestrcpy(np->name, curproc->name, sizeof(curproc->name));
2611
2612     pid = np->pid;
2613
2614     acquire(&ptable.lock);
2615
2616     np->state = RUNNABLE;
2617
2618     release(&ptable.lock);
2619
2620     return pid;
2621 }
```

xv6: fork system call explanation

- Parent process invokes fork to create new child
 - Allocates new process in ptable, get new PID for child
 - Variable “np” is pointer to newly allocated struct proc of child
 - Variable “currproc” is pointer to struct proc of parent
 - Copies information (memory, files, size, ...) from currproc to np
- Child process set to runnable, scheduler runs it at a later time
- Return value in parent is PID of child
- Return value in child is set to 0 (by changing child’s EAX register)

xv6: exit system call implementation

```
2626 void
2627 exit(void)
2628 {
2629     struct proc *curproc = myproc();
2630     struct proc *p;
2631     int fd;
2632
2633     if(curproc == initproc)
2634         panic("init exiting");
2635
2636     // Close all open files.
2637     for(fd = 0; fd < NOFILE; fd++){
2638         if(curproc->ofile[fd]){
2639             fileclose(curproc->ofile[fd]);
2640             curproc->ofile[fd] = 0;
2641         }
2642     }
2643
2644     begin_op();
2645     iput(curproc->cwd);
2646     end_op();
2647     curproc->cwd = 0;
2648
2649     acquire(&ptable.lock);
2650     // Parent might be sleeping in wait().
2651     wakeup1(curproc->parent);
2652
2653     // Pass abandoned children to init.
2654     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
2655         if(p->parent == curproc){
2656             p->parent = initproc;
2657             if(p->state == ZOMBIE)
2658                 wakeup1(initproc);
2659         }
2660     }
2661
2662     // Jump into the scheduler, never to return.
2663     curproc->state = ZOMBIE;
2664     sched();
2665     panic("zombie exit");
2666 }
```

xv6: exit system call explanation

- Exiting process cleans up some state (e.g., close files)
- Wakes up parent process that may be waiting to reap
- Passes abandoned children (orphans) to init
- Marks itself as zombie and invokes scheduler, never gets scheduled again

xv6: wait system call implementation

```
2670 int
2671 wait(void)
2672 {
2673     struct proc *p;
2674     int havekids, pid;
2675     struct proc *curproc = myproc();
2676
2677     acquire(&ptable.lock);
2678     for(;;){
2679         // Scan through table looking for exited children.
2680         havekids = 0;
2681         for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
2682             if(p->parent != curproc)
2683                 continue;
2684             havekids = 1;
2685             if(p->state == ZOMBIE){
2686                 // Found one.
2687                 pid = p->pid;
2688                 kfree(p->kstack);
2689                 p->kstack = 0;
2690                 freevm(p->pgdir);
2691                 p->pid = 0;
2692                 p->parent = 0;
2693                 p->name[0] = 0;
2694                 p->killed = 0;
2695                 p->state = UNUSED;
2696                 release(&ptable.lock);
2697                 return pid;
2698             }
2699         }
2700         // No point waiting if we don't have any children.
2701         if(!havekids || curproc->killed){
2702             release(&ptable.lock);
2703             return -1;
2704         }
2705         // Wait for children to exit. (See wakeup1 call in proc_exit.)
2706         sleep(curproc, &ptable.lock);
2707     }
2708 }
2709 }
```

xv6: wait system call explanation

- Search for dead children in process table
- If dead child found, clean up memory of zombie, return its PID
- If no children, return -1, no need to wait
- If children exist but haven't terminated yet, wait until one dies

xv6: exec system implementation overview

- Copy new executable into memory from disk
- Create new stack, heap
- Copy command line arguments to new stack
- Switch process page table to use new memory image
- Process begins to run new code after system call ends
- Revert back to old memory image in case of any error

Trap handling

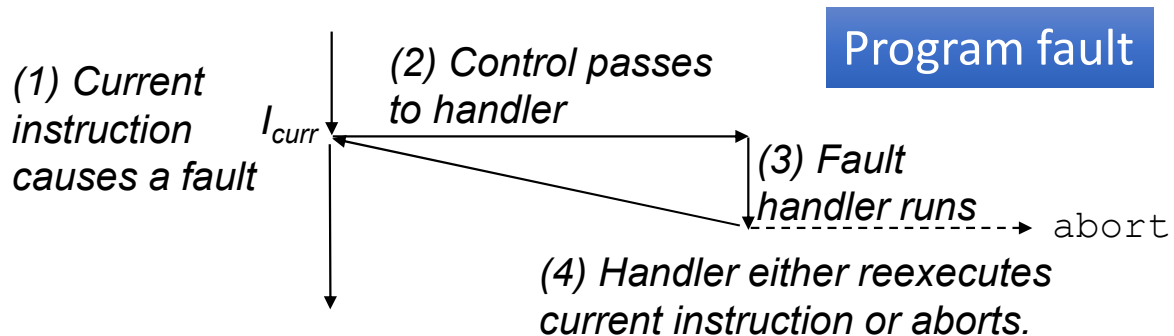
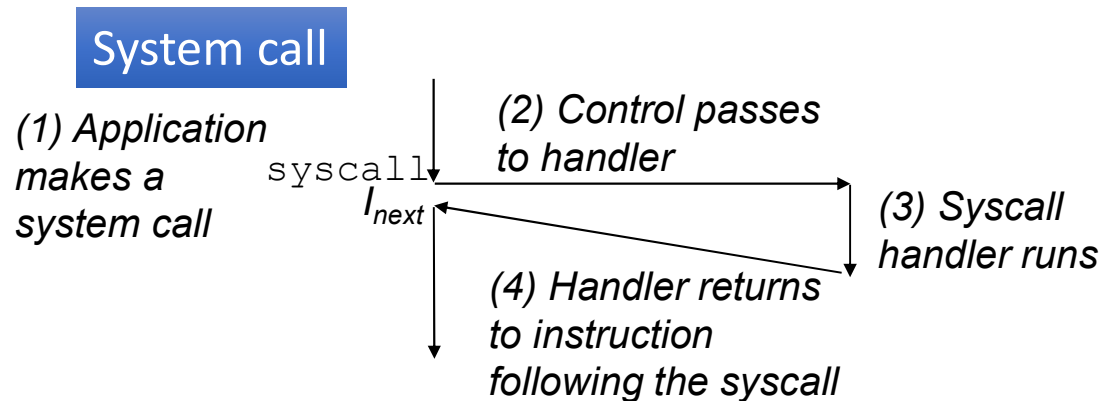
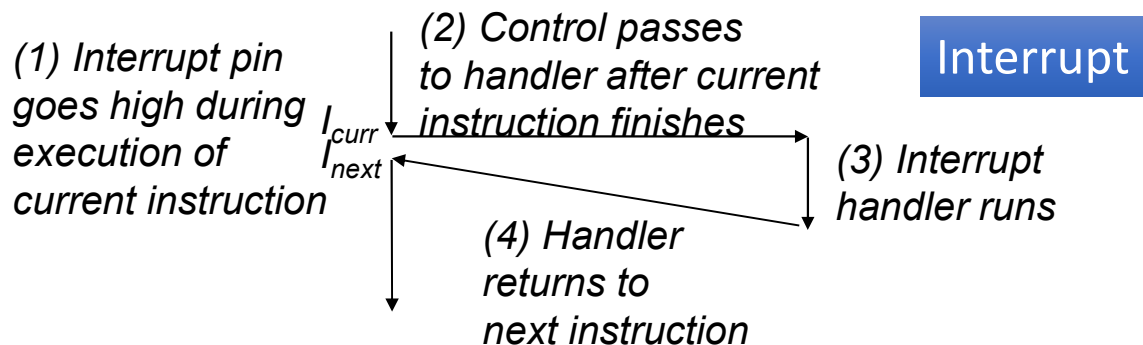
Mythili Vutukuru
CSE, IIT Bombay

Recap: OS runs processes

- OS manages multiple active processes **concurrently**
- What is a process?
 - **Memory image** in RAM = compiled code, data (compile-time, run-time)
 - **CPU context** (in CPU registers when running, else saved in PCB)
 - Other things like I/O connections, ..
- Processes created by **fork** from parent processes
- Periodically, **OS scheduler** loops over ready processes
 - Find a suitable process to run, save old process context, restore new context
- Once process is context switched in, OS is out of picture, CPU in user mode, runs user code directly
- When does the OS run again?

User mode vs. Kernel mode of a process

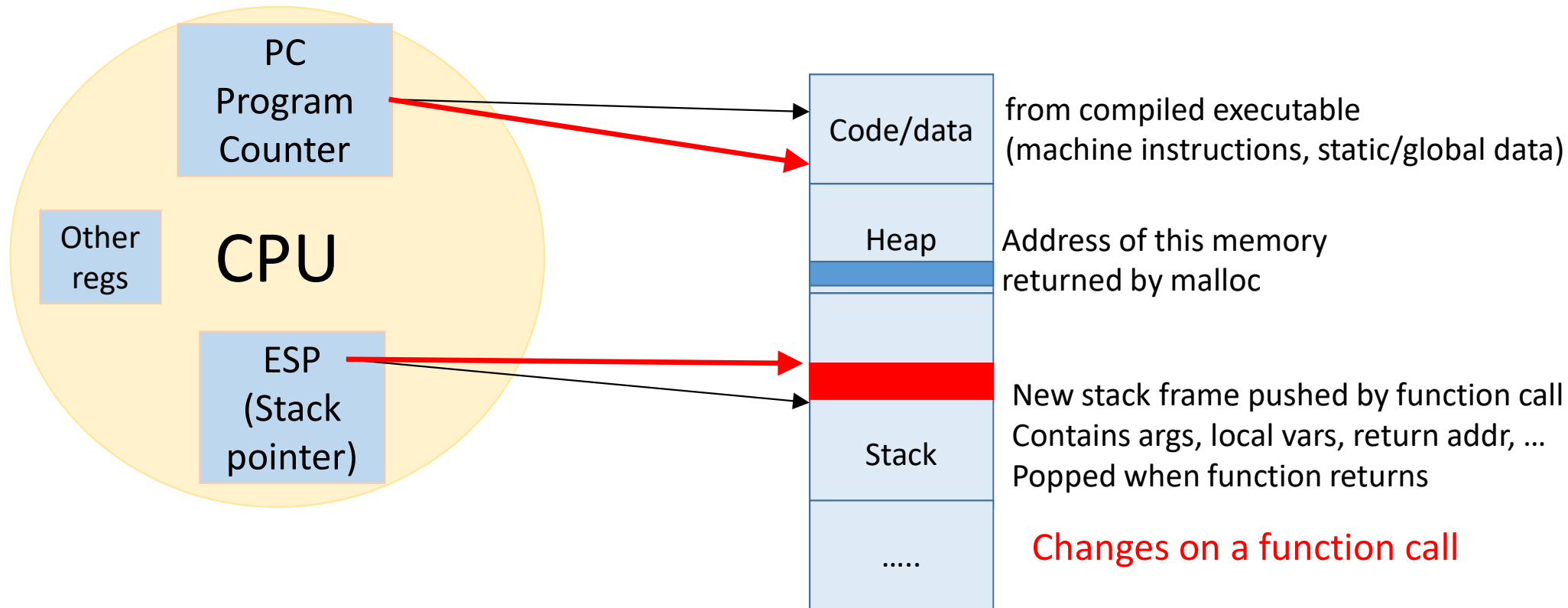
- CPU runs user code in **user mode (low privilege)** most of the time
- CPU switches to **kernel mode** execution when
 - Process makes system call, needs OS services
 - External device needs attention, raises interrupt
 - Some fault has happened during program execution
- All such events are called **traps**: CPU “traps” into OS code
 - CPU shifts to high privilege level (kernel mode), runs OS code to handle event
 - Later, CPU switches to low privilege level, back to user code in user mode
- Process P goes to kernel mode to run OS code, but it is still process P itself that is in running state
- **OS not a separate process, runs in kernel mode of existing processes**



Function call vs. system call

- What happens when a user program makes a function call?
 - Allocate memory on user stack for function arguments, local variables, ..
 - Push return address, PC jumps to function code
 - Push register context (to resume execution when function returns)
 - Execute function code
 - When returning from function, pop return address, pop register context
- System call also must
 - Use a stack to push register context
 - Save old PC, change PC to point to OS code to handle system call
 - Run system call, restore context back to user code

Understanding a function call



Located at some memory addresses in RAM

What is different for a system call?

- Changing PC in function call vs. system call
 - In function call, address of function code known in executable, can jump to function code directly using a CPU instruction (“call” in x86)
 - For system call, cannot trust user to jump to correct OS code (what if user jumps to inappropriate privileged code?)
- Saving register context on stack in function call vs. system call
 - In function call, register context is saved and restored from user stack
 - For system call, OS does not wish to use user stack (what if user has setup malicious values on the stack?)
- We require: a **secure stack**, a **secure way of jumping to OS code**

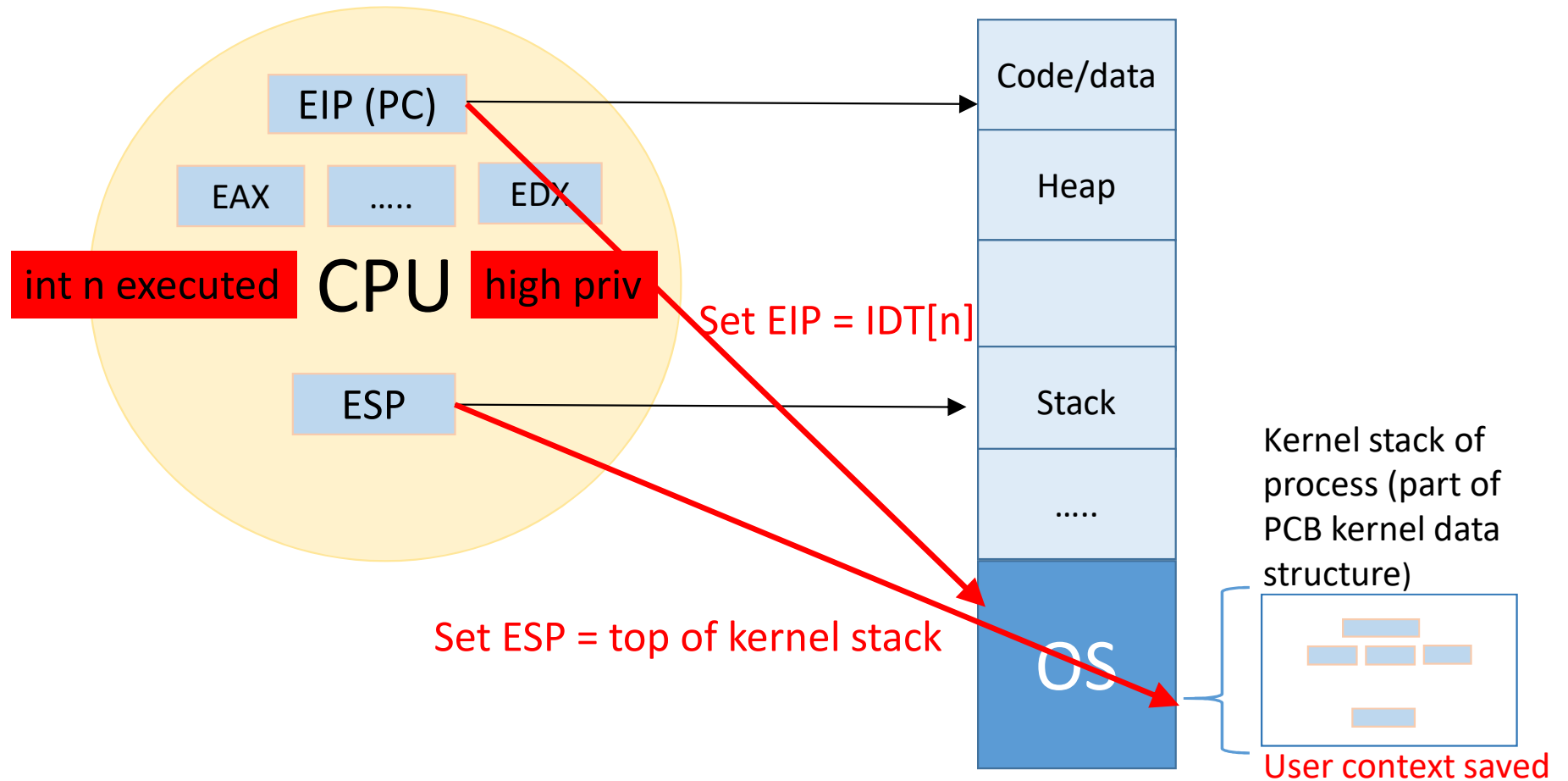
Kernel stack and IDT

- Every process uses a separate **kernel stack** for running kernel code
 - Part of **PCB** of process, in OS memory, not accessible in user mode
 - Used like user stack, but for kernel mode execution
 - Context pushed on kernel stack during system call, popped when done
- To set PC, CPU accesses **Interrupt Descriptor Table (IDT)**
 - Data structure with addresses of kernel code to jump to for events
 - Setup by OS during bootup, not accessible in user mode
 - CPU uses IDT to locate address of OS code to jump to
- Together: secure way of locating OS code, secure stack for OS to run

Hardware trap instruction

- When user code wants to make system call, it invokes special “trap instruction” with an argument
 - Example: “int n” in x86, argument “n” indicates type of trap (syscall, interrupt)
 - The value of “n” specifies index into IDT array, which OS function to jump to
- When CPU runs the trap instruction:
 - CPU moves to higher privilege level
 - CPU shifts stack pointer register to kernel stack of process
 - Register context is saved on kernel stack (part of PCB)
 - Address of OS code to jump to is obtained from IDT, PC points to OS code
 - OS code starts to run, on a secure stack

Trap handling



IDT lookup

- IDT configured by OS
- Base address of IDT stored in CPU register
- Upon trap, CPU looks up IDT to find address of interrupt handler

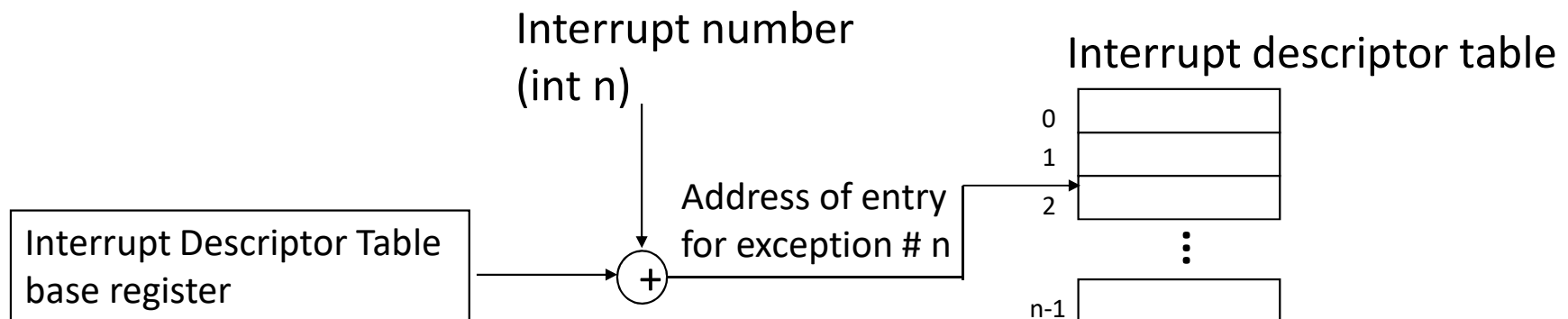
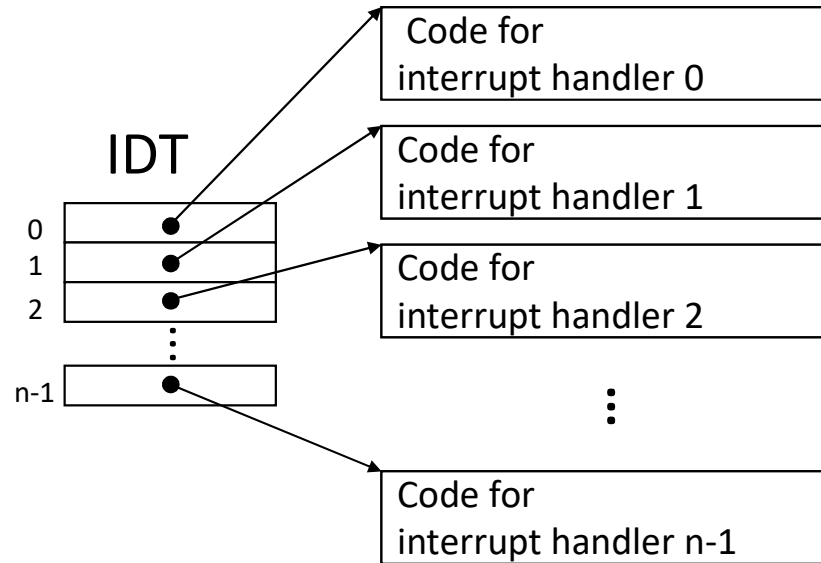


Image credit: CSAPP

Why trap instruction?

- Need a secure way of jumping to OS code to handle traps
 - User code cannot be trusted to jump to correct OS code
 - Only CPU can be trusted to handover control from user to OS securely
- Who calls **trap instruction**?
 - System call code in a language library (printf invokes system call via int n)
 - External hardware raises interrupt, causes CPU to execute “int n”
 - Argument “n” indicates whether system call /IRQ number of hardware device
- Across all cases, the mechanism is: save context on kernel stack, switch to OS address in IDT, run OS code to handle trap

Return from trap

- When OS is done handling syscall or interrupt, it calls a special instruction return-from-trap
 - Restore context of CPU registers from kernel stack
 - Change CPU privilege from kernel mode to user mode
 - Restore PC and jump to user code after trap
- User process unaware that it was suspended, resumes execution at the point it stopped before
- Always return to the same user process from kernel mode? No
 - Before returning to user mode, OS checks if it must switch to another process

Trap handling in xv6

- The following events in xv6 cause a user process to “trap” into the kernel
 - System calls (requests by user for OS services)
 - Interrupts (external device wants attention)
 - Program fault (illegal action by program)
- When above events happen, CPU executes the special “int” instruction
 - Example seen in `usys.S`, “int” invoked to handle system calls
 - For hardware interrupts, device sends a signal to CPU, and CPU executes int
- Trap instruction has a parameter (int `n`), indicating type of interrupt
 - E.g., `syscall` has a different value of `n` from keyboard interrupt
 - The value of “`n`” is used to index into IDT, get address of kernel code to run
- xv6 trap handling code saves register context, handles trap, returns

xv6 system calls

- In xv6, system calls available to user programs are defined in user library header “user.h”
 - Equivalent to C library headers (xv6 doesn't use standard C library)
- These system call functions invoked in user programs after including “user.h”
- The actual invoking of system call is done in usys.S

```
struct stat;
struct rtcdate;

// system calls
int fork(void);
int exit(void) __attribute__((noreturn));
int wait(void);
int pipe(int*);
int write(int, const void*, int);
int read(int, void*, int);
int close(int);
int kill(int);
int exec(char*, char**);
int open(const char*, int);
int mknod(const char*, short, short);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int getpid(void);
char* sbrk(int);
int sleep(int);
int uptime(void);
```

xv6 system calls

- The user library makes the actual system call to invoke OS code
- User library invokes trap instruction to make system call, code seen in usys.S
 - Defined using a macro
 - Move system call number to eax
 - Invoke int n where n is T_SYSCALL
- The trap (int) instruction causes a jump to kernel code that handles the system call

```
#include "syscall.h"
#include "traps.h"

#define SYSCALL(name) \
    .globl name; \
    name: \
        movl $SYS_ ## name, %eax; \
        int $T_SYSCALL; \
        ret

SYSCALL(fork)
SYSCALL(exit)
SYSCALL(wait)
```

Trap frame in xv6

- Trap frame is the structure pushed on kernel stack before trap handling, popped when returning from trap
- Contains various registers that are saved on kernel stack before trap handling
- The “int n” instruction pushes a few registers (old PC, old SP etc.) and jumps to kernel code to handle trap
- The kernel code that is run next will push remaining registers on kernel stack, and then proceed to handle the trap
- Think: why are EIP, ESP pushed by hardware and not by kernel code?

```
0600 // Layout of the trap frame built on the stack by the
0601 // hardware and by trapasm.S, and passed to trap().
0602 struct trapframe {
0603     // registers as pushed by pusha
0604     uint edi;
0605     uint esi;
0606     uint ebp;
0607     uint oesp;      // useless & ignored
0608     uint ebx;
0609     uint edx;
0610     uint ecx;
0611     uint eax;
0612
0613     // rest of trap frame
0614     ushort gs;
0615     ushort padding1;
0616     ushort fs;
0617     ushort padding2;
0618     ushort es;
0619     ushort padding3;
0620     ushort ds;
0621     ushort padding4;
0622     uint trapno;
0623
0624     // below here defined by x86 hardware
0625     uint err;
0626     uint eip;
0627     ushort cs;
0628     ushort padding5;
0629     uint eflags;
0630
0631     // below here only when crossing rings, such as from user to kernel
0632     uint esp;
0633     ushort ss;
0634     ushort padding6;
0635 };
```


xv6 kernel trap handler

- IDT entries for all interrupts will set eip to point to the kernel trap handler “alltraps”
- Alltraps assembly code pushes remaining registers to complete trapframe on kernel stack
- Invokes C trap handling function named “trap”
 - Push pointer to trapframe (current top of stack, esp) as argument to the C function

```
3300 #include "mmu.h"
3301
3302 # vectors.S sends all traps here.
3303 .globl alltraps
3304 alltraps:
3305 # Build trap frame.
3306     pushl %ds
3307     pushl %es
3308     pushl %fs
3309     pushl %gs
3310     pushal
3311
3312 # Set up data segments.
3313     movw $(SEG_KDATA<<3), %ax
3314     movw %ax, %ds
3315     movw %ax, %es
3316
3317 # Call trap(tf), where tf=%esp
3318     pushl %esp
3319     call trap
3320     addl $4, %esp
3321
3322 # Return falls through to trapret...
3323 .globl trapret
3324 trapret:
3325     popal
3326     popl %gs
3327     popl %fs
3328     popl %es
3329     popl %ds
3330     addl $0x8, %esp # trapno and errcode
3331     iret
```

C trap handler function in xv6

- C trap handler performs different actions based on kind of trap
- Different types of traps identified using value of “n” in “int n”
- For system call, “n” equal to a value T_SYSCALL (in usys.S), indicating this trap is a system call
 - Trap handler invokes common system call function
 - Looks at system call number stored in eax and calls the corresponding function (fork, exec, ...)
 - Return value of syscall stored in eax

```

3400 void
3401 trap(struct trapframe *tf)
3402 {
3403     if(tf->trapno == T_SYSCALL){
3404         if(myproc()->killed)
3405             exit();
3406         myproc()->tf = tf;
3407         syscall();
3408         if(myproc()->killed)
3409             exit();
3410         return;
3411     }

```

```

3700 void
3701 syscall(void)
3702 {
3703     int num;
3704     struct proc *curproc = myproc();
3705
3706     num = curproc->tf->eax;
3707     if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
3708         curproc->tf->eax = syscalls[num]();
3709     } else {
3710         cprintf("%d %s: unknown sys call %d\n",
3711             curproc->pid, curproc->name, num);
3712         curproc->tf->eax = -1;
3713     }
3714 }

```

- If interrupt from a device, corresponding driver code called
- Timer is special hardware interrupt, generated periodically to trap to kernel

```

3413     switch(tf->trapno){
3414     case T_IRQ0 + IRQ_TIMER:
3415         if(cpuid() == 0){
3416             acquire(&tickslock);
3417             ticks++;
3418             wakeup(&ticks);
3419             release(&tickslock);
3420         }
3421         lapiceoi();
3422         break;
3423     case T_IRQ0 + IRQ_IDE:
3424         ideintr();
3425         lapiceoi();
3426         break;
3427     case T_IRQ0 + IRQ_IDE+1:
3428         // Bochs generates spurious IDE1 interrupts.
3429         break;
3430     case T_IRQ0 + IRQ_KBD:
3431         kbdintr();
3432         lapiceoi();
3433         break;

```

```
3471 // Force process to give up CPU on clock tick.
3472 // If interrupts were on while locks held, would need to check nlock.
3473 if(myproc() && myproc()->state == RUNNING &&
3474     tf->trapno == T_IRQ0+IRQ_TIMER)
3475     yield();
3476
```

```
2826 // Give up the CPU for one scheduling round.
2827 void
2828 yield(void)
2829 {
2830     acquire(&ptable.lock);
2831     myproc()->state = RUNNABLE;
2832     sched();
2833     release(&ptable.lock);
2834 }
```

- On timer interrupt, a process “yields” CPU to scheduler
- Ensures a process does not run for too long

Return from trap

- Assembly code “trapret”
- Pop all state from kernel stack
- Return-from-trap instruction “iret” does the opposite of int
 - Pop values pushed by “int”
 - Change back privilege level
- Execution of pre-trap code can resume

```
3300 #include "mmu.h"
3301
3302 # vectors.S sends all traps here.
3303 .globl alltraps
3304 alltraps:
3305 # Build trap frame.
3306 pushl %ds
3307 pushl %es
3308 pushl %fs
3309 pushl %gs
3310 pushal
3311
3312 # Set up data segments.
3313 movw $(SEG_KDATA<<3), %ax
3314 movw %ax, %ds
3315 movw %ax, %es
3316
3317 # Call trap(tf), where tf=%esp
3318 pushl %esp
3319 call trap
3320 addl $4, %esp
3321
3322 # Return falls through to trapret...
3323 .globl trapret
3324 trapret:
3325 popal
3326 popl %gs
3327 popl %fs
3328 popl %es
3329 popl %ds
3330 addl $0x8, %esp # trapno and errcode
3331 iret
```

xv6 trap handling: the complete story

- System calls, program faults, or hardware interrupts cause CPU to run “int n” instruction and “trap” to OS
- The trap instruction (int n) causes CPU to switch ESP to kernel stack, EIP to kernel trap handling code “alltraps”
- Pre-trap CPU state is saved on kernel stack in the trap frame by int instruction + alltraps code
- Alltraps assembly code calls C trap handling function
- C trap handler handles trap suitably and returns to trapret code
- Trapret pops register context and runs “iret” instruction to return from trap to user mode of process

Context switching

Mythili Vutukuru
CSE, IIT Bombay

Why switch between processes?

- Sometimes when OS is in kernel mode, it cannot return back to the same process that was running in user mode before
 - Process has exited or must be terminated (e.g., segfault)
 - Process has made a blocking system call
- Sometimes, the OS does not want to return back to the same process
 - The process has run for too long
 - Must timeshare CPU with other processes
- In such cases, OS performs a context switch from one process to another
 - Switch from kernel mode of one process to kernel mode of another
 - OS scheduler decides which process to run next and switches to it

OS scheduler

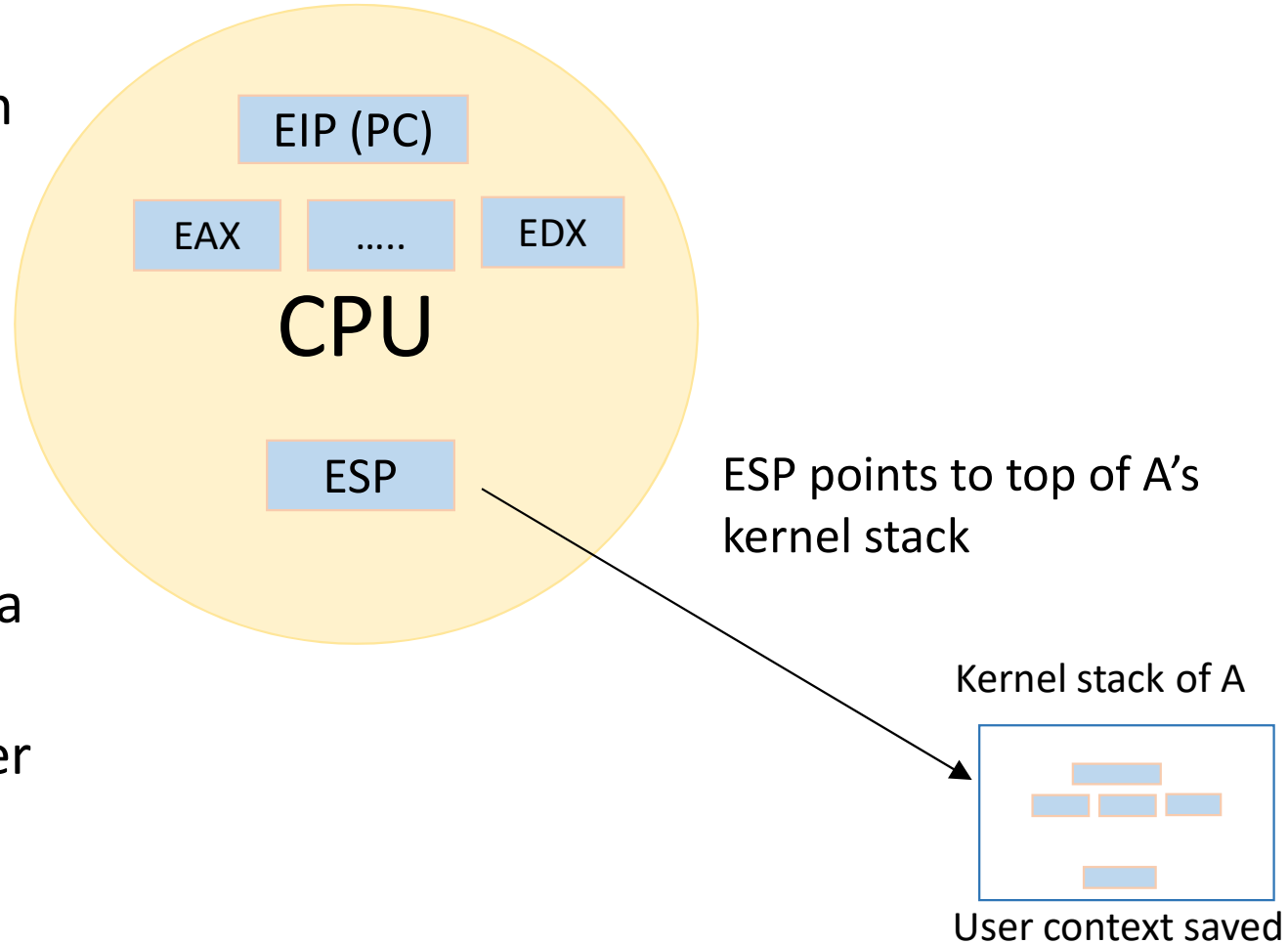
- OS maintains list of all active processes (PCBs) in a data structure
 - Processes added during fork, removed after clean up in wait
- OS **scheduler** is special code in the OS that periodically loops over this list and picks processes to run
- Basic outline of scheduler code
 - When invoked, **save context** of currently running process in its PCB
 - Loop over all **ready/runnable** processes and identify a process to run next
 - **Restore context** of new process from PCB and get it to run on CPU
 - Repeat this process as long as system is running

Scheduling and context switching

- OS scheduling involves two tasks
 - Policy to pick which process to run (next lecture)
 - Mechanism to switch to that process (this lecture)
- **Non preemptive** (cooperative) schedulers are polite
 - Switch only if process blocked or terminated
- **Preemptive** (non-cooperative) schedulers can switch even when process is ready to continue
 - CPU generates periodic timer interrupt
 - After servicing interrupt, OS checks if the current process has run for too long

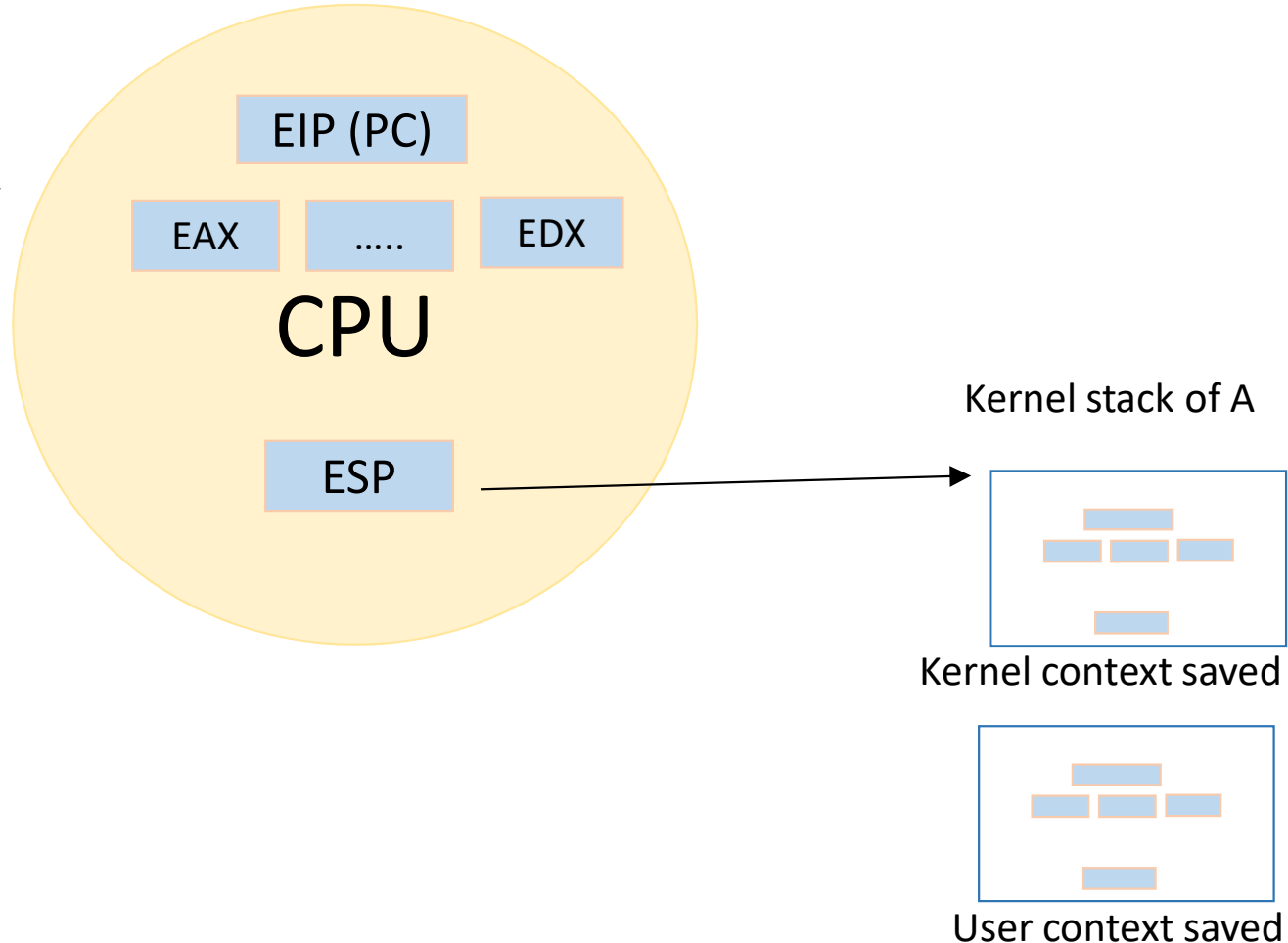
Mechanism of context switch (1)

- Process A has moved from user to kernel mode
- Kernel stack of A already has user register context
- After running for some time in kernel mode, A cannot run anymore (e.g., disk read initiated but data takes time to arrive)
- OS scheduler picks another process B to run next



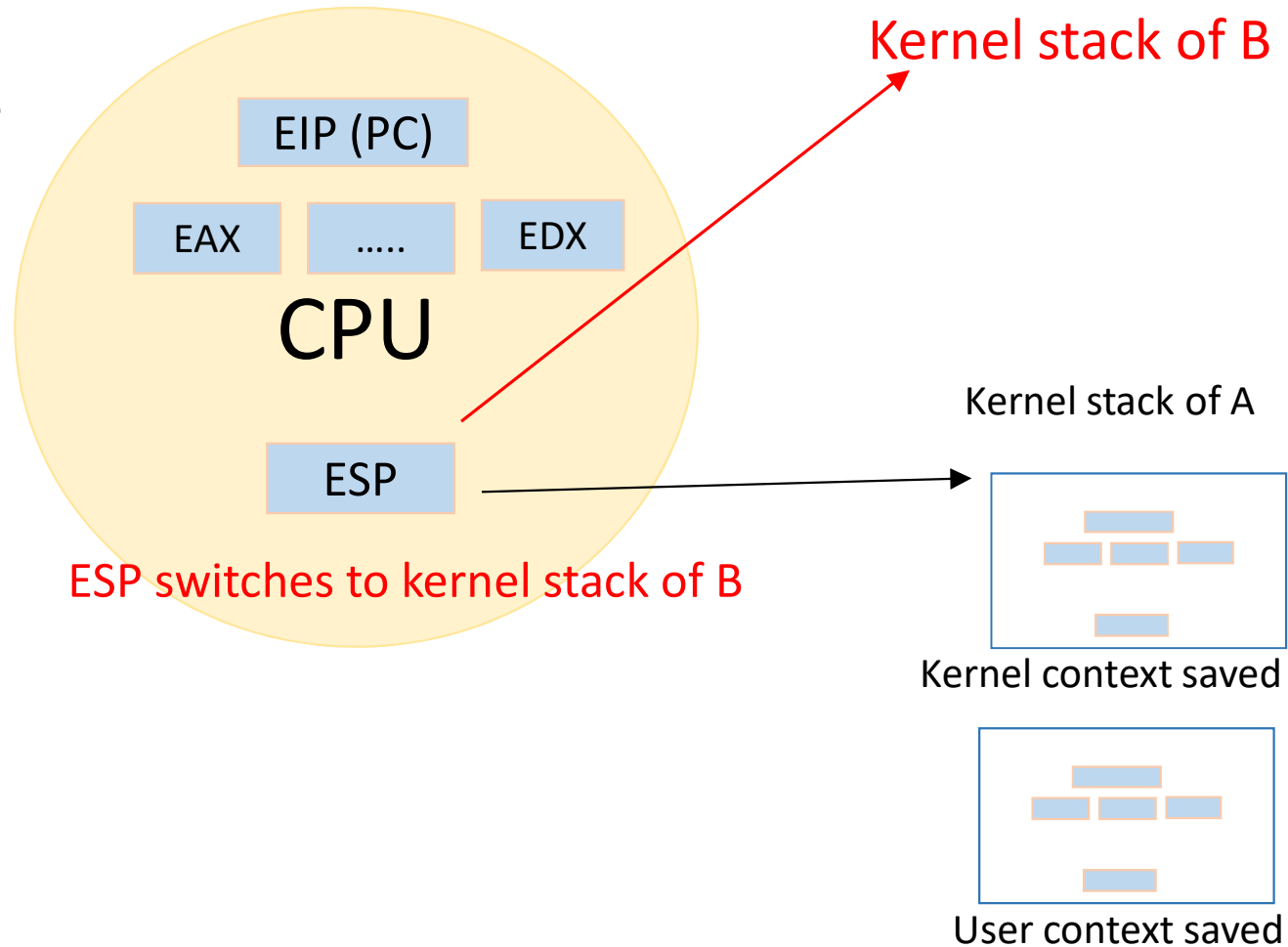
Mechanism of context switch (2)

- OS saves kernel context (PC, registers, kernel stack pointer) of A on kernel stack
- Why save context again?
- User context captures where execution stopped in user mode
- Kernel context captures where execution stopped in kernel mode



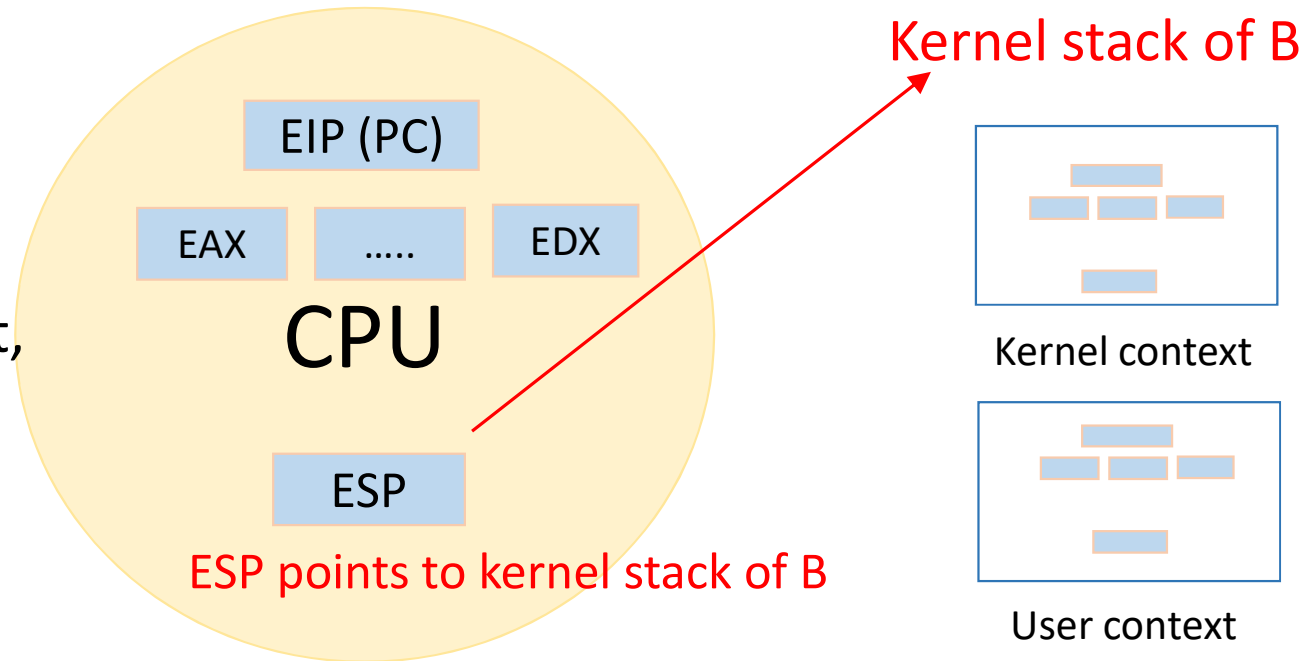
Mechanism of context switch (3)

- The actual moment of the context switch: OS switches ESP from kernel stack of A to kernel stack of next process B
- What will we find on the kernel stack of B?
 - Whatever the OS stored on it when it switched B out in the past



Mechanism of context switch (4)

- Kernel stack of B contains kernel context and user context of B
- OS restores kernel context, resumes execution in kernel mode of B, at the point it gave up CPU
- OS pops user context, resumes execution in user mode of B where it trapped into OS
- Context switch complete!



Understand saving and restoring context

- Context (PC and other CPU registers) saved on the kernel stack in two different scenarios
- When going from user mode to kernel mode, **user context** (e.g., which instruction of user code you stopped at) is saved on kernel stack by the trap instruction
 - Restored by return-from-trap when process goes to user mode
- During a context switch, **kernel context** (e.g., where you stopped in the OS code) is saved on the kernel stack by the context switching code
 - Restored when the process is ready to run and switched back in again

Context switching in xv6

- Every CPU has a scheduler thread (special OS process that runs scheduler code)
- Scheduler goes over list of processes and switches to one of the runnable ones
- The special function “swtch” performs the actual context switch from scheduler thread to user process

```
2757 void
2758 scheduler(void)
2759 {
2760     struct proc *p;
2761     struct cpu *c = mycpu();
2762     c->proc = 0;
2763
2764     for(;;){
2765         // Enable interrupts on this processor.
2766         sti();
2767
2768         // Loop over process table looking for process to run.
2769         acquire(&ptable.lock);
2770         for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
2771             if(p->state != RUNNABLE)
2772                 continue;
2773
2774             // Switch to chosen process. It is the process's job
2775             // to release ptable.lock and then reacquire it
2776             // before jumping back to us.
2777             c->proc = p;
2778             switchvm(p);
2779             p->state = RUNNING;
2780             swtch(&(c->scheduler), p->context);
2781             switchkvm();
2782
2783             // Process is done running for now.
2784             // It should have changed its p->state before coming back.
2785             c->proc = 0;
2786         }
2787         release(&ptable.lock);
2788     }
2789 }
2790 }
2791 }
```

Scheduler and sched

- Scheduler switches to user process in “scheduler” function
- User process switches to scheduler thread in the “sched” function
- The function “swtch” called to context switch from user process to special scheduler process
- Scheduler process picks next process and the cycle repeats

```
2807 void
2808 sched(void)
2809 {
2810     int intena;
2811     struct proc *p = myproc();
2812
2813     if(!holding(&ptable.lock))
2814         panic("sched ptable.lock");
2815     if(mycpu()->ncli != 1)
2816         panic("sched locks");
2817     if(p->state == RUNNING)
2818         panic("sched running");
2819     if(readeflags() & FL_IF)
2820         panic("sched interruptible");
2821     intena = mycpu()->intena;
2822     swtch(&p->context, mycpu()->scheduler);
2823     mycpu()->intena = intena;
2824 }
```

When does user process call sched?

- **Yield**: Timer interrupt occurs, process has run enough, gives up CPU
- **Exit**: Process has called exit, sets itself as zombie, gives up CPU
- **Sleep**: Process has performed a blocking action, sets itself to sleep, gives up CPU

```
2826 // Give up the CPU for one scheduling round.
2827 void
2828 yield(void)
2829 {
2830     acquire(&ptable.lock);
2831     myproc()->state = RUNNABLE;
2832     sched();
2833     release(&ptable.lock);
2834 }
```

```
2662 // Jump into the scheduler, never to return.
2663 curproc->state = ZOMBIE;
2664 sched();
2665 panic("zombie exit");
2666 }
```

```
2894 // Go to sleep.
2895 p->chan = chan;
2896 p->state = SLEEPING;
2897
2898 sched();
2899
```

struct context

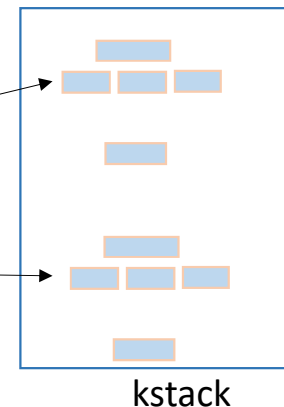
```
2326 struct context {  
2327     uint edi;  
2328     uint esi;  
2329     uint ebx;  
2330     uint ebp;  
2331     uint eip;  
2332 };
```

- In both scheduler and sched functions, the function “swtch” switches between two “contexts”
- Context structure: set of registers to be saved / restored when switching from one process to another
 - EIP where the process stopped execution, so that it can resume from same point when it is scheduled again in future
 - And a few more registers (why not all? more later)
- Context is pushed onto kernel stack, and pointer to the structure is stored in struct proc (p->context)

Context structure vs. trap frame in xv6

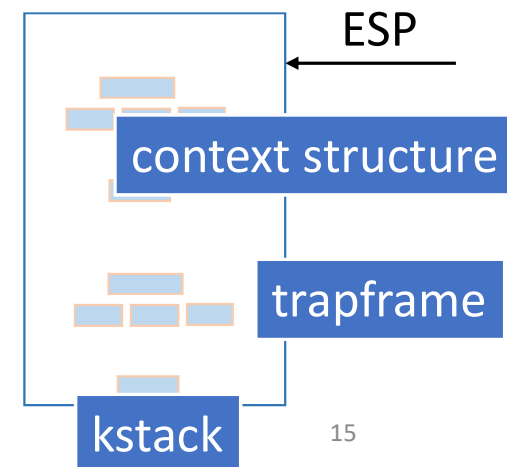
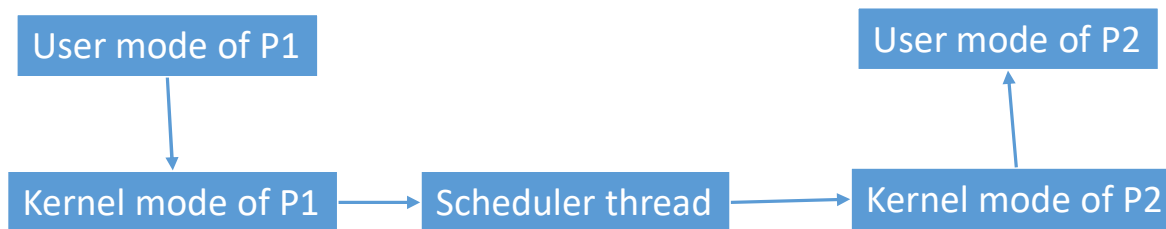
- Struct proc stores pointers to two structures on kernel stack
 - Trapframe is saved when CPU switches to kernel mode (e.g., PC in trapframe is PC value when syscall was made in user code)
 - Context structure is saved when process switches to another process (e.g., PC value when swtch function is invoked)
 - Both reside on kernel stack, struct proc has pointers to both
 - Example: Process has timer interrupt, saves trapframe on kstack, then context switch, saves context structure on kstack

```
2342 int pid; // Process ID
2343 struct proc *parent; // Parent process
2344 struct trapframe *tf; // Trap frame for current syscall
2345 struct context *context; // swtch() here to run process
```



Summary of context switching in xv6

- What happens during context switch from process P1 to P2?
 - P1 goes to kernel mode and gives up CPU (timer interrupt or exit or sleep)
 - P1 switches to CPU scheduler thread
 - Kernel stack of P1 has context structure and trap frame below it
 - Scheduler thread finds runnable process P2 and switches to it
 - P2 had given up CPU after saving context on its kernel stack in the past, so its kernel stack also has context structure and trap frame
 - P2 restores context structure, resumes in kernel mode
 - P2 returns from trap to user mode



What about new processes?

- The context switching code in xv6 restores context from kernel stack of a process and resumes execution where process stopped earlier
- But what if a process has never run before? Where will newly forked process resume execution when it is switched in by scheduler?
- Kernel stack of new processes (artificially created context structure and trap frame) setup in such a way that
 - EIP of function where it has to start is saved in context structure, so that it appears that process was switched out at the location where we want it to resume in kernel mode
 - Trap frame copied from parent, so it resumes in user mode just after fork
 - Process resumes execution in kernel mode, returns from trap to user space

xv6: fork system call implementation

```
2579 int
2580 fork(void)
2581 {
2582     int i, pid;
2583     struct proc *np;
2584     struct proc *curproc = myproc();
2585
2586     // Allocate process.
2587     if((np = allocproc()) == 0){
2588         return -1;
2589     }
2590
2591     // Copy process state from proc.
2592     if((np->pgdir = copyvm(curproc->pgdir, curproc->sz)) == 0){
2593         kfree(np->kstack);
2594         np->kstack = 0;
2595         np->state = UNUSED;
2596         return -1;
2597     }
2598     np->sz = curproc->sz;
2599     np->parent = curproc;
```

```
2600     *np->tf = *curproc->tf;
2601
2602     // Clear %eax so that fork returns 0 in the child.
2603     np->tf->eax = 0;
2604
2605     for(i = 0; i < NOFILE; i++)
2606         if(curproc->ofile[i])
2607             np->ofile[i] = filedup(curproc->ofile[i]);
2608     np->cwd = idup(curproc->cwd);
2609
2610     safestrcpy(np->name, curproc->name, sizeof(curproc->name));
2611
2612     pid = np->pid;
2613
2614     acquire(&ptable.lock);
2615
2616     np->state = RUNNABLE;
2617
2618     release(&ptable.lock);
2619
2620     return pid;
2621 }
```


allocproc (1)

- Find unused entry in ptable, mark is as embryo
 - Marked as runnable after process creation completes
- New PID allocated
- New memory allocated for kernel stack, stack pointer points to bottom of stack

```
2468 // Look in the process table for an UNUSED proc.
2469 // If found, change state to EMBRYO and initialize
2470 // state required to run in the kernel.
2471 // Otherwise return 0.
2472 static struct proc*
2473 allocproc(void)
2474 {
2475     struct proc *p;
2476     char *sp;
2477
2478     acquire(&ptable.lock);
2479
2480     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
2481         if(p->state == UNUSED)
2482             goto found;
2483
2484     release(&ptable.lock);
2485     return 0;
2486
2487 found:
2488     p->state = EMBRYO;
2489     p->pid = nextpid++;
2490
2491     release(&ptable.lock);
2492
2493     // Allocate kernel stack.
2494     if((p->kstack = kalloc()) == 0){
2495         p->state = UNUSED;
2496         return 0;
2497     }
2498     sp = p->kstack + KSTACKSIZE;
2499
```

allocproc (2)

- Leave space for trapframe (copied from parent)
- Push return address of “trapret”
- Push context structure, with eip pointing to function “forkret”
- Why? When new process scheduled, begins execution at forkret, then returns to trapret, then returns from trap to userspace
- Hand-crafted kernel stack to make it look like process had a trap and context switch
 - Scheduler can switch this process in like others

```
2500 // Leave room for trap frame.
2501 sp -= sizeof *p->tf;
2502 p->tf = (struct trapframe*)sp;
2503
2504 // Set up new context to start executing at forkret,
2505 // which returns to trapret.
2506 sp -= 4;
2507 *(uint*)sp = (uint)trapret;
2508
2509 sp -= sizeof *p->context;
2510 p->context = (struct context*)sp;
2511 memset(p->context, 0, sizeof *p->context);
2512 p->context->eip = (uint)forkret;
2513
2514 return p;
2515 }
```

Forking new processes: summary

- Fork creates new process (PCB, PID, kernel stack) via allocproc
- Parent memory and file descriptors copied
- Trap frame of child copied from that of parent
 - Result: child returns from trap to exact line of code as parent
 - Only return value of system call in eax is changed, so parent and child have different return values from fork
- State of new child set to runnable, so scheduler thread will context switch to child process sometime in future
- Parent returns normally from trap/system call
- Child runs later when scheduled (forkret, trapret) and returns to user space like parent process

swtch function

- Save registers in context structure on kernel stack of old process
- Switches ESP to context structure of new process
- Pops registers from new context structure
- CPU now has context of new process

```
3050 # Context switch
3051 #
3052 # void swtch(struct context **old, struct context *new);
3053 #
3054 # Save the current registers on the stack, creating
3055 # a struct context, and save its address in *old.
3056 # Switch stacks to new and pop previously-saved registers.
3057
3058 .globl swtch
3059 swtch:
3060     movl 4(%esp), %eax
3061     movl 8(%esp), %edx
3062
3063     # Save old callee-saved registers
3064     pushl %ebp
3065     pushl %ebx
3066     pushl %esi
3067     pushl %edi
3068
3069     # Switch stacks
3070     movl %esp, (%eax)
3071     movl %edx, %esp
3072
3073     # Load new callee-saved registers
3074     popl %edi
3075     popl %esi
3076     popl %ebx
3077     popl %ebp
3078     ret
```

Arguments to switch function

- Both CPU thread and process maintain a context structure pointer variable (struct context *)
- switch takes two arguments: address of old context pointer to switch from, new context pointer to switch to

- When invoked from scheduler: address of scheduler's context pointer, process context pointer

```
2781      switch(&(c->scheduler), p->context);
```

- When invoked from sched: address of process context pointer, scheduler context pointer

```
2822  switch(&p->context, mycpu()->scheduler);
```

- Understand why the first argument is address and second is not

Why save and restore only some registers?

- What is on the kernel stack when a process/thread has just invoked the switch? Caller save registers and return address (EIP)
- What does switch do?
 - Push remaining (callee save) registers on old kernel stack
 - Save pointer to this context in old process PCB
 - Switch ESP from old kernel stack to new kernel stack
 - ESP now points to saved context of new process
 - Pop callee-save registers from new stack
 - Return from function call (pops return address, caller save registers)

swtch function code explanation

- When swtch function call is made, old kernel stack has return address (eip) and arguments to swtch (address of old context pointer, new context pointer)
- Store address of old context pointer into eax
- Store value of new context pointer into edx
- Push callee save registers on kernel stack of old process
- Top of stack esp now points to complete context structure of old process
- Update old context pointer (eax) to point to updated context
- Switch stacks: Copy new context pointer from edx to esp
- Pop registers from new context structure
- Return from swtch in new process

CPU Scheduling Policies

Mythili Vutukuru
CSE, IIT Bombay

OS scheduler

- OS scheduler schedules process on CPU
 - One process at a time per core
 - Multiple processes can run in parallel on multiple cores
- **Scheduling policy**: which one of the ready/runnable processes should be run next on a given CPU core?
 - Mechanism of context switching (save context of old process in its kernel stack/PCB and restore context of new process) is independent of policy
- Simple scheduling policies have good theoretical guarantees, but not practical for real operating systems
 - Real-life schedulers are very complex, involve many heuristics

Preemptive vs. non preemptive schedulers

- When is the OS scheduler invoked to trigger a context switch?
 - Only when a process is in kernel mode for a trap, but not on every trap
- **Non-preemptive scheduler** performs only **voluntary** context switches
 - Process makes blocking system call
 - Process has exited or has been terminated
- **Preemptive scheduler** performs **involuntary** context switches also
 - Process can be context switched out even if process is still runnable/ready
 - OS can ensure that no process runs for too long on CPU, starving others
- Modern systems use preemptive schedulers
 - Process can be context switched out any time in its execution

Timer interrupts

- Scheduler (OS) can run only when process is in kernel mode
- What if process never goes to kernel mode? How to perform pre-emptive scheduling?
- Timer interrupts: special interrupts that go off periodically
- Used by OS to get back in control periodically
 - Increment clock ticks, do other book keeping
 - Run scheduling algorithm if required to do involuntary context switch of currently running process
- Hardware support in the form of timer interrupts essential to implementing pre-emptive scheduling policies

Goals of CPU scheduling policy

- Maximize **utilization**: efficient use of CPU hardware
- Minimize **completion time / turnaround time** of a process = time from process creation to completion
- Minimize **response time** of a process = time from process creation to first time it is run (important for interactive processes)
- **Fairness**: all processes get a fair share of CPU (account for priorities also)
- Low **overhead** of scheduling policy
 - Scheduler does not take too long to make a decision (even with large #processes)
 - Scheduler does not cause too many context switches (~1 microsecond to switch)

Simplest policy: First In First Out

- Newly created processes are put in a **FIFO queue**, scheduler runs them one after another from queue
- **Non-preemptive**: process allowed to run till it terminates or blocks
 - When process unblocks, the next run is separate “job”, added to queue again
 - That is, if a process comes back after I/O wait, it counts as a fresh CPU burst (**CPU burst** = the CPU time used by a process in a continuous stretch)
- Example schedule: P1 (1-5), P2 (6-8), P3 (9 to 10)

Process	CPU time needed (units)	Arrives at end of time unit	Execution time slots
P1	5	0	1-5
P2	3	1	6-8
P3	2	3	9-10

Problem with FIFO

- Example: three processes arrive at $t=0$ in the order A,B,C
- Problem: convoy effect (small processes get stuck behind long processes)
- Average turnaround times tend to be high

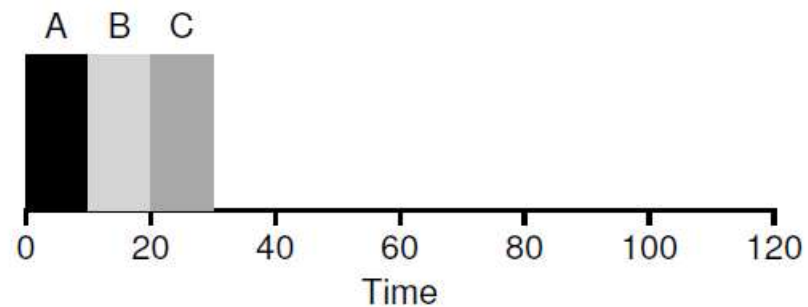


Figure 7.1: FIFO Simple Example

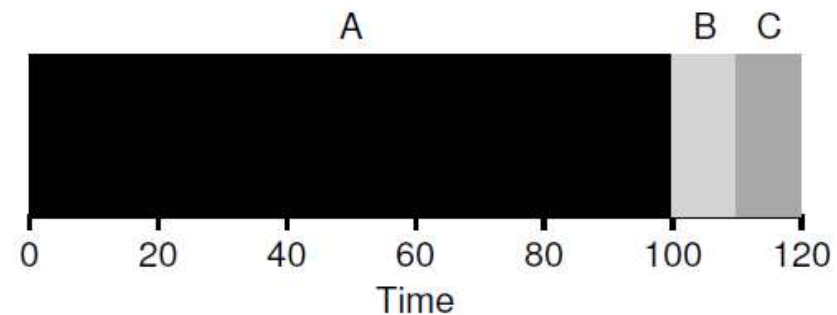


Figure 7.2: Why FIFO Is Not That Great

Shortest Job First (SJF)

- Assume **CPU burst** of a process (amount of time a process runs on CPU until termination/blocking) is known apriori
- Pick process with smallest CPU burst to run next, non-preemptive
 - Store PCBs in a heap-like data structure, extract process with min CPU burst
- Example schedule: P1 (1-5), P3 (6-7), P2 (8-10)

Process	CPU burst	Arrival time	Execution time slots
P1	5	0	1-5
P2	3	1	8-10
P3	2	3	6-7

Problem with SJF

- Provably optimal when all processes arrive together
 - Theoretically guaranteed to have the lowest average turnaround time across all policies (under certain assumptions)
- SJF is non-preemptive, so short jobs can still get stuck behind long ones.

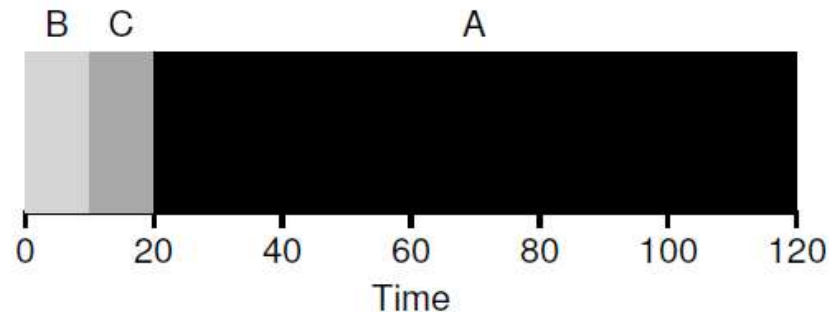


Figure 7.3: SJF Simple Example

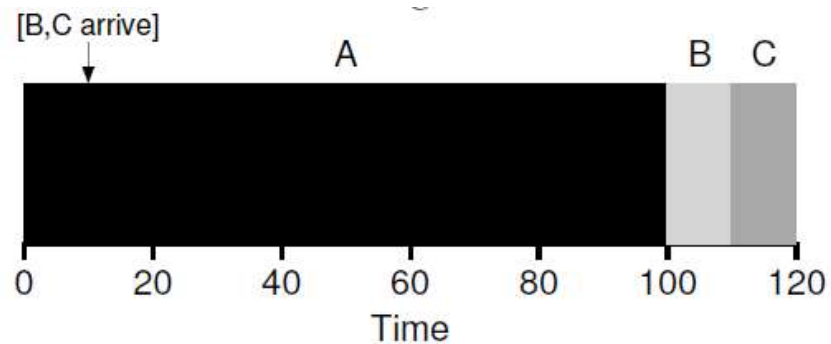


Figure 7.4: SJF With Late Arrivals From B and C

Shortest Remaining Time First (SRTF)

- Preemptive version of SJF
- A newly arrived process can preempt a running process, if CPU burst of new process is shorter than remaining time of running process
 - Avoids problem of short process getting stuck behind long one
- Example schedule: P1 runs for 1 unit, P2 (2-4), P3 (5-6), P1 (7-10)

Process	CPU burst	Arrival time	Execution time slots
P1	5	0	1, preempted , then 7-10
P2	3	1	2-4
P3	2	3	5-6

Round Robin (RR)

- Every process executes for a fixed quantum slice
 - Slice not too small (to amortize cost of context switch)
 - Slice not too big (to provide good responsiveness)
- Preemptive policy
 - Timer interrupt used to enforce periodic scheduling
- Good for response time, fairness
- Bad for turnaround time
- xv6 is a simple RR scheduler

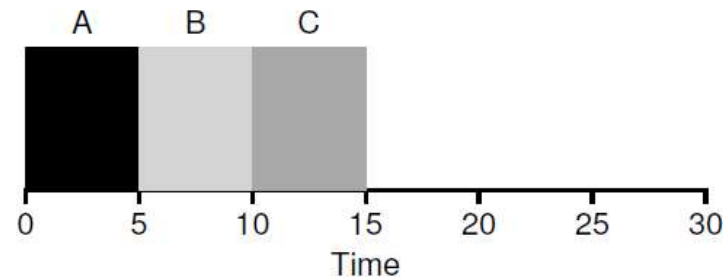


Figure 7.6: SJF Again (Bad for Response Time)

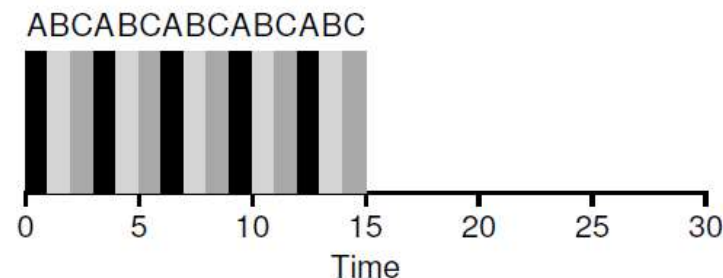


Figure 7.7: Round Robin (Good for Response Time)

Weighted Fair Queueing (WFQ)

- Round robin with different **weights** or **priorities** to processes
 - Decided by scheduler or can be set by users
 - Time slice will be in proportion to the weight or priority
- Real life schedulers may not be able to enforce time slice exactly
 - What if timer interrupt is not exactly aligned with time slice?
 - What if process blocks before its time slice?
- Practical modification: keep track of run time of process, schedule process that has used least fraction of its fair share
 - Compensate excess/deficit running time in future time slices
- **Linux scheduler** is a variant of weighted fair queueing
 - CFS (completely fair scheduler) uses red-black trees to keep track of the fair share run time of processes, schedules the one with least run time

Multi-level feedback queue (MLFQ)

- Another practical algorithm, with realistic assumptions
- What problem does it solve?
- Ideally, we want to optimize for turnaround time like SJF, give priority to shorter jobs (less time on CPU, more time on I/O)
 - But we do not know running time beforehand
- Also ensure low response time like round robin
- How to optimize for both turnaround time and response time without knowing job size apriori?

Overview of MLFQ

- Multiple queues, one for each priority level
 - Schedule processes from highest priority queue to lowest
 - Use round robin scheduling for processes within same priority level
- What is priority? Set by user but decays with age. Why?

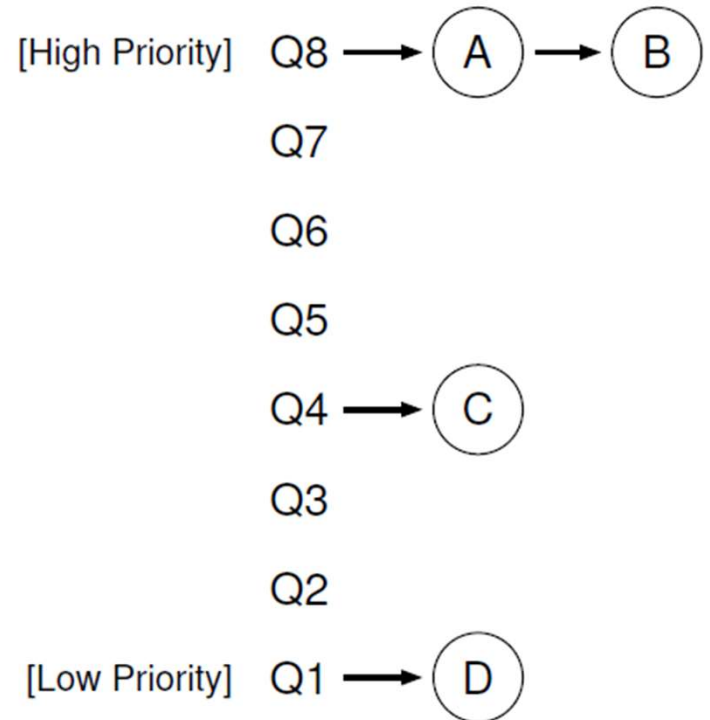


Figure 8.1: MLFQ Example

Priority decays with age

- Job that uses up its time slice at a priority level goes to lower priority
- Why? Ensures short I/O-bound processes get priority over long CPU-bound processes, but without knowing CPU burst apriori

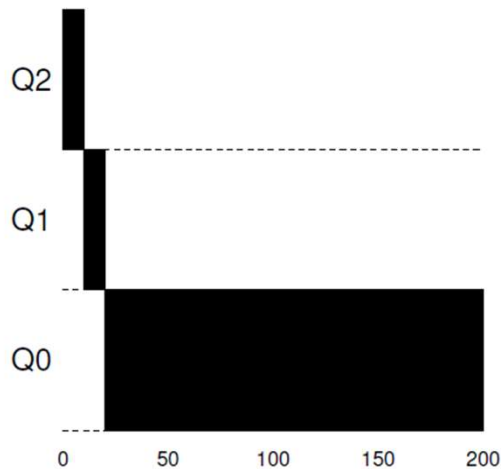


Figure 8.2: Long-running Job Over Time

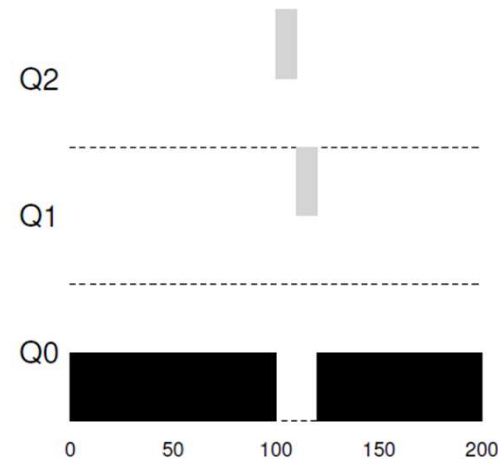


Figure 8.3: Along Came An Interactive Job

Image credit: OSTEP

Avoiding starvation

- Periodically reset all processes to highest priority level to avoid starvation of low priority or CPU-bound processes

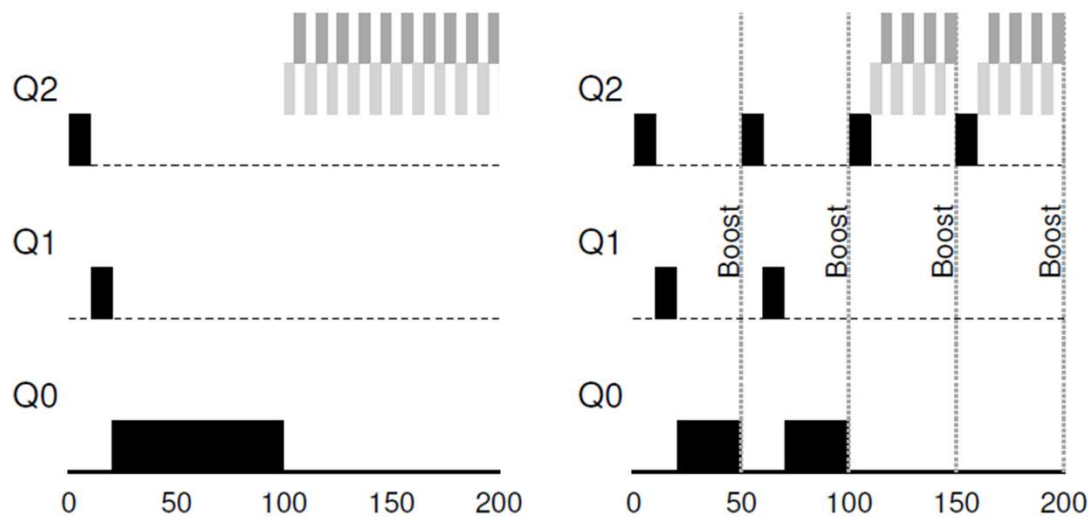


Figure 8.5: Without (Left) and With (Right) Priority Boost

Other considerations

- What if a job always gives up CPU just before its time slice ends? Do we keep it always at highest priority?
- We can consider total time spent by a job at a given priority level, not time in just one execution
- Time slice can vary with priority level
 - Longer time slice for lower priority long running jobs
- How to parameterize? How many priority levels? What is time slice?
 - No easy answers, must be tuned based on workload
 - Every OS comes with some default parameters

MLFQ summary

- **Rule 1:** If $\text{Priority}(A) > \text{Priority}(B)$, A runs (B doesn't).
- **Rule 2:** If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in RR.
- **Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue).
- **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).
- **Rule 5:** After some time period S , move all the jobs in the system to the topmost queue.

So, what have we achieved with MLFQ?

- Prioritize short, I/O bound processes over long, CPU bound processes
- Why? System works best when short processes don't have to wait for too long behind longer processes (that's why SJF is optimal!)
- How is this done? I/O bound processes will stay at highest priority level
- We do this without knowing the run times apriori, so practical to work in real life scenarios
- Real life schedulers cannot make assumptions on knowing run times etc, but still try to achieve good properties of theoretically optimal schedulers

Multicore scheduling

- Scheduling decision needs to be made separately for each core
- Do we bind a process to a particular CPU core always, or do we let a process run on any CPU core that is free?
- Ensuring a process runs on the same core as far as possible is better
 - Avoids coordination overheads across cores, better CPU cache performance
- But, we must be flexible too
 - If CPU core overloaded, some of its processes must move to another core
 - Load balancing across cores to ensure uniform workload distribution

Inter-process communication (IPC)

Mythili Vutukuru
CSE, IIT Bombay

Why inter-process communication

- Application logic in a single system is often distributed across multiple processes: why?
 - Different processes developed independently by different teams
 - Different programming languages and frameworks used for different tasks
- Processes in a system do not share any memory with each other by default, so how do they communicate information with each other?
 - Cannot share variables or data structures in programs across processes
 - Parent and child have identical but separate memory images after fork, changes made in one process and not seen by other
- **Inter-process communication (IPC)** mechanisms, available via operating system syscalls, allow processes to exchange information

Example: web application architecture

- Example: web applications typically composed of multiple processes
- **Web server** process handles HTTP (web) requests/responses
 - Written in a language like C/C++ for high performance
 - Returns responses for static content directly by reading files from disk
- Requests needing dynamic response are handled by **application server**
 - App server parses HTTP requests, generates HTTP response according to the business logic specified by user, sends response back to client via web server
 - Scripting languages may be used for easy text parsing and manipulation
- Application server stores/retrieves app data in a **database**
- Several web application frameworks available to build web applications having such architectures, e.g., Python Django, React etc.

IPC mechanisms

- **Unix domain sockets**: processes open sockets, send and receive messages to each other via socket system calls
- **Message queues**: sender posts a message to a mailbox, receiver retrieves message later on from mailbox
- **Pipes**: unidirectional communication channel between two processes
- **Shared memory**: same physical memory frame mapped into virtual address space of multiple processes in order to share memory
- **Signals**: specific messages via kill system call
- Different IPC mechanisms are useful in different scenarios

Sockets

- Sockets = abstraction to communicate between two processes
 - Each process opens socket, and pair of sockets can be connected
 - Client-server paradigm: one process opens socket first (**server**) and another process connects its socket to the first one (**client**)
 - One process writes a message into one socket, another process can read it, and vice versa (**bidirectional communication**)
 - Processes can be in same machine or on different machines
 - If processes on same machine, messages stored temporarily in OS memory before delivering to destination process
 - If processes on different machines, messages sent over network

Types of sockets (1)

- **Unix domain (local) sockets** are used to communicate between processes on the same machine
- **Internet sockets** are used to communicate between processes in different machines
- Local sockets identified by a pathname, Internet sockets identified by IP (Internet Protocol) address and port number
- Client and server sockets differentiated by who starts first and who connects later: server sockets started first on a well-known “address”, client process connects to server using the server address

Types of sockets (2)

- **Connection-based sockets**: one client socket and one server socket are explicitly connected to each other
 - After connection, the two sockets can only send and receive messages to each other
- **Connection-less sockets**: one socket can send/receive messages to/from multiple other sockets
 - Address of other endpoint can be mentioned on each message
- Type of socket (local or internet, connection-oriented or connection-less) is specified as arguments to system call that creates sockets
- In this course: Unix domain sockets, connection-less

Creating a socket

```
sockfd = socket(...)  
bind(sockfd, address)
```

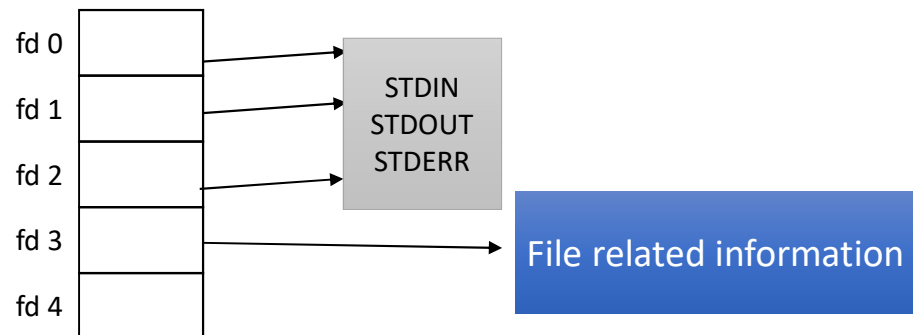
- System call “socket” used to create a socket
 - Takes type of socket as arguments
 - Returns **socket file descriptor** (similar to file descriptor when file is opened)
 - Used as handle for all future operations on the socket
- A socket can optionally **bind to an address** (pathname for Unix domain sockets or IP address/port number for Internet sockets) using “bind” system call
 - Server sockets bind to well known address, so that clients can connect
 - Client sockets need not bind, OS can assign temporary address
- Close system call closes a socket when done

The concept of file descriptors

```
fd = open("/home/foo/a.txt")
char buf[64]
read(fd, buf, 64)
buf[0] = ...
write(fd, buf, 64)
```

- Many IPC mechanisms like sockets return a file descriptor, which is simply an integer “handle” to access a file or socket or pipe
- PCB of process contains list of all open files in an array
- On file open, new entry is created in array, new index returned
- All future system calls (read, write) will be given the file descriptor as one of the arguments

File descriptor table
(list of open files of a process)
(part of PCB of process)



Data exchange using connection-less sockets

- Function `sendto` is used to send a message from one socket to another connection-less socket in another process
 - Arguments: socket fd, message to send, address of remote socket
- Function `recvfrom` is used to receive a message from a socket
 - Arguments: socket fd, message buffer into which received message is copied, socket address structure into which address of remote endpoint is filled
 - When a process receives a message on connection-less socket, it can find out the address of other endpoint, and use this address to reply back

Client

```
sockfd = socket(..)  
char message[1024]  
sendto(sockfd, message, server_sockaddr, ..)
```

Server

```
sockfd = socket(..)  
bind(sockfd, server_address)  
recvfrom(sockfd, message, client_sockaddr, ..)
```

Message queues

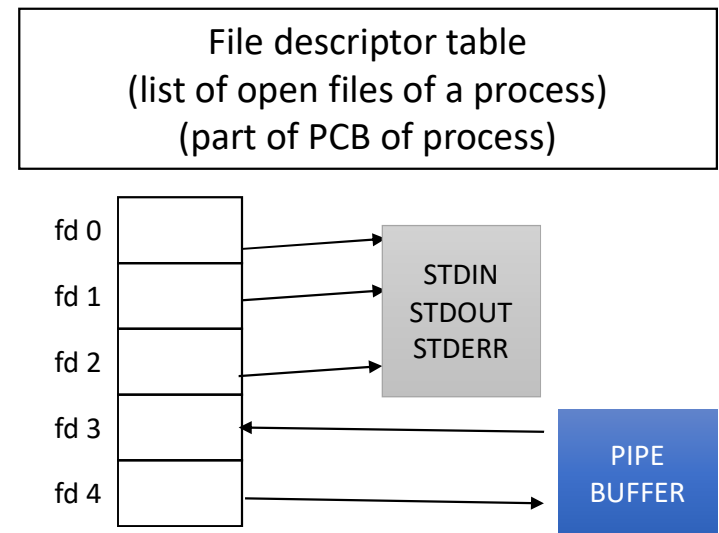
```
msgid = msgget(key, ...)  
msgsnd(msgid, message, ...)  
msgrcv(msgid, message, ...)
```

- Message queues used for exchanging messages between processes
 - Open connection to message queue identified by a “key”, get a handle
 - Sender opens connection to message queue, sends message
 - Receiver opens connection to message queue, retrieves message later on
 - Message buffered within message queue / mailbox until retrieved by receiver
- Example: IPC in web application using message queues
 - Web server posts dynamic HTTP requests into message queue
 - App server retrieves requests and processes them
 - App server posts responses into message queue for web server

Pipes

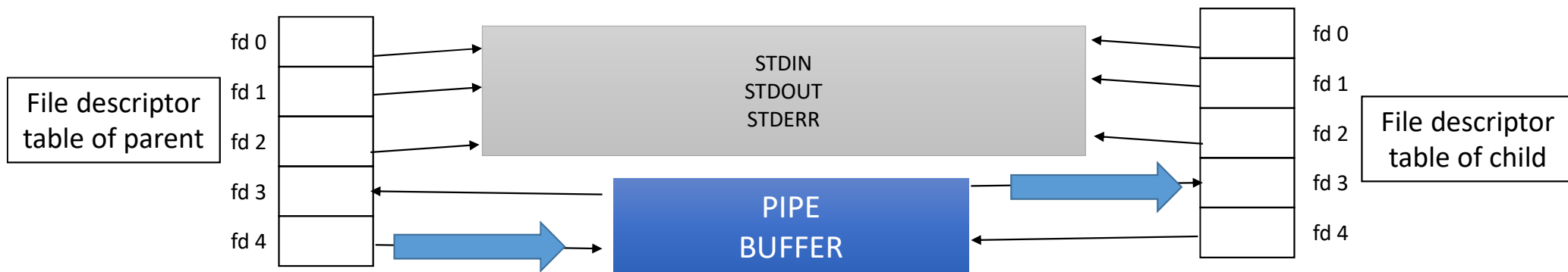
- Pipe is a **unidirectional FIFO channel** into which bytes are written at one end, read from other end
- System call “pipe” creates a pipe channel, with two file descriptors for endpoints, returns 2 integers
- One file descriptor used to write into pipe, one to read from pipe
- Data written into pipe is stored in a buffer of the pipe channel until read
- Bi-directional communication needs two pipes

```
int fd[2]
pipe(fd) //anonymous
read(fd[0], message, ..)
write(fd[1], message, ..)
```



Anonymous pipes

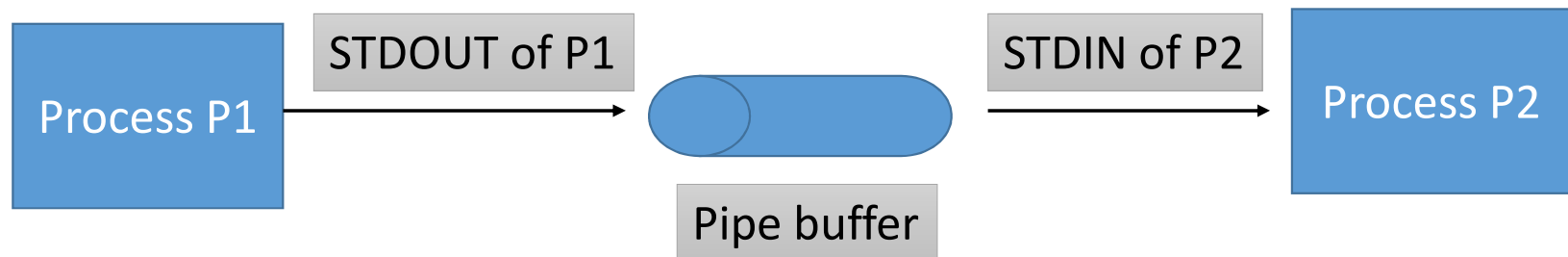
- **Anonymous pipes** (using pipe system call) only available for use within process and its children
- Open pipe before fork, so pipe file descriptors shared between parent and child, point to same pipe structure
- One of parent/child closes read end, other closes write end
- Pipe file descriptors used to read/write messages between parent/child



Pipes in shell commands

```
$cat foo.txt | grep something
```

- How does shell run commands with pipes (output of one command given as input to another command)?
- Shell opens a pipe, shared with child processes that run commands
- Shell duplicates stdout of first child to write end of pipe, read end of pipe to stdin of second process
- Processes must close file descriptors they are not using



Named pipes

- How to use pipes between unrelated processes? [Named pipes](#)
- Named pipes opened with a pathname, accessible across processes
- One process accesses read end of pipe, another opens write end
- Named pipe also provides uni-directional communication
- Writing to pipe with no reader open will throw an error

Reader

```
mkfifo(name, ..)  
fd = open(name, O_RDONLY)  
read(fd, message, ..)
```

Writer

```
fd = open(name, O_WRONLY)  
write(fd, message, ...)
```

Blocking vs. non-blocking IPC

- Same high level concept across sockets, pipes, message queues
 - Sender sends message, temporarily stored in some memory inside OS
 - Receiver retrieves message later on from temporary OS memory
- Send/receive system calls can **block**
 - Sender can block if temporary buffer is full
 - Receiver can block if temporary buffer is empty
- Possible to configure IPC to be **non-blocking** using syscalls
 - Send/receive will return with error instead of blocking

Shared memory

```
shmid = shmget(key, ..)  
char *data = shmat(shmid, ..)
```

- Processes in a system do not share any memory by default
 - Child process gets copy of parent memory image, modifies independently
- Shared memory: a way for two processes to share memory
 - Same memory appears in memory image of multiple processes
 - Shared memory **segment** identified by a **unique key**
 - Process can request to map or “attach” a specific shared memory segment into its memory image by using key
- Processes may need extra mechanisms for coordination besides shared memory
 - E.g., how does one process know when another process has modified shared memory?

Signals

- Signals are also a type of IPC
- But can only send limited amount of information
- Signals can be used with other IPC mechanisms
- Example: P1 places data in shared memory, signals P2 that data is ready
- User-defined signals can be used to avoid terminating process

Introduction to memory management

Mythili Vutukuru
CSE, IIT Bombay

Memory management in OS

- When program is run, memory image of process is created by OS
 - Code + compile-time data from executable loaded into main memory
 - Extra memory allocated for stack and heap
- CPU begins executing process
- How does CPU locate code/data in memory? Using memory addresses

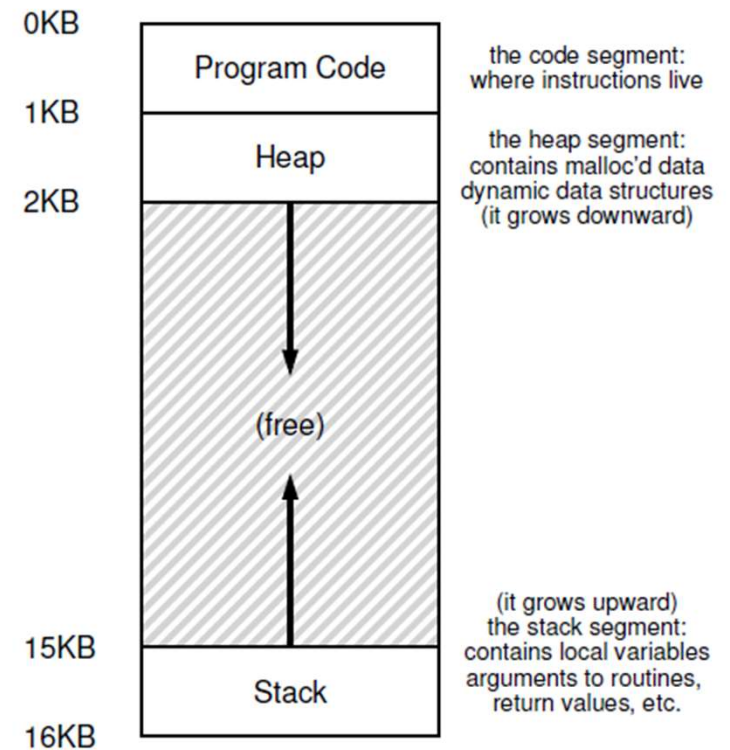


Image credit: OSTEP

Memory access using physical addressing

- CPU can use the actual physical address (byte # in RAM) of the instruction or data to fetch it
- Not very convenient or practical
 - How does compiler know the physical addresses it must assign to code/data in the executable at compile time?
 - What if we need to move the memory image to another location in RAM?
- Modern systems use the concept of virtual addressing

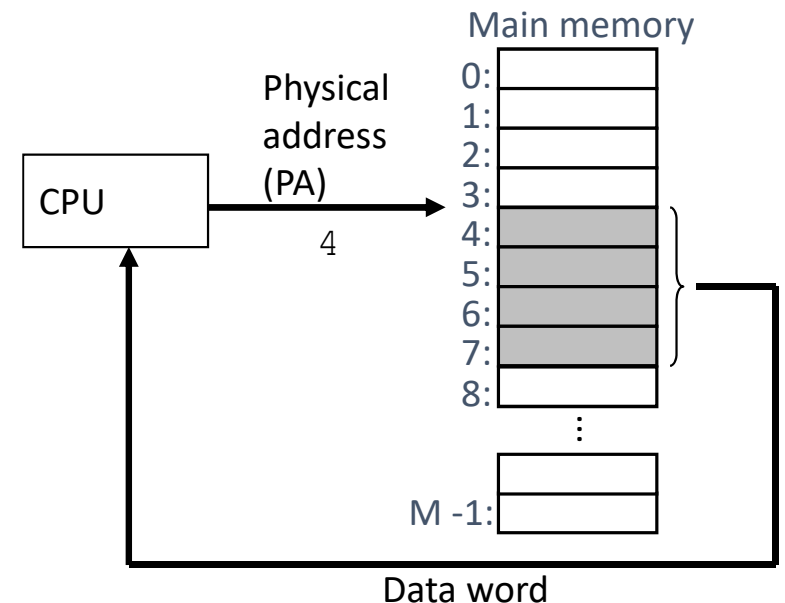


Image credit: CSAPP

Abstraction: (Virtual) Address Space

- Virtual address space: every process assumes it has access to a large space of memory from address 0 to a max value
- Max value depends on #bits available to address memory
 - $2^{32} = 4\text{GB}$ in 32-bit machines
- Virtual address space contains all code/data that a process can access
- Addresses in CPU registers and pointer variables = virtual addresses
- CPU issues loads and stores to virtual addresses

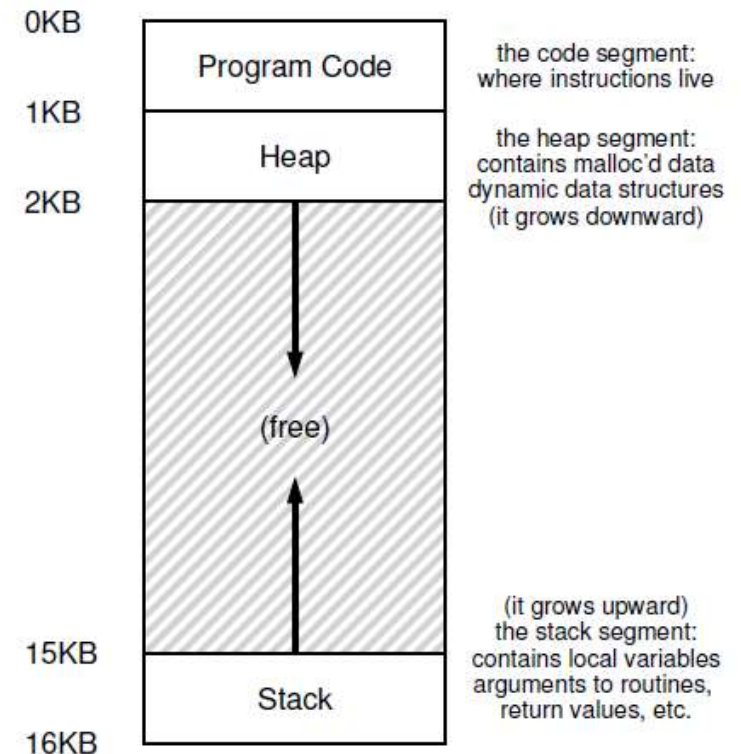


Figure 13.3: An Example Address Space

Image credit: OSTEP⁴

How is actual memory reached?

- Physical address space: actual physical memory in RAM
- Every virtual address space is mapped to physical addresses in memory by OS
- On every memory access, virtual address (VA) translated to physical address (PA) by special hardware called **Memory Management Unit (MMU)**
- OS allocates physical memory to a process, has translation information, provides it to MMU

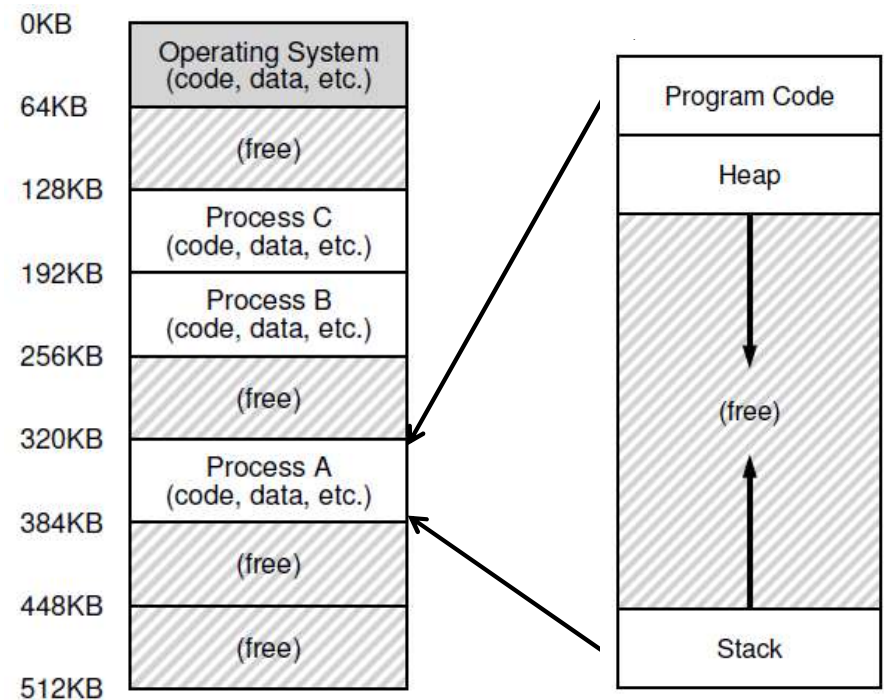
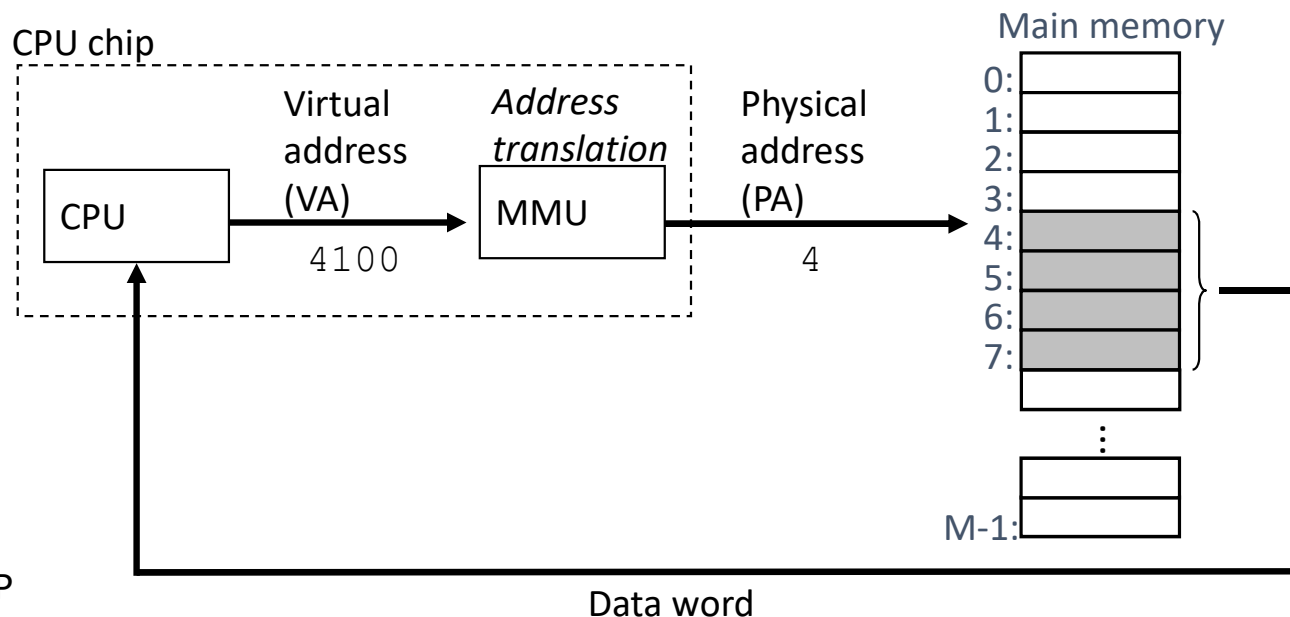


Image credit:⁵ OSTEP

Memory access using virtual addressing

- Code+data in memory image is assigned virtual addresses starting from 0 (by compiler and OS)
- CPU fetches code/data using virtual addresses, MMU translates to physical addresses (using info provided by OS)



Why virtual addresses?

- Because real view of memory is messy!
- Earlier, main memory had only code of one running process (and OS code)
- Now, multiple active processes timeshare CPU
- Memory allocation can be non-contiguous
- Need to hide this complexity from user
- Also, physical address not known at compile time

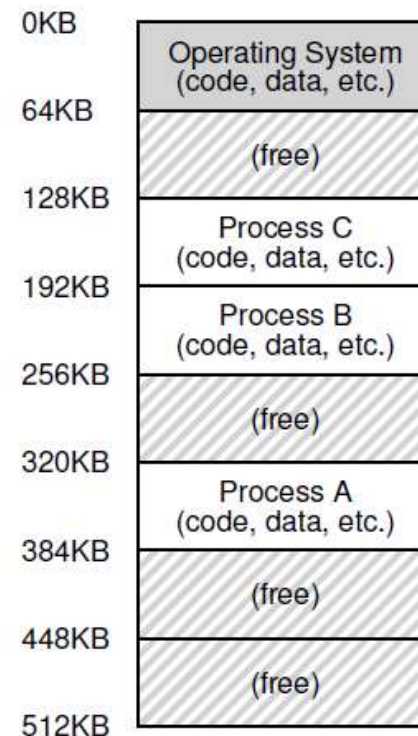


Figure 13.2: Three Processes: Sharing Memory

Base and bound

- How are virtual address spaces mapped to physical memory?
- Simplest form of memory management: **base and bound**
 - Place memory image $[0, N]$ contiguously starting at memory address base B
 - Virtual address X translated to physical address $B + X$
 - Access to virtual addresses beyond N will not be permitted
- OS provides base and bound to MMU for translation/error checking
- When CPU access a VA, MMU computes $PA = VA + \text{base}$, physical memory accessed with PA

A simple example

- Consider a simple C function

```
void func() {  
    int x = 3000;  
    x = x + 3;  
}
```

- It is compiled as follows

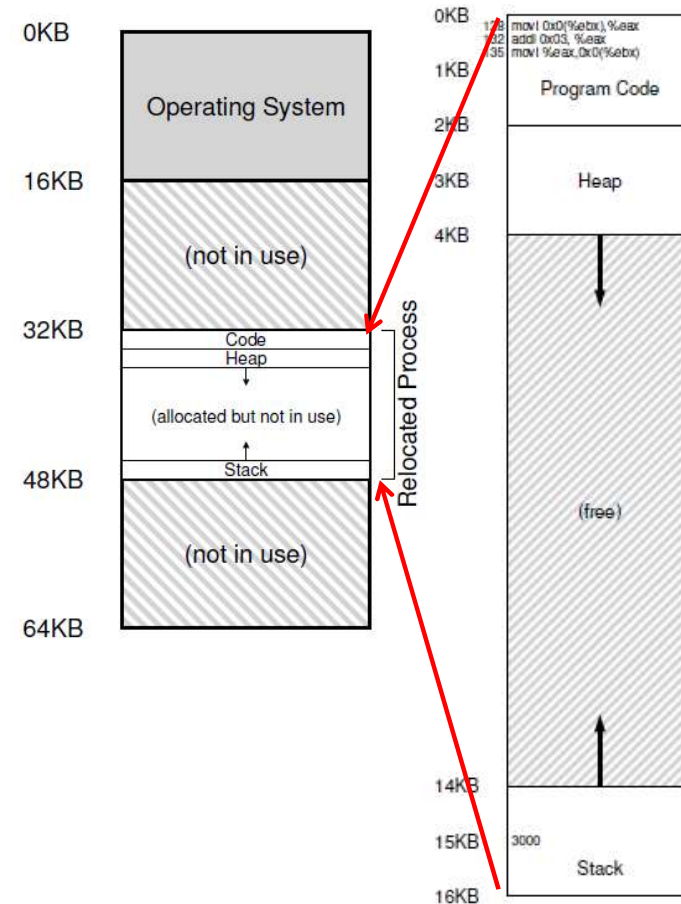
```
128: movl 0x0(%ebx), %eax    ;load 0+ebx into eax  
132: addl $0x03, %eax        ;add 3 to eax register  
135: movl %eax, 0x0(%ebx)    ;store eax back to mem
```

- Virtual address space is setup by OS during process creation



Example: address translation

- Suppose OS places entire memory image in one chunk, starting at physical address 32KB
- OS indicates base and bound to MMU
- MMU performs the following translation from VA to PA
- $PA = VA + \text{base}$
 - $VA = 128, PA = 32896 (32KB + 128)$
 - $VA = 1KB, PA = 33 KB$
 - $VA = 20KB? PA = ???$
- MMU raises trap when address out of bound



Role of OS vs MMU

- OS allocates memory, builds translation information of process
 - But OS does not do the actual address translation on every memory access
 - Why? Once user code starts running on CPU, OS is out of the picture (until a trap)
- When process is switched in, translation information is provided to MMU
- CPU runs process code, accesses code/data at virtual addresses
 - Virtual addresses translated to physical addresses by MMU
 - Actual physical memory is accessed using physical addresses
- MMU raises a trap if there is any error in the address translation
 - CPU executes trap instruction, OS code runs to handle the error
- OS gives new information to MMU on every context switch

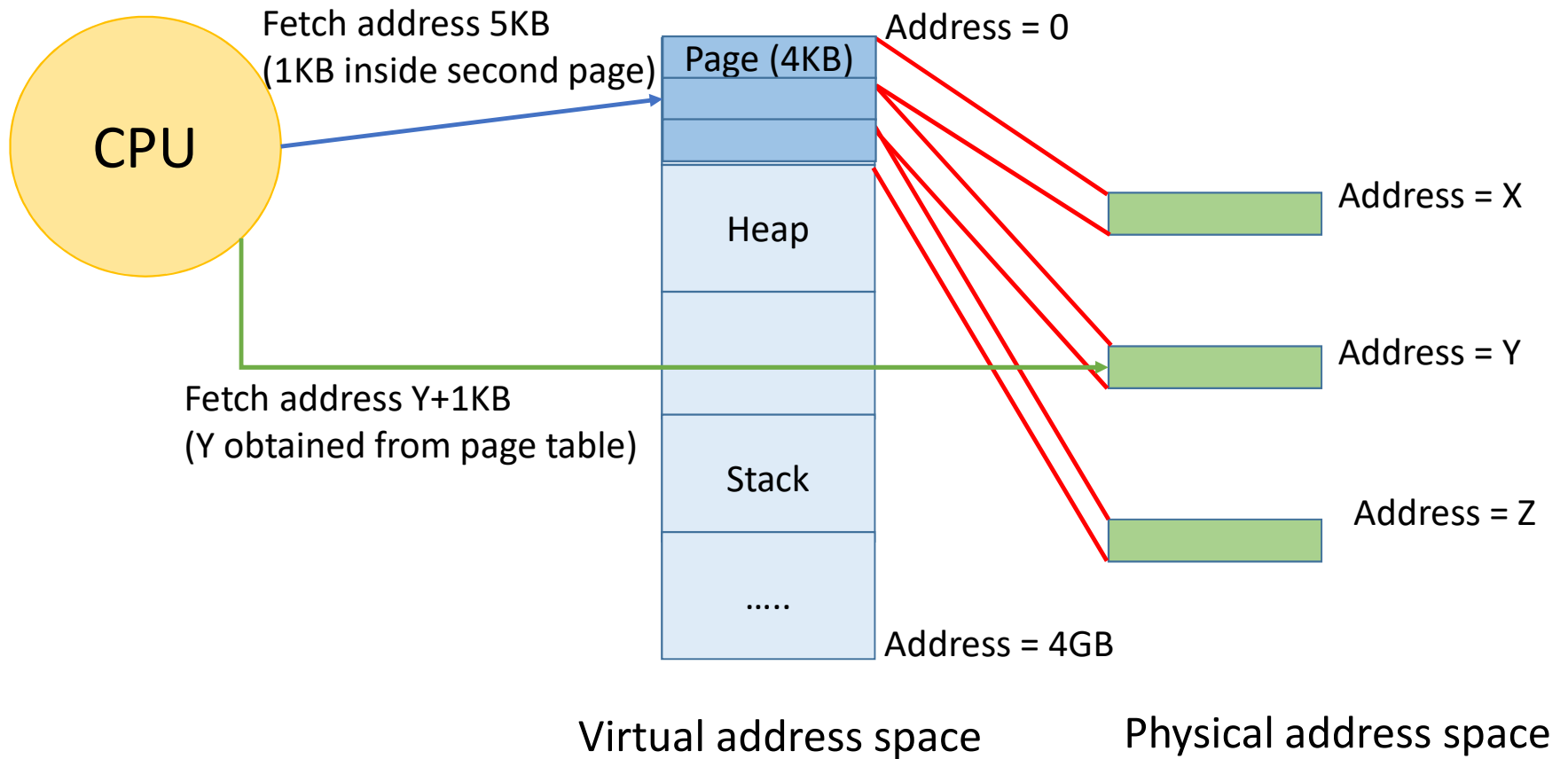
Segmentation

- Older way of memory management, generalized base and bounds
- Each **segment** of the program (code, data, stack,..) is placed separately in memory at a different base
 - Every segment has a separate base and bound
- Virtual address = segment identifier : offset within segment
- Physical address = base address of segment + offset within segment
 - Bound of a segment checked for incorrect access
- Multiple base, bound values stored in MMU for translation
- MMU throws a **segmentation fault** if a segment accessed beyond bound
 - Program fault, traps to OS to handle error, may terminate process

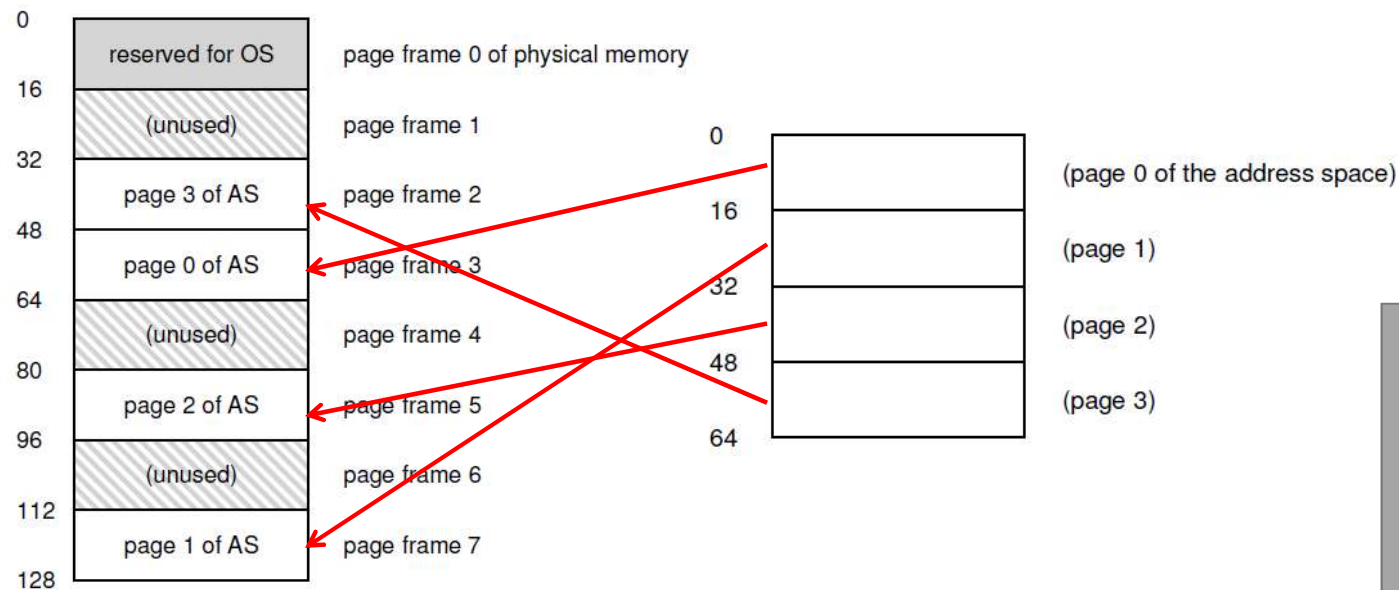
Paging

- Widely used memory management system today
- Virtual address space divided into fixed size **pages**
- Each page is assigned a free **physical frame** by OS
- Memory allocation is at granularity of **fixed size** pages (e.g., 4KB)
- Why paging? Avoids **external fragmentation**
 - No wastage of space due to gaps between allocated and free memory
 - Internal fragmentation may be there (space wasted inside partially filled page)
- Disadvantage: **internal fragmentation** (partially filled pages)
- **Page table** maps logical page numbers to physical frame numbers

Paging



Example of paging

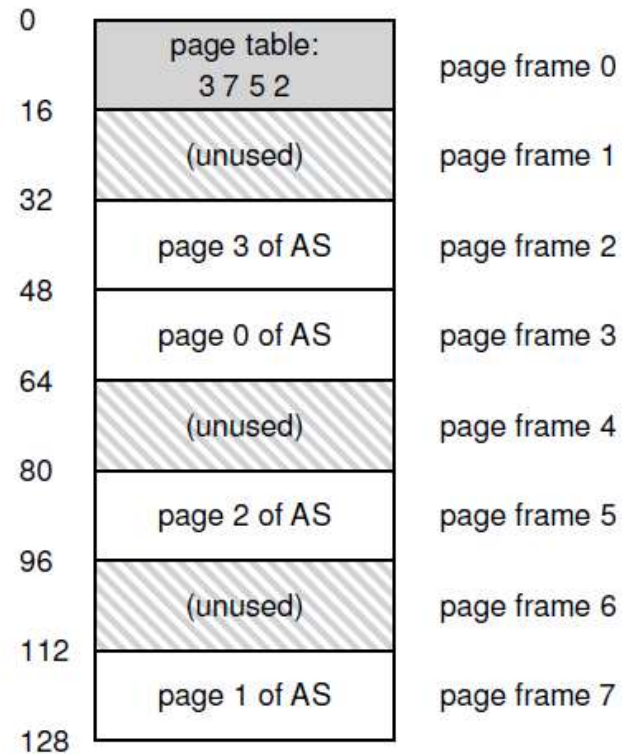


Page to frame mappings:

0 → 3
1 → 7
2 → 5
3 → 2

Page table

- Per process data structure to translate virtual address (VA) to physical address (PA)
- Stores frame numbers for all pages of a process in array
 - [3 7 5 2] corresponding to pages 0 to 3 of the process
- Part of OS memory (in PCB)
- MMU has access to page table of current process, uses it for address translation



View of physical memory

Address translation using paging

- Address translation performed by MMU using page table
- Most significant bits (MSB) of VA give virtual page number, least significant bits (LSB) give offset within page
- Page table maps virtual page number (VPN) to physical frame number (PFN)
- MMU maps VPN to PFN, adds offset to get PA
- Location of page table of currently running process known to MMU
 - Written into special CPU register by the OS, updated on every context switch/page table change

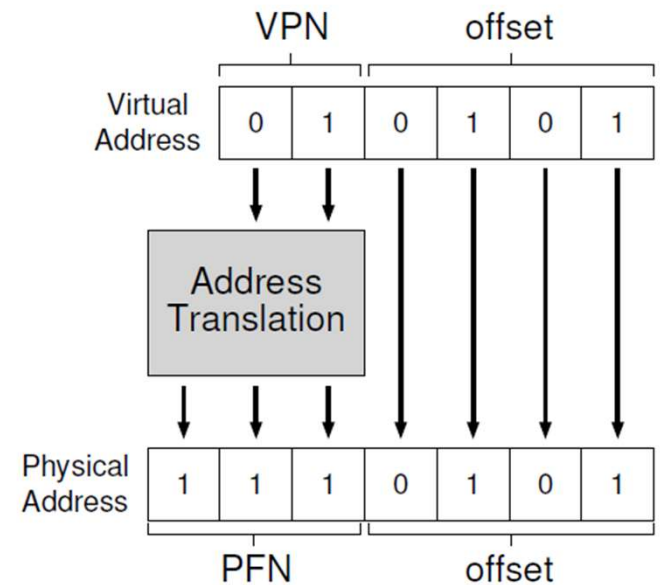


Figure 18.3: The Address Translation Process

Example: 32-bit systems

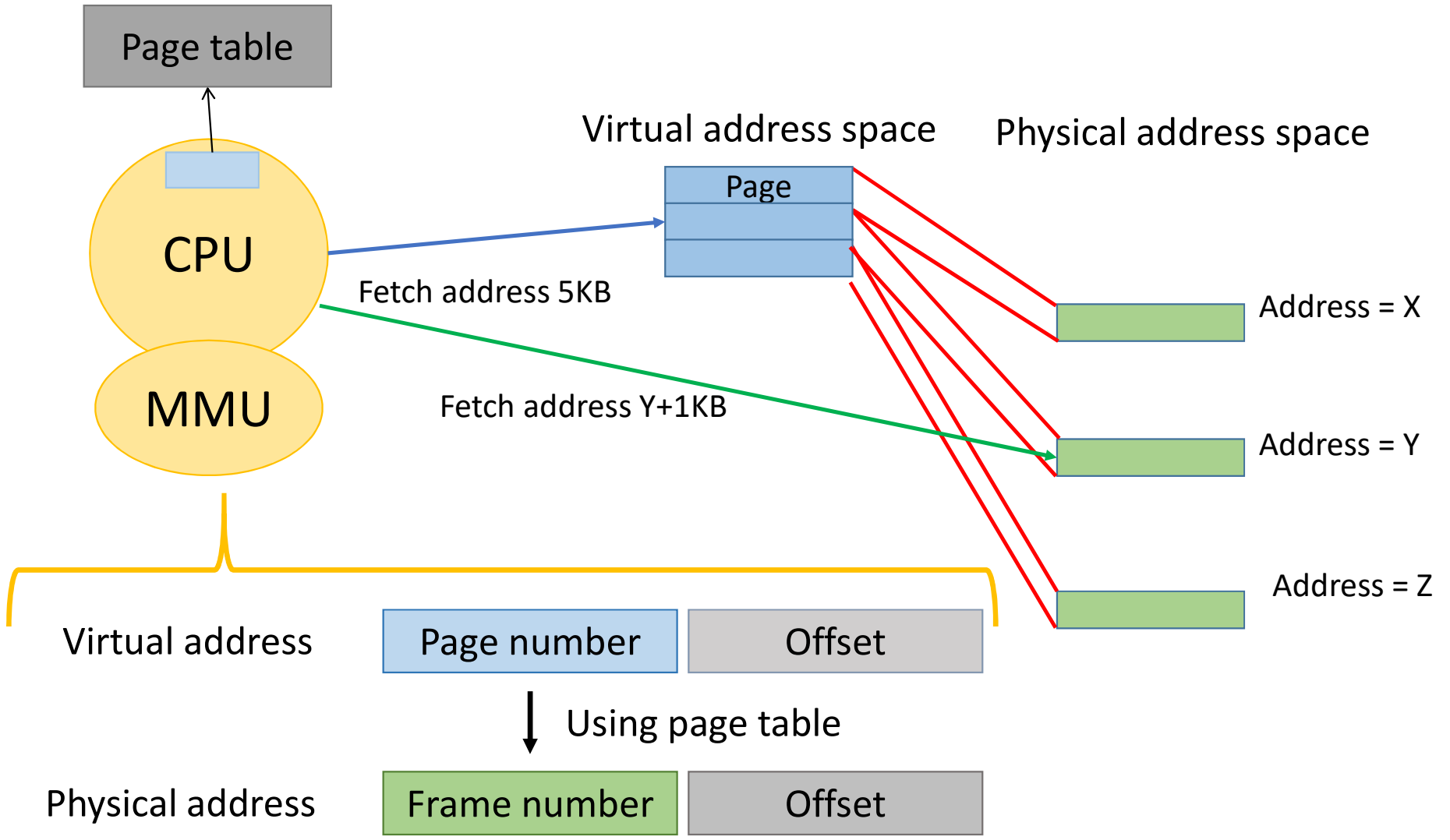
$$1K = 2^{10} = 1024$$

$$1M = 2^{20} = 1024 * 1024$$

$$1G = 2^{30} = 1024 * 1024 * 1024$$

B = byte, b = bit

- 32 bit virtual addresses, so virtual address space size is $2^{32} = 4GB$
- Typical page size = $4KB = 2^{12}$ bytes
- Offset within page needs 12 bits
- 32 bit VA = 20 bit VPN + 12 bit offset
- Number of pages in address space of process = $4GB / 4KB = 1M$
- Size of page table array is 1M entries per process!



Page table structure

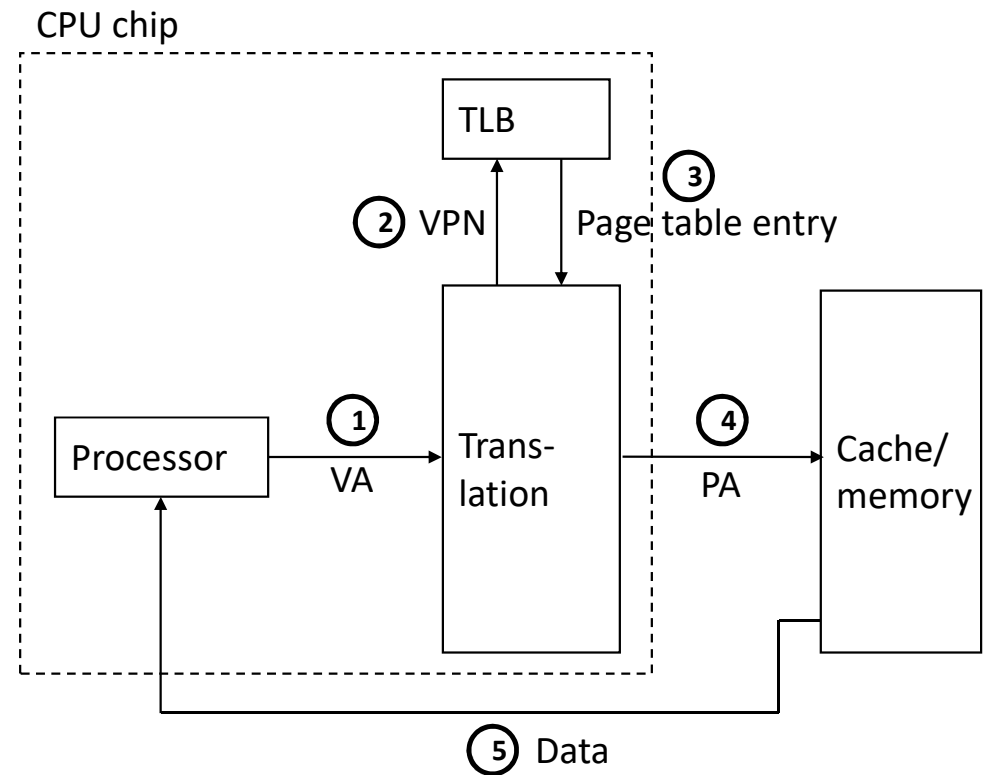
- Page table is an array, where i -th entry contains the information (physical frame number etc) of the i -th page of the process
- Page table has entries for all pages in address space, even those for which there is no physical frame number
 - It is an array with fixed number of entries, not a dynamic data structure
 - Why this design? Why not a dynamic data structure whose size depends on number of used pages?
- Page table structure fixed because the logic to traverse the page table is baked into MMU hardware

Translation Lookaside Buffer (TLB)

- Overhead of memory translation: every memory access preceded by extra memory accesses to read page table
- To reduce this overhead, MMU caches the most recent translations in **translation lookaside buffer (TLB)**
- TLB only **caches page table entries** (VPN → PFN mappings), not actual memory contents
 - Different from CPU caches that cache actual memory contents
- If TLB hit, fetch memory contents in one memory access
- If TLB miss, MMU must perform extra memory access for page table access (“page table walk”)
- **TLB flush** on context switch: mappings cached in TLB change

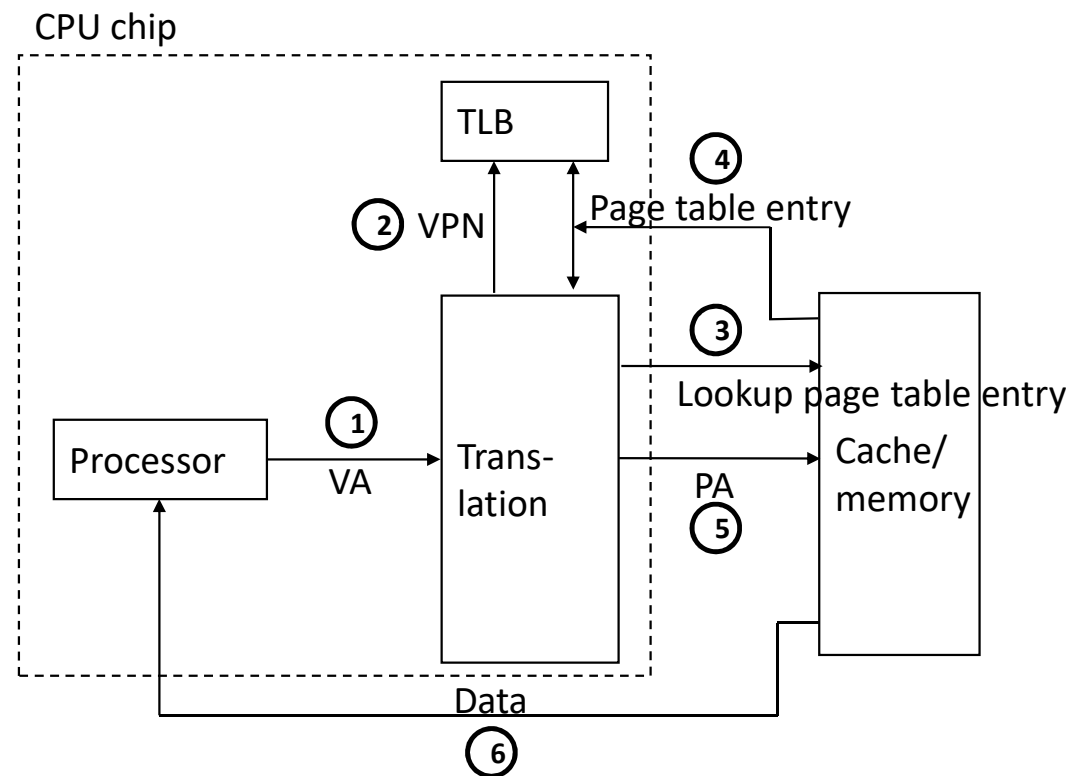
TLB hit

1. CPU accesses virtual address
2. MMU looks up page number in TLB
3. If TLB hit, page table entry is available, physical address computed
4. CPU directly accesses required code/data using physical address



TLB miss

1. CPU accesses virtual address
2. MMU looks up page number in TLB, cannot find entry
3. MMU looks up page table in memory to find page table entry
4. Page table entry populated in TLB for future use
5. MMU computes physical address using which CPU accesses main memory



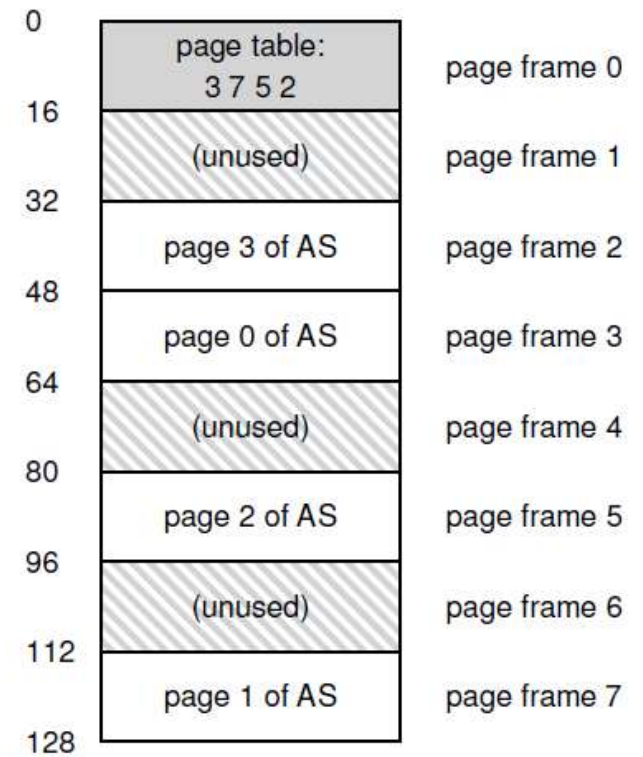
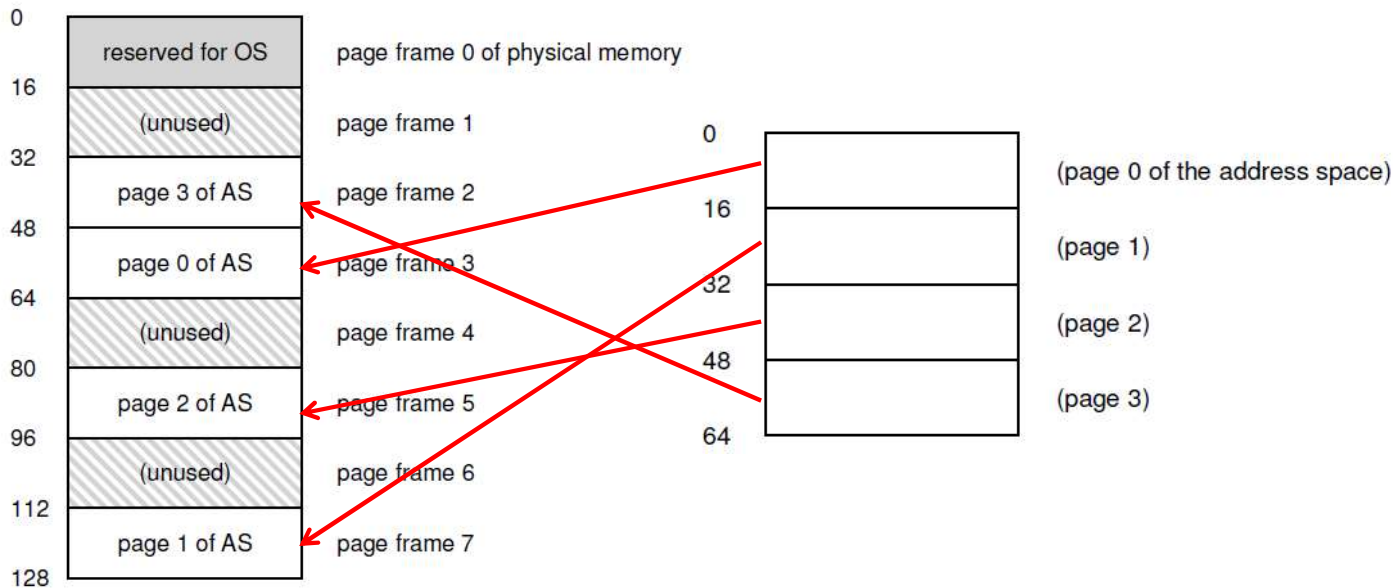
Putting it all together: what happens on a memory access?

- CPU has requested data (or instruction) at a certain memory address
 - If requested address not in CPU cache, CPU must fetch data from main memory
 - CPU knows only virtual address of instruction or data required
 - MMU looks up TLB to find frame number corresponding to page number
 - If TLB hit, physical address is found, main memory is accessed to fetch data
 - If TLB miss, MMU first accesses page table in main memory, computes physical address, then accesses main memory again to fetch data
 - Fetched page table entries and data are populated in TLB / caches
- High CPU cache hit rates and high TLB hit rates are important for good performance of the system

Paging

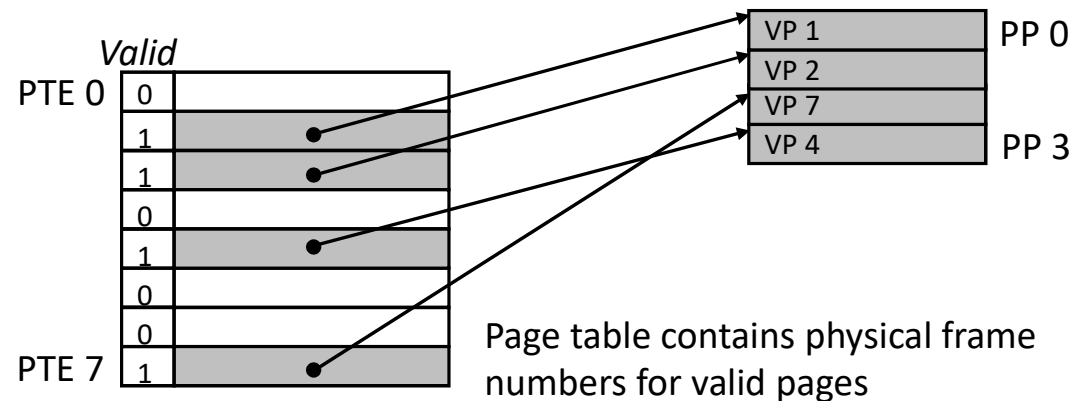
Mythili Vutukuru
CSE, IIT Bombay

Recap: Paging and page table



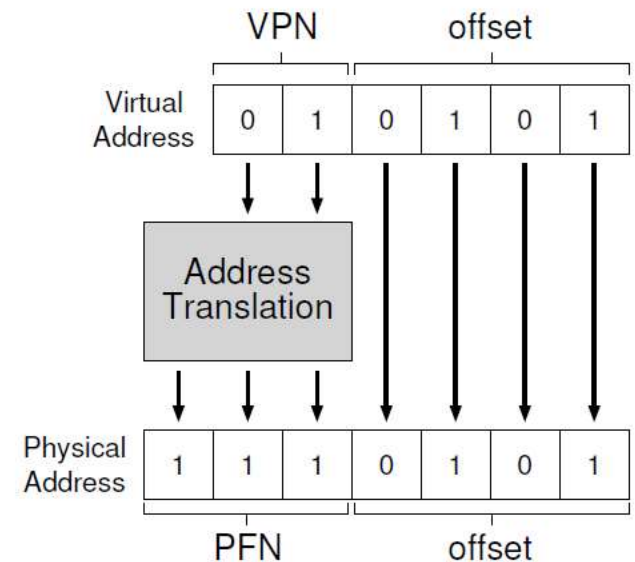
Page table entry

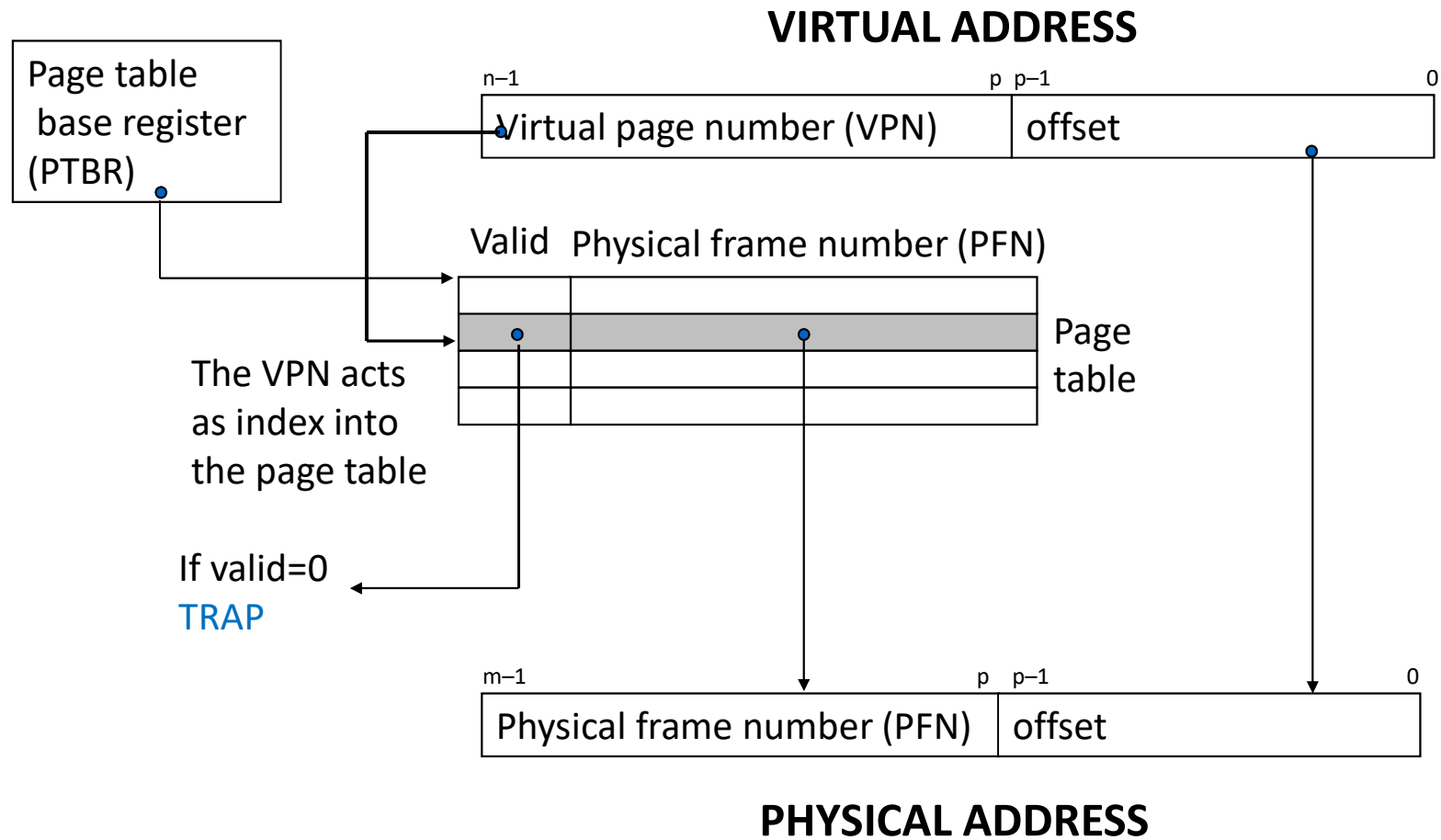
- Page table is array of page table entries, one per page of process
- i-th **page table entry (PTE)** contains physical frame number and other details (permissions, status, ..) of i-th page of process
 - Valid: is this page in use by process (not all virtual addresses are used by process)
 - Various permission bits (more later)
 - Other status bits: present, dirty, accessed (more later)



Address translation in MMU

- MMU stores starting (physical) address of page table array in CPU register called page table base register
- Page size determines number of bits in offset
 - 4KB pages need $\log_2(4K) = 12$ bits as offset within page
- Remaining most significant bits give VPN
 - For 32-bit machines and 4KB pages, 20 bit VPN
- MMU uses VPN as index into page table array, accesses PTE, gets PFN, adds offset bits to get PA
- If no valid PTE found, MMU traps to OS





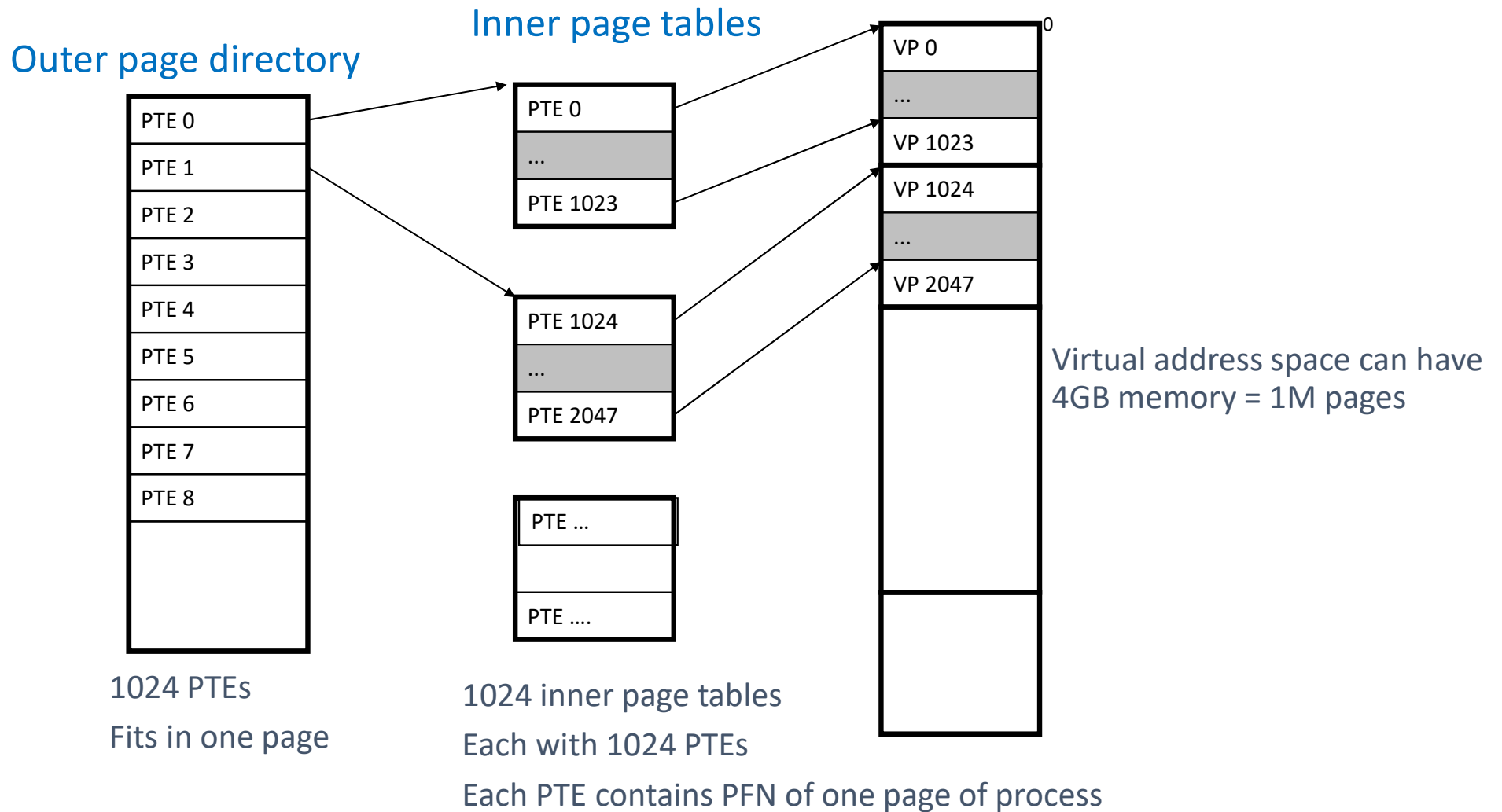
Size of page tables

$1K = 2^{10} = 1024$
 $1M = 2^{20} = 1024 * 1024$
 $1G = 2^{30} = 1024 * 1024 * 1024$
B = byte, b = bit

- What is typical size of page table in a 32-bit system?
- $2^{32} = 4GB$ virtual address space
- Assume page size = $4KB = 2^{12}$
- Number of PTEs = number of pages in virtual address space = $(2^{32}/2^{12}) = 2^{20} = 1M$
- If each PTE is 4 bytes, page table size = 4 bytes * 1M entries = 4MB
- How are page tables stored in memory?
 - All memory is only allocated in 4KB chunks, so how to store 4MB?
- Solution: split page table into pages (much like memory image), use another page table to keep track of original page table!

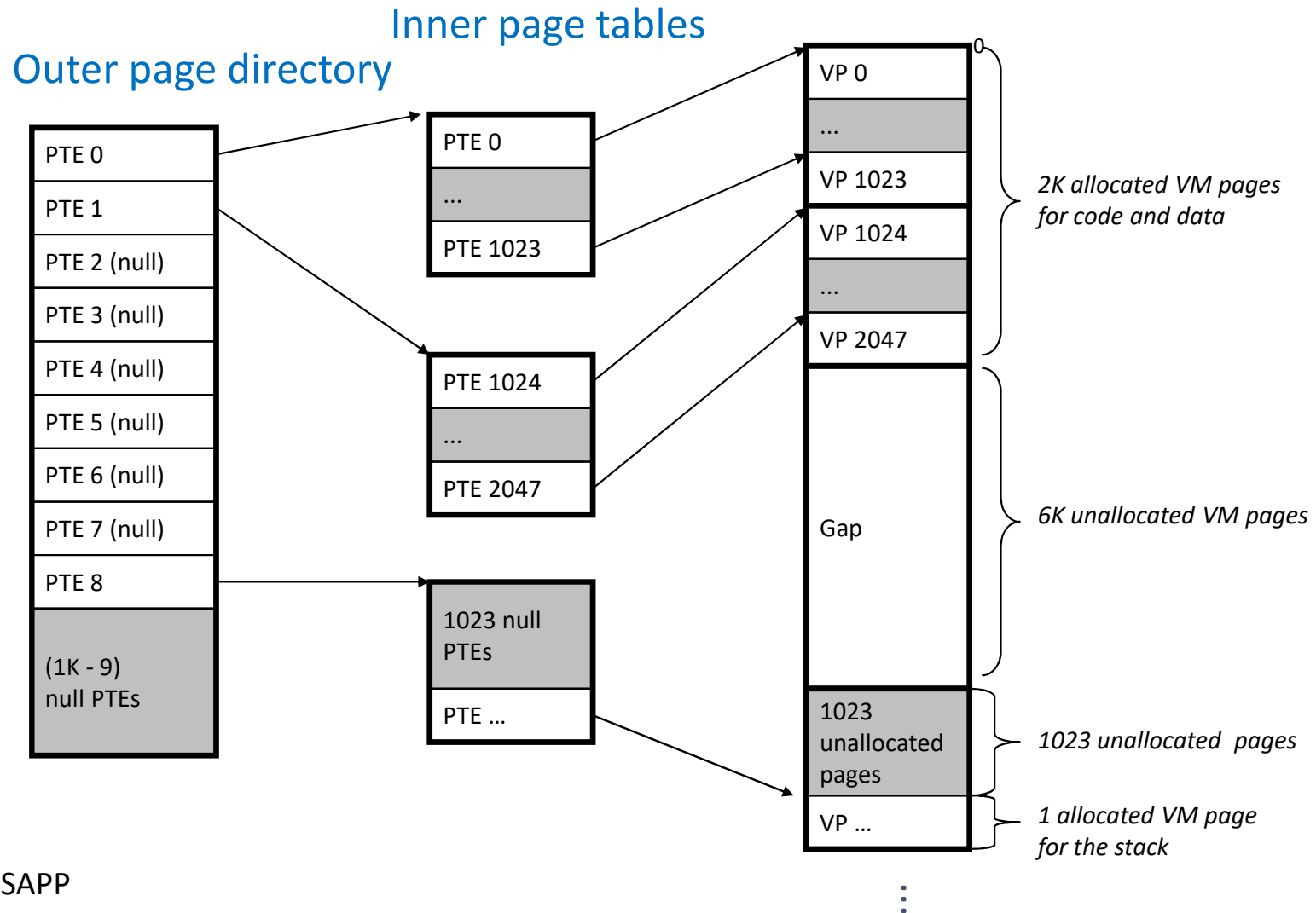
Two-level page table in 32-bit systems

- 4MB page table split into 1024 chunks of 4KB each (to fit in page)
- 1M PTEs split across 1024 pages, each containing 1024 PTEs
- Physical frame numbers of these 1024 chunks stored in an outer page table or page directory
 - 4 byte page table entry each, so outer page directory fits in one page here
- Page table now has two levels
 - **Outer page table (page directory)** has physical frame numbers of 1024 “inner” page table pages
 - Each **inner page table** has physical frame numbers (PTEs) of 1024 pages of the process virtual address space



Inner page tables on demand

- Note: not all inner page tables need to be created always, only those with at least one valid entry needed
- Example: Process with 2K pages of code+data, 6K + 1023 unallocated pages in address space, then one page allocated for stack
 - First two inner page tables are allocated, hold the 2K valid PTEs
 - Next 6 inner page tables are not created, the corresponding entries in outer page directory are invalid / null
 - In next inner page table, 1023 invalid entries and one valid PTE containing frame number of stack page
 - Remaining inner page tables not created, corresponding outer page directory entries are invalid

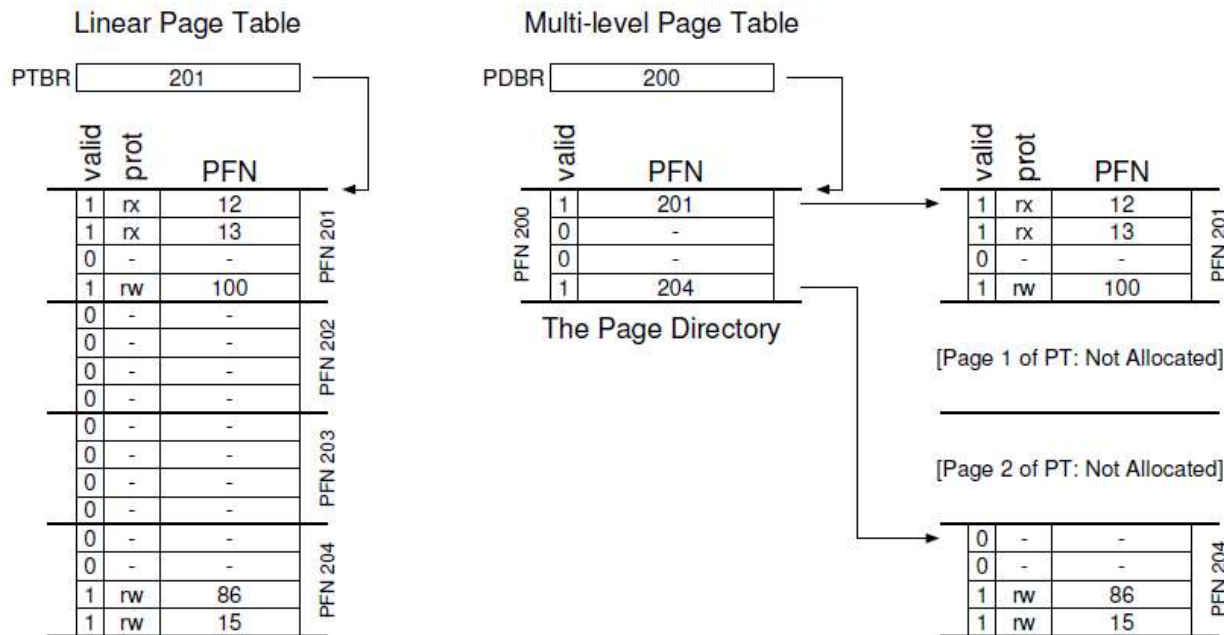


Address translation in 2-level page table

- Virtual address of 32 bits = 20 bit page number + 12 bit offset
- 20 bits index into a single page table is now used as
 - Most significant 10 bits index into page directory, locate PTE of one of the 1024 inner page tables contain our desired address
 - Next 10 bits index into inner page table to locate PTE of page
- Locate PTE, computer physical address using frame number and 12-bit offset into page
- MMU “walks” the multiple levels of the page table to translate virtual addresses

Page table/directory base register

- Single level: MMU stores starting address of page table in page table base register
- Multi-level: MMU stores starting address of outer page directory in page directory base register (CR3 register in x86)



Multi-level page tables

- What if outer page directory does not fit into one page?
- Store page directory across many pages, use yet another page table to store frame numbers of page directory pages
- This can go on until outermost page table fits in one page
- Example: 48-bit CPU, 4KB pages, 8 byte page table entries
 - 2^{48} bytes in virtual address space = 2^{36} pages for each process
 - Each page can store $4\text{KB}/8 = 2^9 = 512$ page table entries
 - Innermost level (actual page table) has 2^{36} page table entries = needs 2^{27} pages
 - Innermost page table split into multiple pages = 2^{27} page table entries to track innermost page table pages
 - Next level of page table stores 2^{27} page table entries = needs 2^{18} pages
 - Next level stores 2^{18} page table entries = needs $2^9 = 512$ pages
 - Outermost level can store all 512 page table entries in 1 page

Address translation with 4-level page table

- Example: 48-bit CPU, 4KB pages, 8 byte page table entries
 - 4 level page table required
 - Outermost page directory has 512 entries, containing frame numbers of next level page table pages, each of those contain frame numbers of next level page table, ...
 - Page table at i -th level has frame numbers of 512 $(i+1)$ -th level page table pages
- How to translate VA to PA?
 - 48-bit VA = 36 bits + 12 bit offset
 - 36 bits = 9 bit offset into each of the 4 levels of page table
- If TLB miss, MMU has to access 4 different memory locations for 4 levels of page table, in order to translate one VA to PA
- MMU page table walks become even longer, TLB hit rate is critical

Address translation with multi-level page table

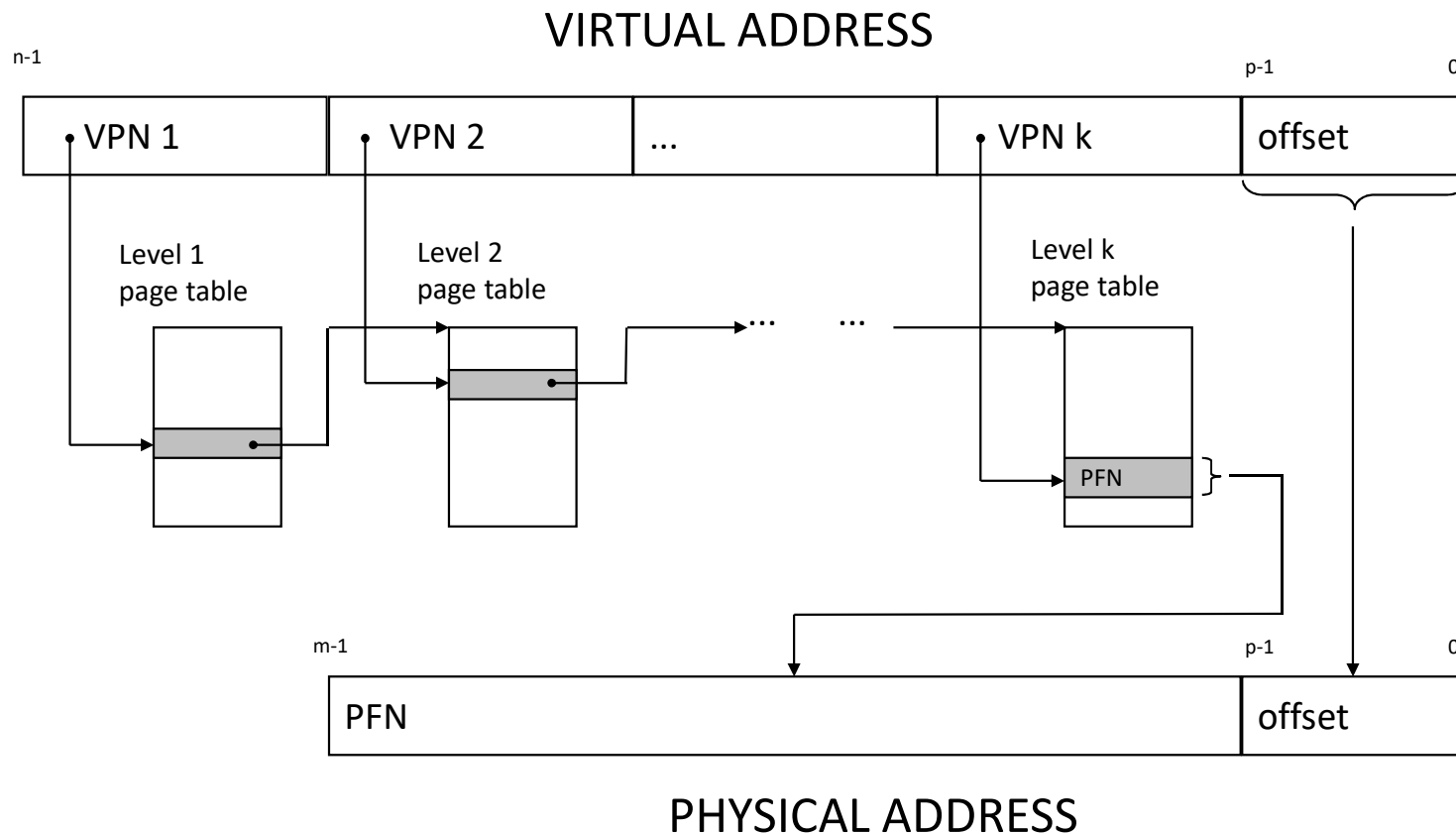


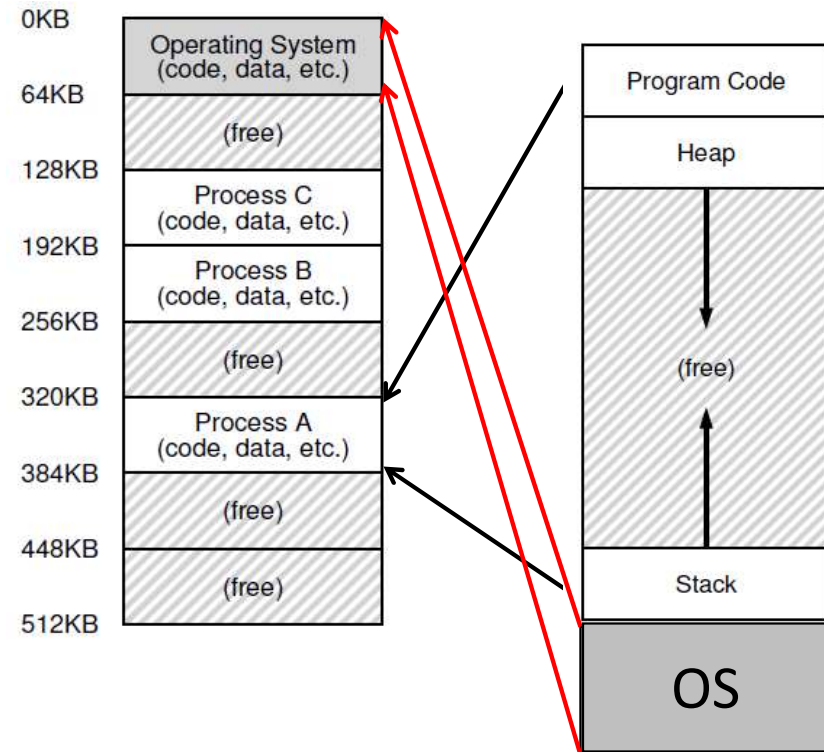
Image credit: CSAPP

Revisiting process virtual address space

- What should virtual address space/page table of process have? Any memory that the process needs to access during its execution
 - Its own memory image: code, data, stack, heap
 - Other common memory it needs to access: shared language libraries, OS
- Why? MMU allows access to memory **only via virtual addresses**
 - Can only access physical memory mapped in page table at some virtual address
 - So all physical memory needed by process should be mapped into address space
- OS binary image (kernel code, data) is mapped into the virtual address space of every process at addresses not used by process (high VA)
- Why is this done? Easy to jump to OS code during a trap

A subtle point

- OS is not a separate process with its own address space
- Instead, OS code is part of the address space of every process
- A process sees OS as part of its code (e.g., like a library)
- During trap, process jumps to high virtual addresses and executes OS code



OS is part of address space of every process

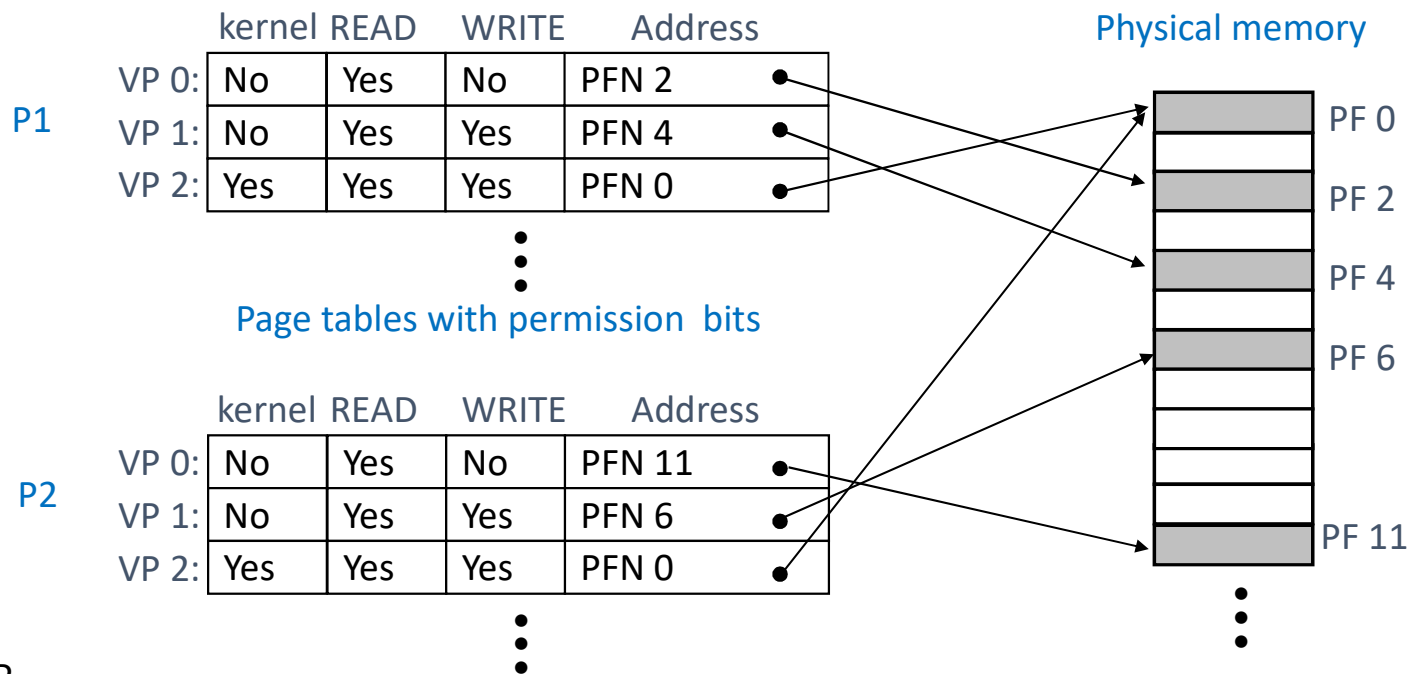
- OS code/data assigned virtual addresses
 - Compiler ensures high virtual addresses not used by user code
- OS virtual addresses are mapped to physical addresses of OS via page table entries of every process
- There is only one copy of OS code/data in RAM
 - Loaded into RAM at low physical addresses during system bootup
- Page tables of all processes have mappings to same OS physical addresses
 - Same high virtual addresses map to same physical addresses of OS code

Page-level isolation and security

- How is OS code/data protected from illegal access by user?
- Page table has permissions for every memory page
 - Whether read/write or read-only (code pages are read-only)
 - Whether page can be accessed in user mode or kernel mode
- Page table mappings for OS code are protected to allow access only when CPU is in kernel mode
 - CPU in user mode cannot access high virtual addresses of OS code
 - CPU in kernel mode (after trap instruction) can access OS code/data
- MMU traps to OS if any violation detected during memory access, ensures user programs can only access memory they are permitted to access

Example: page-level protection using page tables

- Example: process P1 and P2 each have one read-only page, one read-write page, and one page with OS code accessible in kernel mode



Memory management in xv6

- 32-bit OS, so $2^{32}=4\text{GB}$ virtual address space for every process
- 4KB pages, so 32 bit VA = 20 bit page number + 12 bit offset
- Each PTE has 20 bit physical frame number, and some flags
 - PTE_P indicates if page is valid/present (if not set, access will cause page fault)
 - PTE_W indicates if writeable (if not set, only reading is permitted)
 - PTE_U indicates if user page (if not set, only kernel can access the page)
- Address translation: use page number (top 20 bits of virtual address) to index into page table, find physical frame number, add 12-bit offset

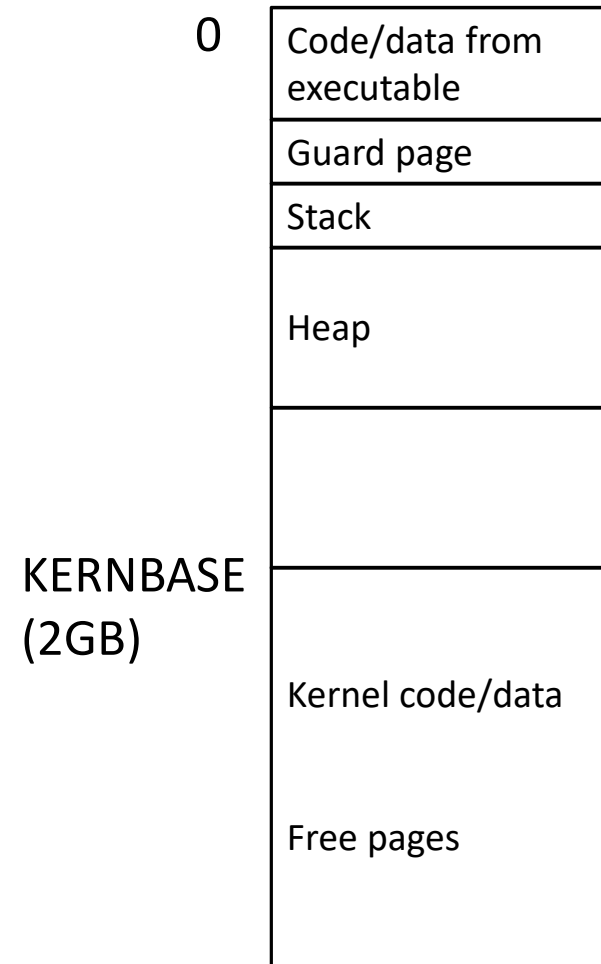
Two level page table in xv6

- xv6 has two-level page table
 - 1024 “inner” page table pages, each with 1024 PTEs
 - Outer page directory stores PTE-like references to 1024 inner page table pages
 - Physical address of outer page directory is stored in CPU’s cr3 register, used by MMU during address translation
- 32 bit virtual address = 10 bits index into page directory, next 10 bits index into inner page table, last 12 bits are offset within page
 - PFN from PTE + offset = physical address

```
0773 // A virtual address 'la' has a three-part structure as follows:
0774 //
0775 // +-----10-----+-----10-----+-----12-----+
0776 // | Page Directory |   Page Table   | Offset within Page |
0777 // |      Index     |      Index     |                   |
0778 // +-----+-----+-----+
0779 // \--- PDX(va) ---/ \--- PTX(va) ---/
0780
```

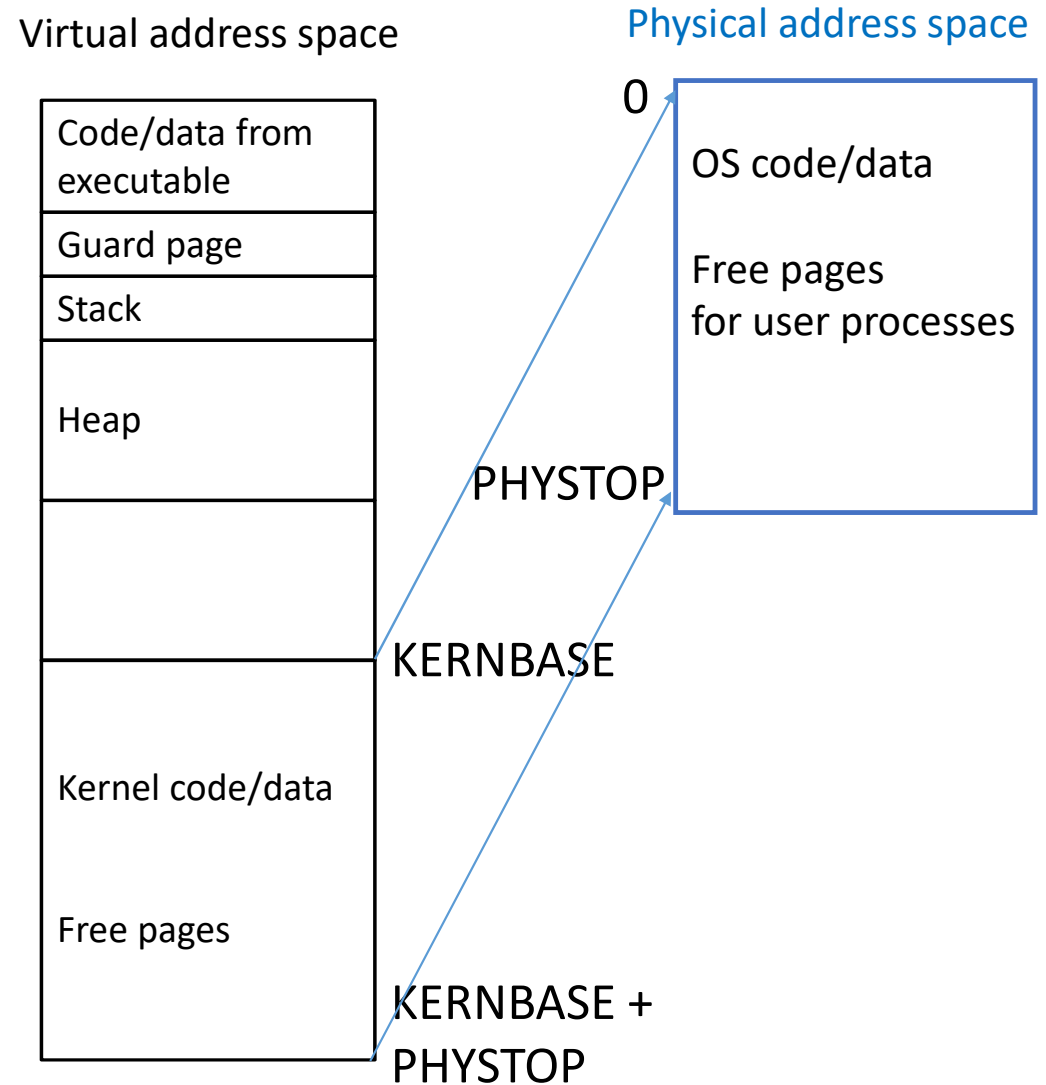
Virtual address space in xv6

- Virtual address space [0, 4GB]
- Physical address space [0, PHYSTOP]
where PHYSTOP is max physical memory that can be used
- Virtual address space contains
 - Low virtual addresses: user code/data, guard page, stack, expandable heap
 - High virtual address starting at KERNBASE (2GB): kernel code/data, free pages that OS assigns to user processes, memory reserved for I/O devices, ...



Page table mappings

- Page table contains two sets of PTEs
- User entries: low VA to PA used to process for code, data, stack, heap
- Kernel entries: high VA to PA containing OS code/data/free pages
 - $[KERNBASE, KERNBASE + PHYSTOP]$ mapped to $[0, PHYSTOP]$
- Kernel page table entries identical across all processes



Demand paging

Mythili Vutukuru
CSE, IIT Bombay

Recap: Virtual addresses and paging

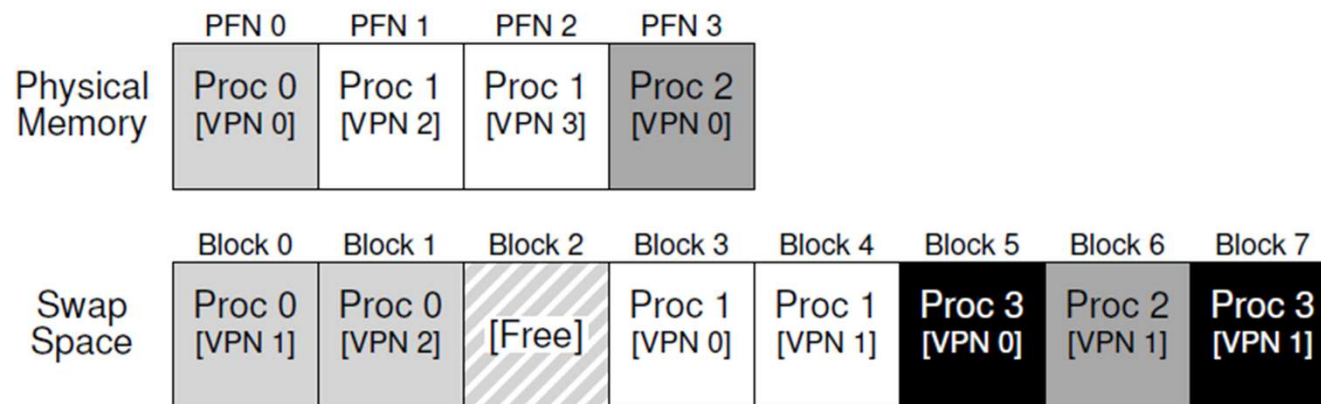
- Instructions and data of a process in memory assigned **virtual addresses**
 - Starting at 0 for user code (OS code also assigned high virtual addresses)
- Virtual address space of a process divided into fixed size logical **pages**, stored in a fixed size physical frames in memory
 - Prevents external fragmentation, cannot prevent internal fragmentation
- **Page table** maps logical page numbers to physical frame numbers
 - One per process, maintained by OS as part of PCB
 - Used by MMU to translate VA to PA when CPU accesses memory

Demand Paging

- Should all pages of all active processes always be in main memory?
 - Not necessary, as process will not use all of it at once
 - Not possible, with large address spaces
- Modern operating systems provide **virtual memory**
 - Not all logical pages are assigned frames, some memory is “virtual”
 - Why? Virtual memory of processes can be much more than physical memory in the system, OS overcommits memory
- Some pages in address space are not allocated at all (not valid)
- Some valid pages are allocated physical frames in memory, some are temporarily saved on disk, brought into DRAM when needed
- No demand paging in the simple xv6 OS

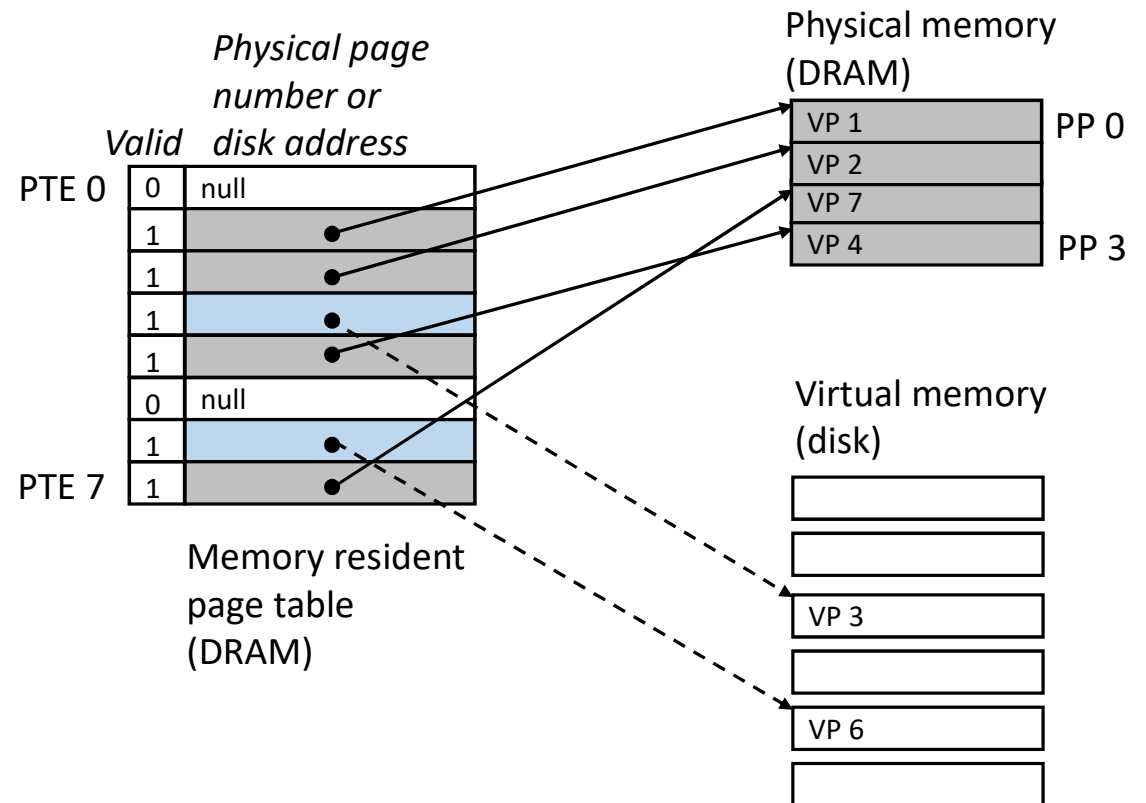
Swap space

- Special area on disk to hold pages that do not fit in DRAM
- Pages pushed to swap space when memory is full, brought into memory when accessed



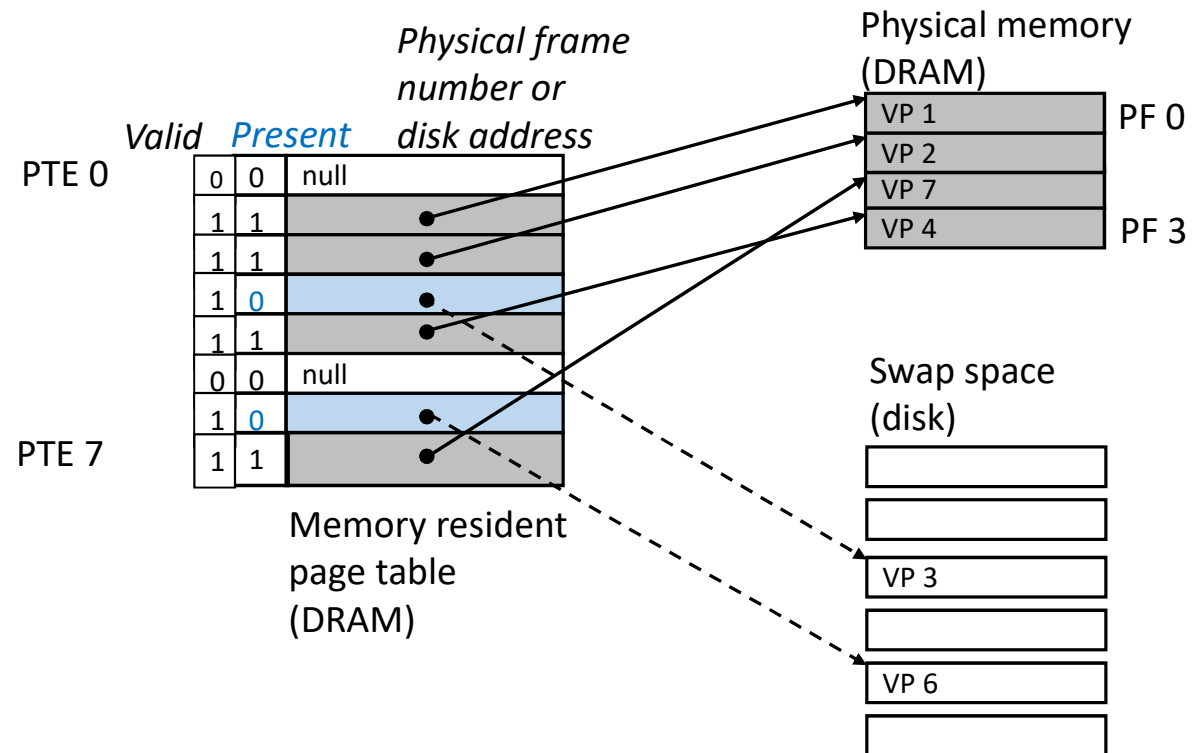
Tracking pages in swap space

- Invalid page addresses not in use by process, no need to store any frame number
- A valid page either has a physical frame number in memory, or a disk address in swap space, both tracked by page table



Valid and present bits in page table entry

- Valid bit in PTE indicates if virtual page is in use by process
- Present bit indicates if page is allocated frame in main memory
- Valid bit and present bit both set → page in DRAM
- Valid set, present not set → page in swap space



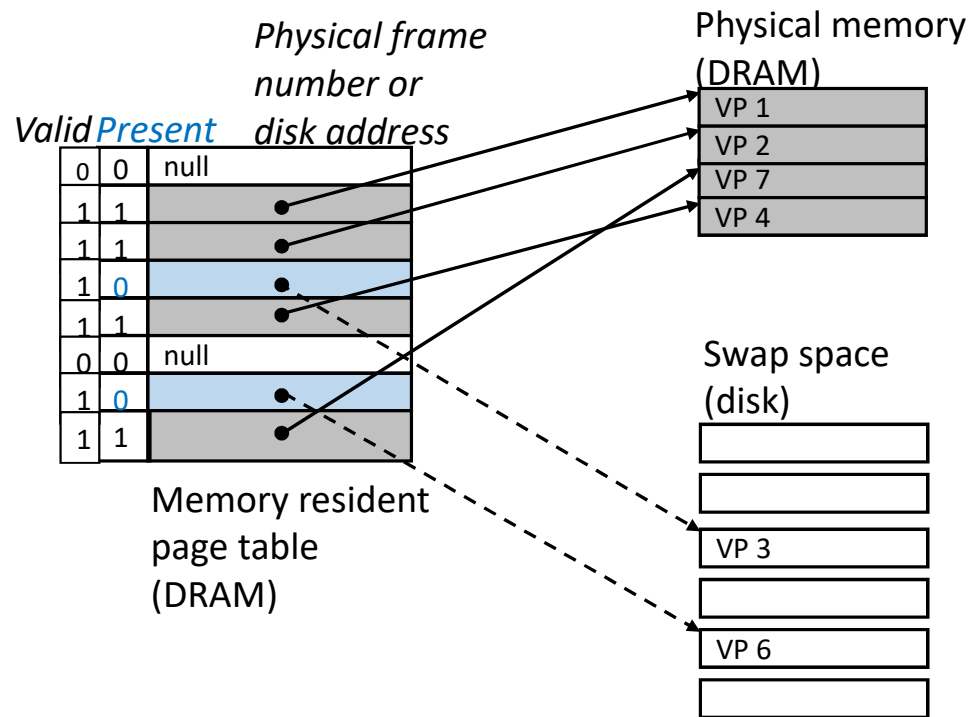
Page fault

- When MMU walks page table to translate a virtual address to physical address, the various bits in page table entry are also examined
- MMU traps to the OS (**page fault**) in case of any unexpected behavior
- CPU switches to kernel mode, runs page fault handler code
- How does OS handle page fault?
 - Valid bit not set → segmentation fault, terminate process
 - Any other illegal access (e.g., writing to read-only page) → terminate process
 - Valid bit set, present bit not set → OS allocates memory frame for page in DRAM, updates page table, restarts process (hopefully, all goes well now!)

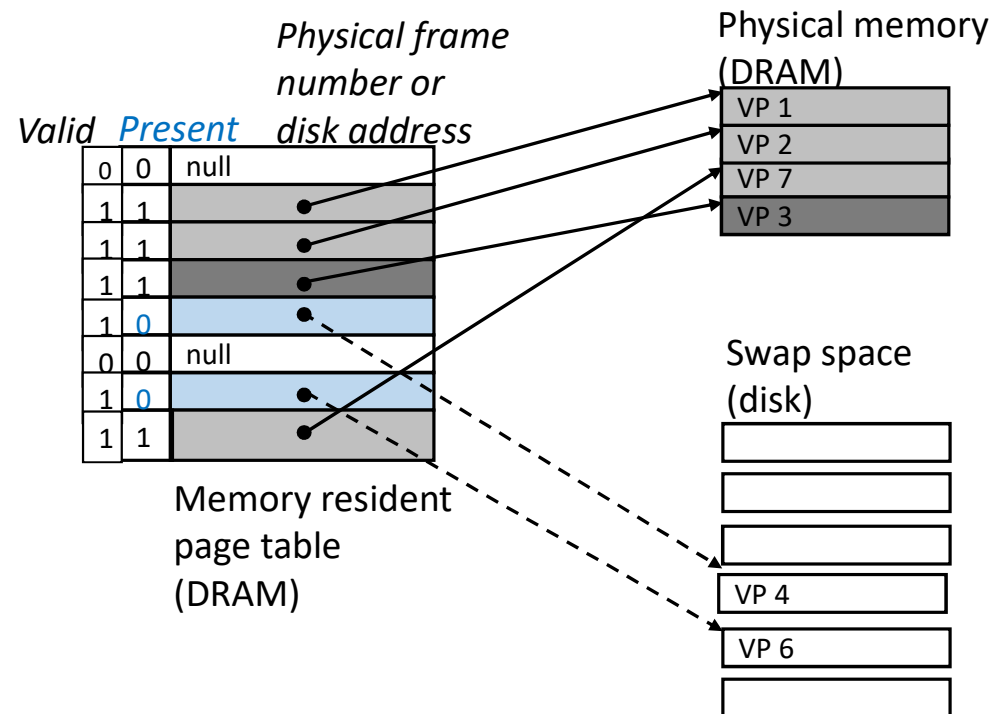
Reclaiming memory

- OS usually keeps track of free pages in memory to use in page fault
- If no free frames in DRAM when servicing page fault, OS evicts a **victim page** from memory to swap space, allocates the freed up physical frame to faulting page
 - Copy contents of victim page from memory frame to swap space on disk
 - Copy contents of allocated page from disk into freed up memory frame
 - Update corresponding page table entries, restart process
- Page replacement policy of OS helps to identify suitable victim page
- Victim page can be from same process or from different process

CPU accesses page 3 (page fault)



Page 3 brought into memory, victim page 4 is moved to swap



File backed, anonymous, dirty pages

- Pages in the memory image of a process are of two types
 - **File-backed pages** contain data from files on disk (e.g., page with executable code)
 - **Anonymous pages** are not backed by files on disk (e.g., pages containing stack, heap)
- Further classification into dirty and non-dirty pages
- **Dirty pages**: pages whose content is different from their copy on disk
 - E.g., file-backed pages whose contents have changed from file
 - E.g., anonymous pages whose contents have changed since last read from swap
- PTE has information to track these types of pages, including dirty bit

Disk access during page fault

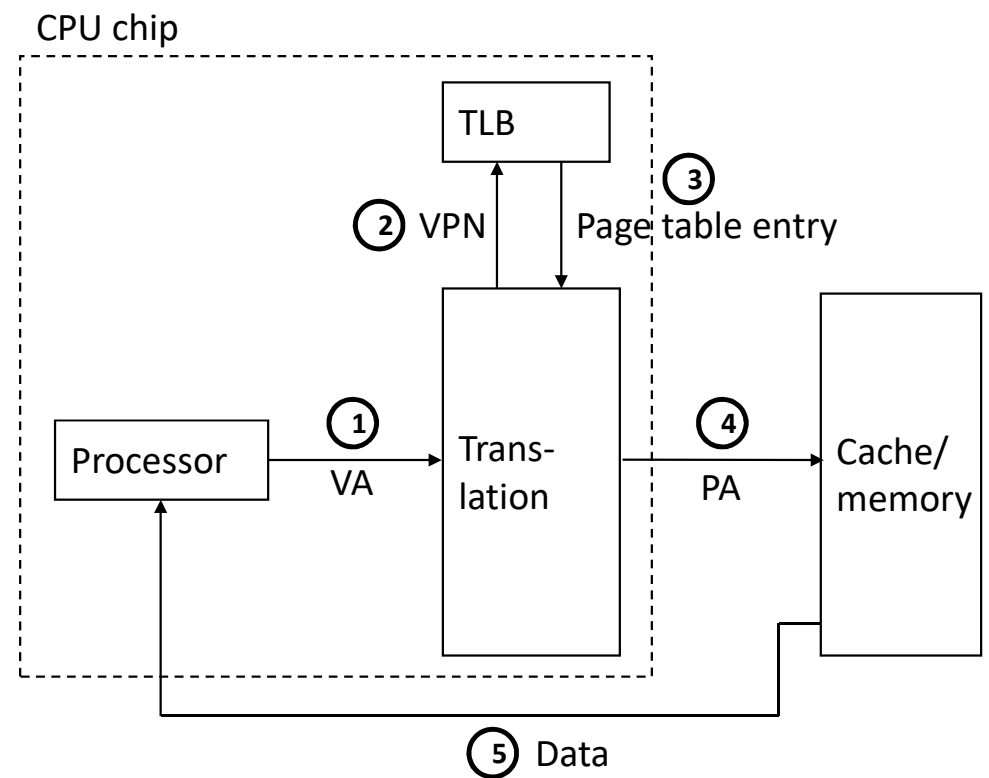
- Actions done by OS when servicing page fault depend on type of pages
- When reclaiming memory from victim page, need to copy content of victim page to swap space only if victim page is dirty
 - For other pages, can simply delete memory content, fetch from disk later
- When allocating memory frame during page fault, free memory frame must be initialized with content from disk for file-backed and non-empty anonymous pages
 - For empty anonymous page that has never been used, can just give empty frame
- Process may be blocked multiple times for disk I/O during page fault!
- Average memory access time (weighted avg across different scenarios during mem access) increases greatly if too many page faults

Summary: What happens on memory access

- CPU accesses code/data using VA
- MMU looks up TLB for VA
 - If TLB hit, get PA, access memory only once to fetch code/data TLB hit, page in memory
 - If TLB miss, access memory to walk page table, get PTE TLB miss, page in memory
 - If PTE is valid and present, compute PA, access memory once to fetch code/data
 - Else, MMU traps to OS
 - If invalid or illegal access, terminate process TLB miss, page not in memory
 - If valid but page not present, allocate free memory frame (maybe by swapping out victim page), swap in contents of page (if needed), update page table, restart process
- Where do CPU caches fit in in this story?
 - Caches can be checked before address translation (virtually addressed caches) or after address translation (physically addressed caches)
 - DRAM accessed only on cache miss

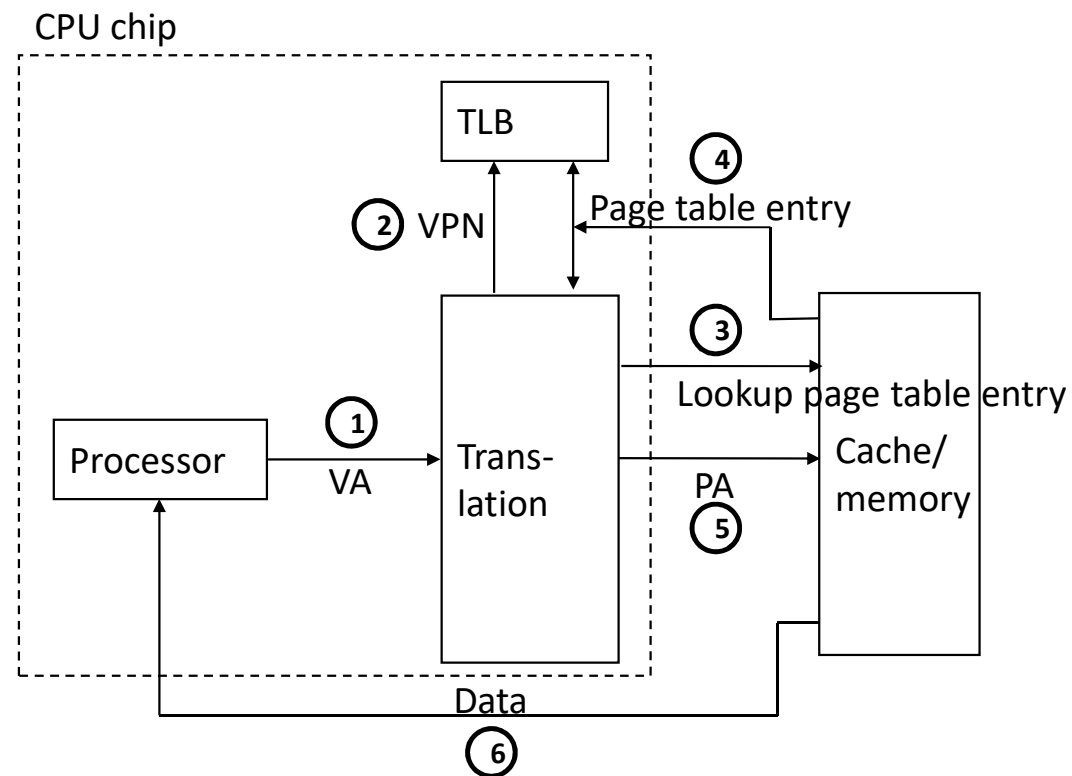
Summary: TLB hit, page in memory

1. CPU accesses virtual address
2. MMU looks up page number in TLB
3. If TLB hit, page table entry is available, physical address computed
4. CPU directly accesses required code/data using physical address

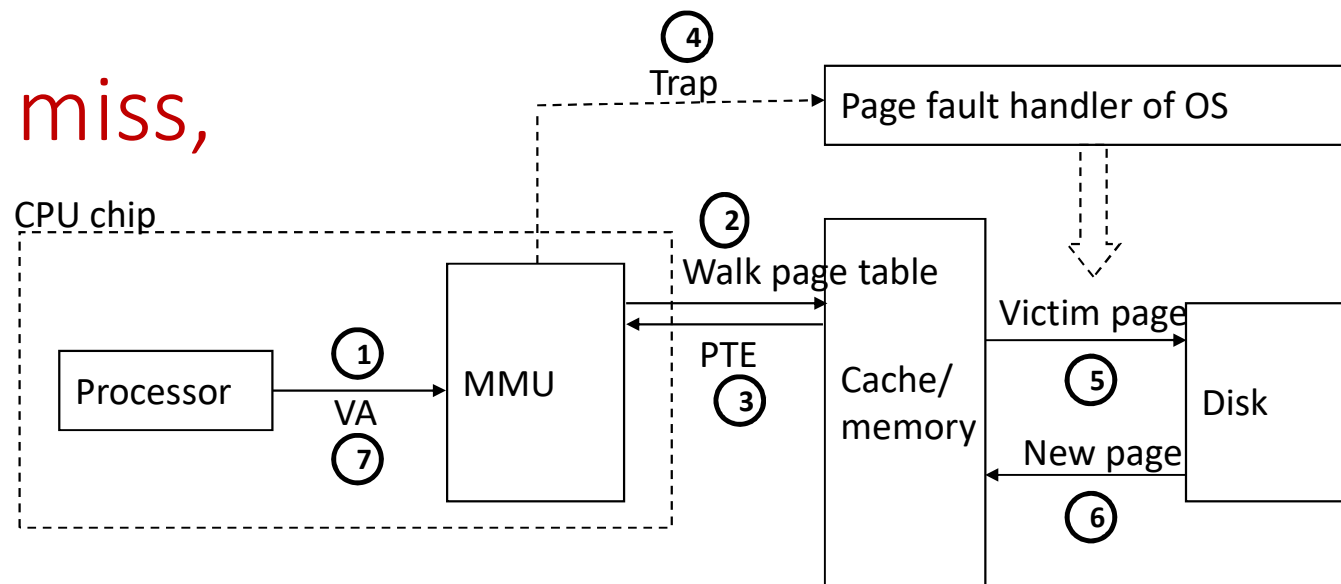


Summary: TLB miss, page in memory

1. CPU accesses virtual address
2. MMU looks up page number in TLB, cannot find entry
3. MMU looks up page table in memory to find page table entry
4. Page table entry populated in TLB for future use
5. MMU computes physical address using which CPU accesses main memory

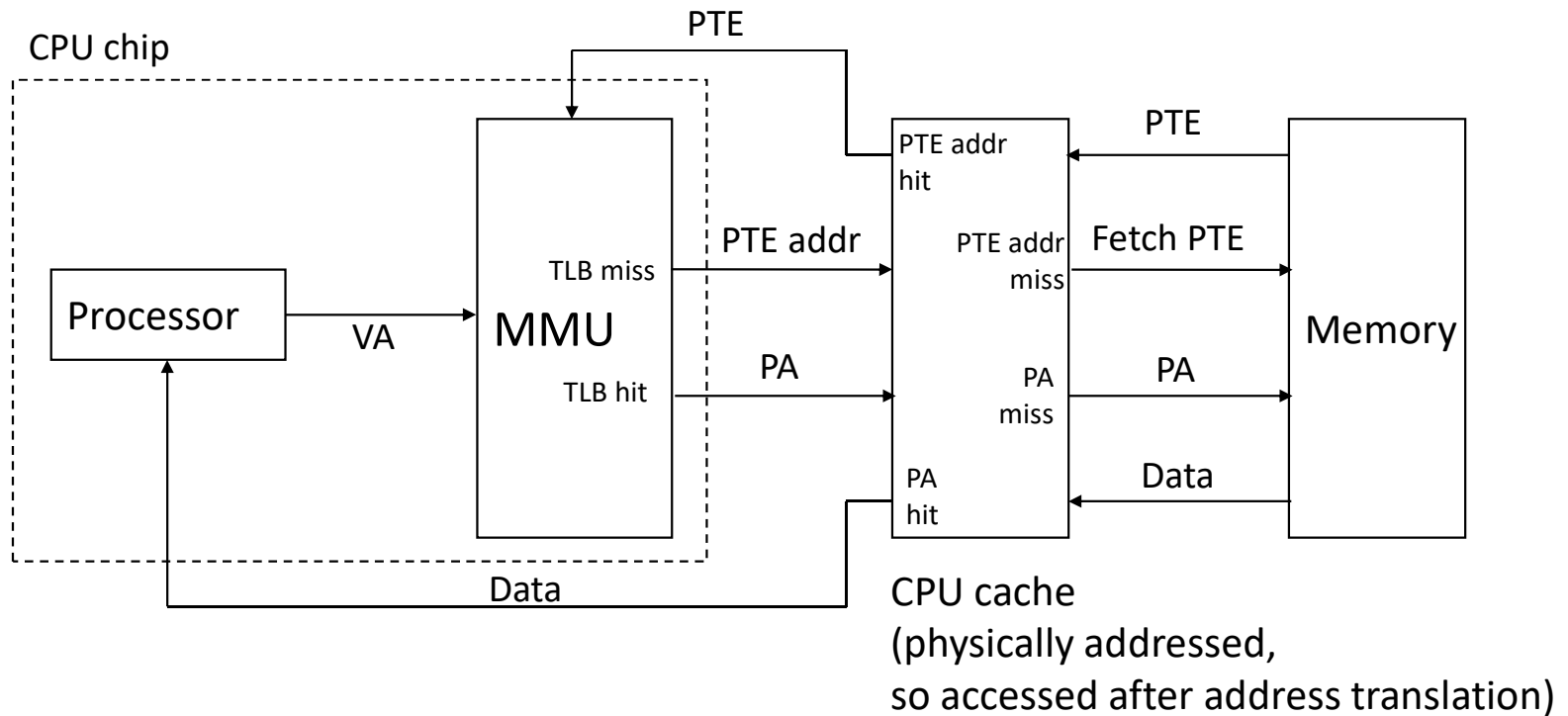


Summary: TLB miss, page fault



1. CPU accesses VA
2. TLB miss, walk page table
3. Get PTE, cannot compute PA
4. MMU traps to OS
5. OS swaps out victim page (if needed)
6. OS swaps in new page (if needed), updates PTE to reflect new mapping
7. Restart original process, memory access succeeds (hopefully!)

Summary: Virtual memory and caches



Thrashing

- Causes for application slowdown: CPU cache miss, TLB miss, page fault, ..
- Page fault particularly dangerous, may involve multiple disk access
- OS should allocate enough physical memory to avoid page faults, but how much?
- Every process has a **working set**: frequently used pages in memory image
 - Can change from time to time, based on code being executed
 - Usually smaller than total virtual memory of process
 - If memory assigned to is less than working set, frequent page faults
- **Thrashing** = system spends too much time servicing page faults and swapping back and forth from disk, and too little time doing useful application work, significant slowdown noticeable by users
- Solution: users can reduce working set of processes, OS can terminate some processes or clean up unnecessary memory, ...

Page replacement policies

- Page replacement policy: which victim page should OS pick to evict?
- Goal: Minimize page faults, evict pages not likely to be used immediately
- Simple policy: **First In First Out (FIFO)** evicts pages in the order in which they have been brought into memory
 - May be suboptimal, e.g., the first assigned pages may be important pages that are in use very often, leading to another page fault in near future
- Most commonly used policy: evict the **Least Recently Used (LRU)** page
 - Page has not been used for sometime, so less likelihood that it will be used in future
- Optimal policy: evict page not needed for longest time in future (impractical!)

Example: Optimal policy

- Example: Process accessed 4 pages (0,1,2,3), only 3 physical frames in memory
- First few accesses are cold (compulsory) misses (if OS doesn't assign any memory to process at start)
- Hit rate = $6/(6+5) = 54.5\%$
- Hit rate modulo compulsory misses 85%

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	2	0, 1, 3
0	Hit		0, 1, 3
3	Hit		0, 1, 3
1	Hit		0, 1, 3
2	Miss	3	0, 1, 2
1	Hit		0, 1, 2

Figure 22.1: Tracing The Optimal Policy

Example: FIFO

- Usually worse than optimal
- Belady's anomaly: performance may get worse when memory size increases!

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		First-in→	0
1	Miss		First-in→	0, 1
2	Miss		First-in→	0, 1, 2
0	Hit		First-in→	0, 1, 2
1	Hit		First-in→	0, 1, 2
3	Miss	0	First-in→	1, 2, 3
0	Miss	1	First-in→	2, 3, 0
3	Hit		First-in→	2, 3, 0
1	Miss	2	First-in→	3, 0, 1
2	Miss	3	First-in→	0, 1, 2
1	Hit		First-in→	0, 1, 2

Example: LRU

- Equivalent to optimal in this simple example
- Works well due to locality of references (recently used pages accessed again with high probability)

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		LRU→	0
1	Miss		LRU→	0, 1
2	Miss		LRU→	0, 1, 2
0	Hit		LRU→	1, 2, 0
1	Hit		LRU→	2, 0, 1
3	Miss	2	LRU→	0, 1, 3
0	Hit		LRU→	1, 3, 0
3	Hit		LRU→	1, 0, 3
1	Hit		LRU→	0, 3, 1
2	Miss	0	LRU→	3, 1, 2
1	Hit		LRU→	3, 2, 1

Figure 22.5: Tracing The LRU Policy

LRU implementation

- How does OS know which page is LRU?
 - OS is not involved in every memory access, so doesn't know which pages have been recently used
- Solution: MMU sets the **accessed bit** for every page table entry it accesses
 - Accessed bit is set implies page has been recently used
- Modern operating systems implement approximate LRU
 - Periodically, look at accessed bit of pages to classify pages into active and inactive pages
 - Pick pages that have been inactive for eviction
 - May also avoid dirty pages for eviction, since it requires extra disk write

Memory management system calls in user programs

Mythili Vutukuru
CSE, IIT Bombay

Recap: Virtual memory management

- Code+data of a process assigned virtual addresses
- Fixed size chunks of virtual addresses (pages) mapped to chunks of physical addressss (frames) in main memory
- Page table of process maps page# to frame#
 - Used by MMU for address translation
 - Page table mappings cached in TLB
- Demand paging: pages assigned physical frames only when used, else saved on swap space in disk

Recap: What happens on a memory access

- CPU accesses code/data at a certain virtual address (VA)
- MMU translates VA to PA, memory accessed using PA
 - If TLB hit, PA directly available, access memory directly
 - If TLB miss, MMU walks page table to compute PA, then access memory
- If any error during memory access, MMU traps to OS
 - If page fault due to frame not being allocated to the page, OS allocates free memory frame, updates page table entry, restarts process
 - Other types of page faults handled suitably
- Page replacement policy decides which pages to evict to swap and which to keep in memory frames

Memory allocation from user point of view

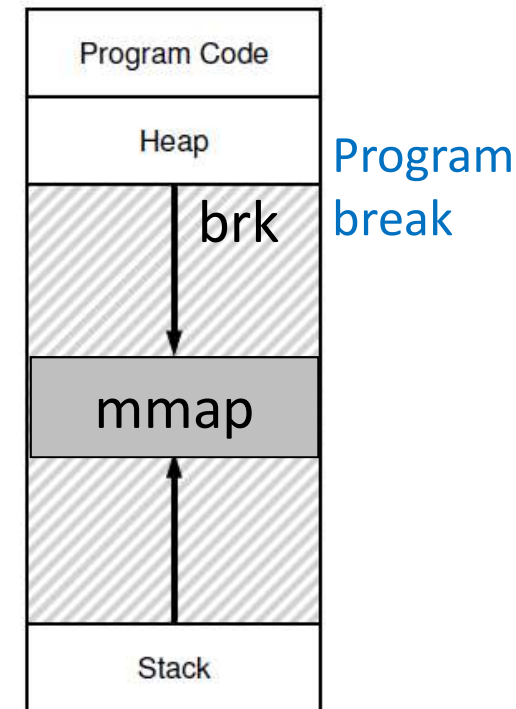
- So far, memory management from OS/hardware point of view
 - How OS manages memory, what data structures used, what happens on memory access, how is address translated, how is page fault handled
- Next: memory management from user program's view point
 - How is memory allocated to store user data in programs?
 - What system calls are available to user process for managing its memory?
 - How can a process request for more memory from OS as it runs?

Memory allocation to user processes

- When process created, OS assigns some memory to store memory image of process (code/data from executable, stack, heap, ..)
 - Stores allocation info in page table, given to MMU for address translation
- With demand paging, OS may not allocate memory for all pages initially, assigns frames only when process runs and accesses memory
 - Too little initial memory allocation → page faults when process starts
 - Too much initial memory allocation → possible waste of memory
- Memory allocation from OS to user processes always at page granularity (one or more pages)
- Within userspace, libraries can allocate memory in smaller chunks
 - C library function malloc assigns smaller chunks of memory from heap

Memory allocation system calls

- Address space created during **fork**, changed during **exec**
- No system calls needed to allocate memory that is already part of address space but unallocated by OS
 - Valid page but not present in RAM: automatically allocated memory by OS during page fault handling
- System calls to add pages, expand virtual address space
 - **sbrk/brk** system call adds pages to increase “program break”, i.e., ending virtual address of heap
 - **mmap** system call adds virtual pages at any address, not just at end of heap
- Stack expansion automatically handled by OS (stack pointer crosses existing stack page, page fault occurs)

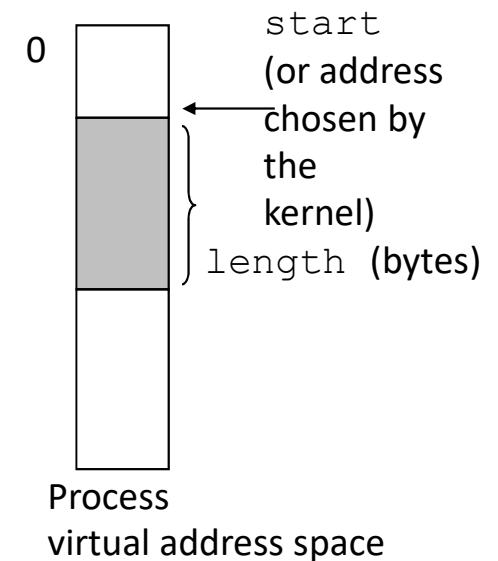


System call mmap

```
char *buf = mmap(start, length, ..)
buf[0] = ...
```

```
munmap(start, length)
```

- **mmap** system call used for mapping new memory into virtual address space of a process
- OS allocates new valid pages to address space, preferably at the requested starting virtual addresses
 - Physical memory allocated on demand
- Syscall returns the starting virtual address of the allocated chunk, used to read/write into the memory
- Memory contiguous in virtual address space, not necessarily in physical address space
- Correspondingly unmapped by **munmap** system call



More on memory mapping

- Two ways of memory mapping
 - Memory map an anonymous page, initialized to zero (e.g., heap page)
 - Memory map a file-backed page, initialized with file contents (e.g., code page)
- Physical memory corresponding to the virtual addresses allocated on demand in both cases
 - User accesses page, page fault, OS allocates and initializes physical frames
- How is memory-mapped page used?
 - Anonymous pages can be split into smaller chunks by heap manager, or can directly store user data
 - File-backed pages can be used to read/write files

Arguments to mmap

```
#include <sys/mman.h>
void *mmap(void *start, size_t length, int prot, int
flags, int fd, off_t offset);
```

- Starting virtual address (preference)
- Length of memory mapped chunk
- Protocol, i.e., permission bits (read, write)
- Flags (whether private or shared)
 - Shared implies changes by one process visible to another, not so with private
- File descriptor and offset if we are memory mapping a file into virtual address space (e.g., executable code or shared library object)

Heap vs Stack

- Program executable already allocates space for compile-time data (global and static local variables)
- Local variables in a function stored on stack

```
void func() {  
    int x; // declares an integer on the stack  
    ...  
}
```

- Memory allocated dynamically via malloc stored on heap (the virtual address of allocated chunk on heap may be stored in a local variable on stack)

```
void func() {  
    int *x = (int *) malloc(sizeof(int));  
    ...  
}
```

Image credit:OSTEP

Memory allocation via malloc

- Normally user programs don't need to call brk/sbrk/mmap functions to dynamically allocate memory at run time
- User programs calls C library functions like malloc to allocate chunks of memory from heap
- C library implementation of malloc uses brk/sbrk/mmap to obtain memory pages from OS when heap runs out of space
- User program can also directly allocate page-sized memory using mmap system call, instead of using malloc

Common errors with malloc

- Must allocate memory of the correct size before use

```
char *src = "hello";  
char *dst;           // oops! unallocated  
strcpy(dst, src);    // segfault and die
```

```
char *src = "hello";  
char *dst = (char *) malloc(strlen(src) + 1);  
strcpy(dst, src);    // work properly
```

- Allocated memory must be explicitly freed by user, else memory leak
 - Some language libraries automatically clean unused chunks via garbage collection algorithms

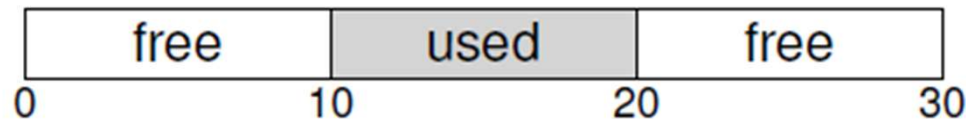
```
int *x = malloc(10 * sizeof(int));  
...  
free(x);
```

Types of memory allocators

- General-purpose heap allocators support variable sized memory allocation using malloc
 - Complex data structures to keep track of variable sized free chunks
 - Frequent calls to malloc/free can slow down user programs
- Some heap allocators optimize for fixed size allocation
 - Useful for user applications that allocate memory in fixed sizes
 - Heap memory is divided into fixed size chunks for allocation
 - More efficient than general-purpose variable sized allocation
- Choice of allocator managed by programming language libraries, but can be configured/changed by users

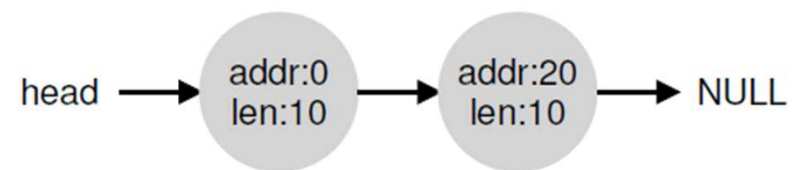
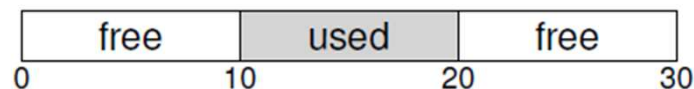
Variable sized memory allocation

- How do we design a general purpose memory allocator?
- Given a large block of memory (one or more pages), how do we use it to satisfy variable-sized memory allocation requests?
- Challenge: need to track information about variable sized chunks, which chunks are allocated and which are free
- External fragmentation: after memory is allocated and freed-up, many gaps of different sizes throughout heap



Free list

- Free list: data structure (e.g., linked list) to track free chunks in heap
- Initialized with entire heap, changes with alloc/free operations
- Malloc searches free list to find a suitable chunk
- Freed up chunks added back to free list
- Example: 30 bytes heap, three chunks of 10 types each allocated, two chunks freed up



Splitting and coalescing

- Exact size free chunk needed for alloc may not be available
- May need to split a bigger chunk, e.g., request to allocate 1 byte, split 10 byte chunk into 1 byte and 9 byte chunks



- May need to coalesce adjacent free chunks into bigger chunk, e.g., once middle 10 byte chunk is also free, can coalesce all free chunks into one 30 byte chunk, to satisfy requests for larger than 10 bytes



Which free chunk to pick in malloc?

- First fit: allocate first free chunk that fits
- Best fit: allocate free chunk that is closest in size
- Worst fit: allocate free chunk that is farthest in size
- Example, consider this free list, and request for malloc(15)



- Best fit allocates the 20-byte chunk
- Worst fit allocates 30-byte chunk, remaining chunk is bigger and more usable



Headers

- Allocated chunk stores some metadata (header), not used by user calling malloc
- Header stores size of allocation
 - Why? We should know how much to free later
- Header also has some random number (“magic”)
 - Detects if previous chunk has overflowed into this chunk

```
typedef struct __header_t {  
    int size;  
    int magic;  
} header_t;
```

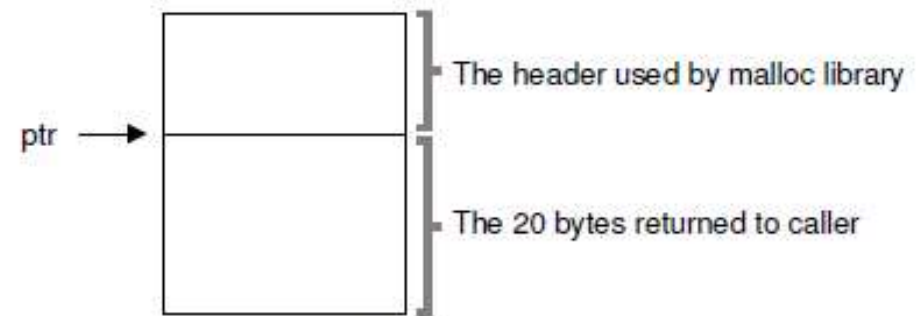


Figure 17.1: An Allocated Region Plus Header

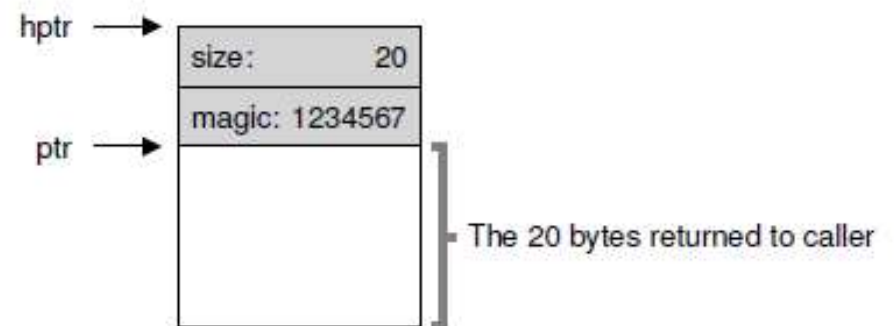


Figure 17.2: Specific Contents Of The Header

Embedded free list

- Where is free list stored? Information can be embedded within free chunk itself
- Pointer to the next free chunk is embedded within the free chunk
- Need to only remember head of list within allocator, e.g., allocations happen from the head, free chunks added back to head
- Example: 100 bytes allocated on heap

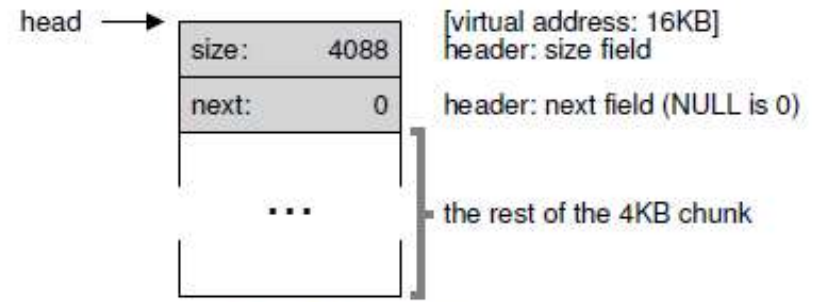


Figure 17.3: A Heap With One Free Chunk

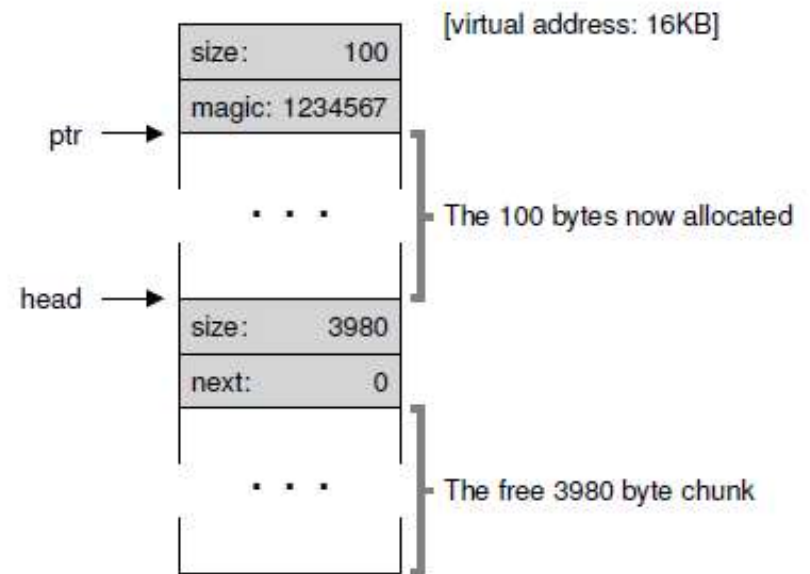


Figure 17.4: A Heap: After One Allocation

Example (contd)

- Heap has 4088 bytes initially (4KB = 4096, less 8 byte header)
- Three calls to malloc for 100 bytes each
- Each call allocates 108 bytes on heap (100 bytes to user, 8 byte header)
- Free list has one chunk left of 3764 bytes

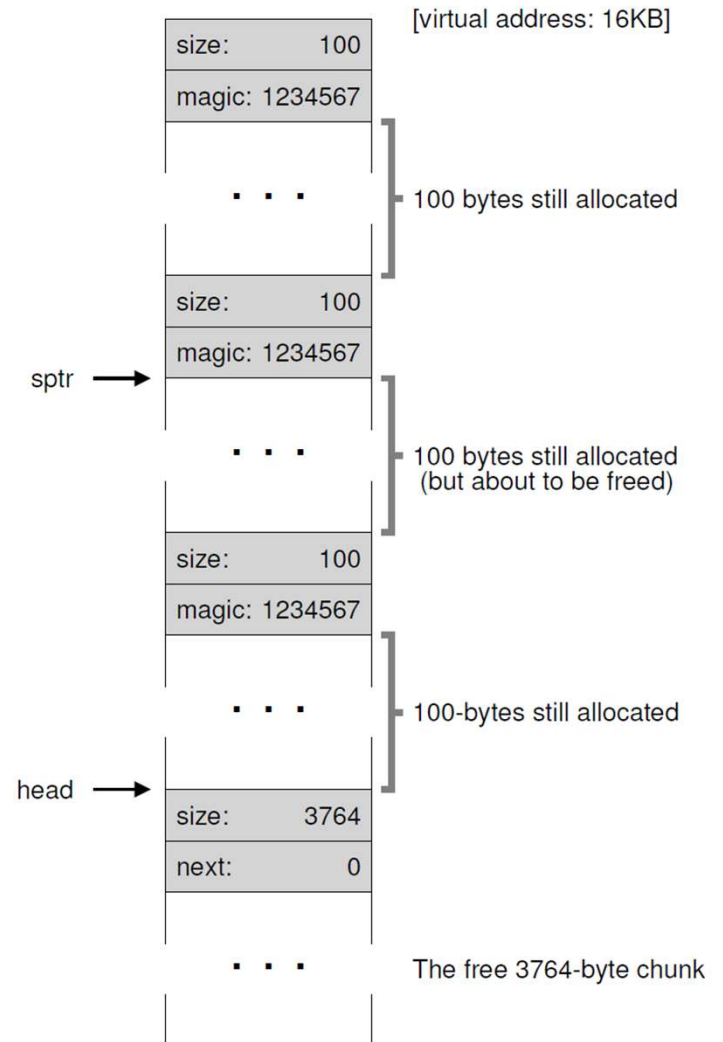
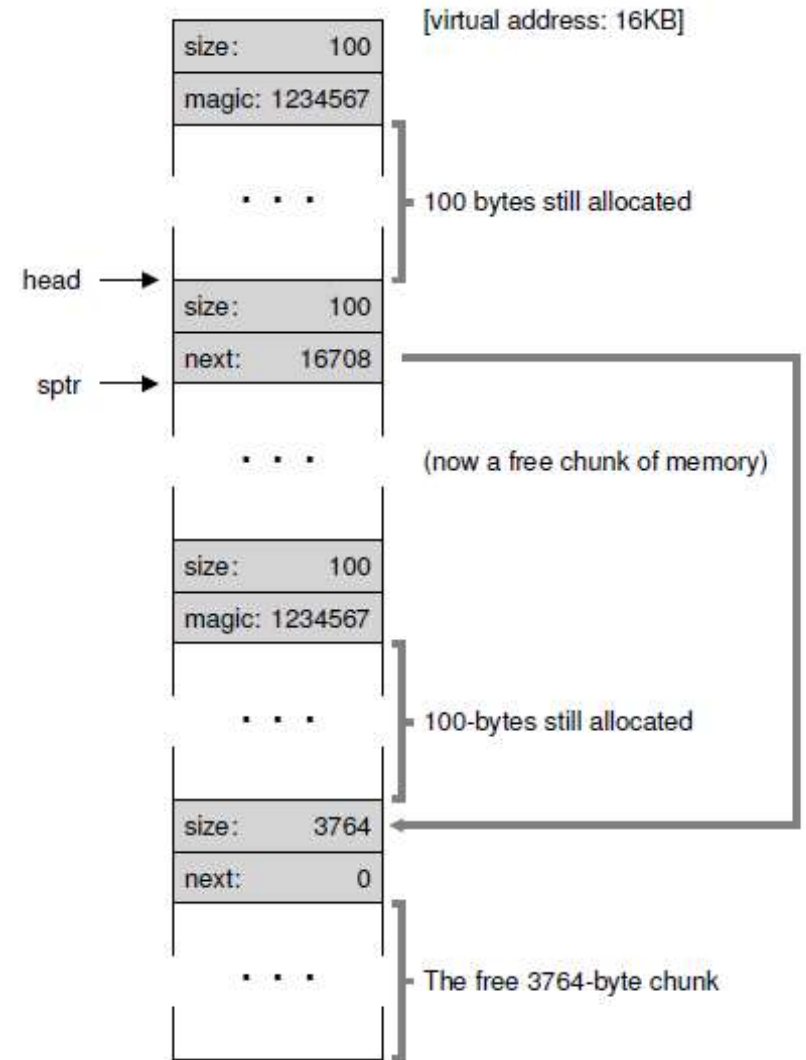


Figure 17.5: Free Space With Three Chunks Allocated

Example (contd)

- Now the middle chunk pointed to by `sptr` is freed
- What is the free list now?
Two non-contiguous chunks of 100 bytes each
- Free space scattered due to external fragmentation
 - Cannot satisfy a request for 3800 bytes even though we have the free space in heap



Example (contd)

- Suppose all the three chunks are freed
- The list now has a bunch of free chunks that are adjacent
- A smart algorithm would merge them all into a bigger free chunk
- Must split and coalesce free chunks to satisfy variable sized requests
- Lot of work to perform variable sized memory allocation

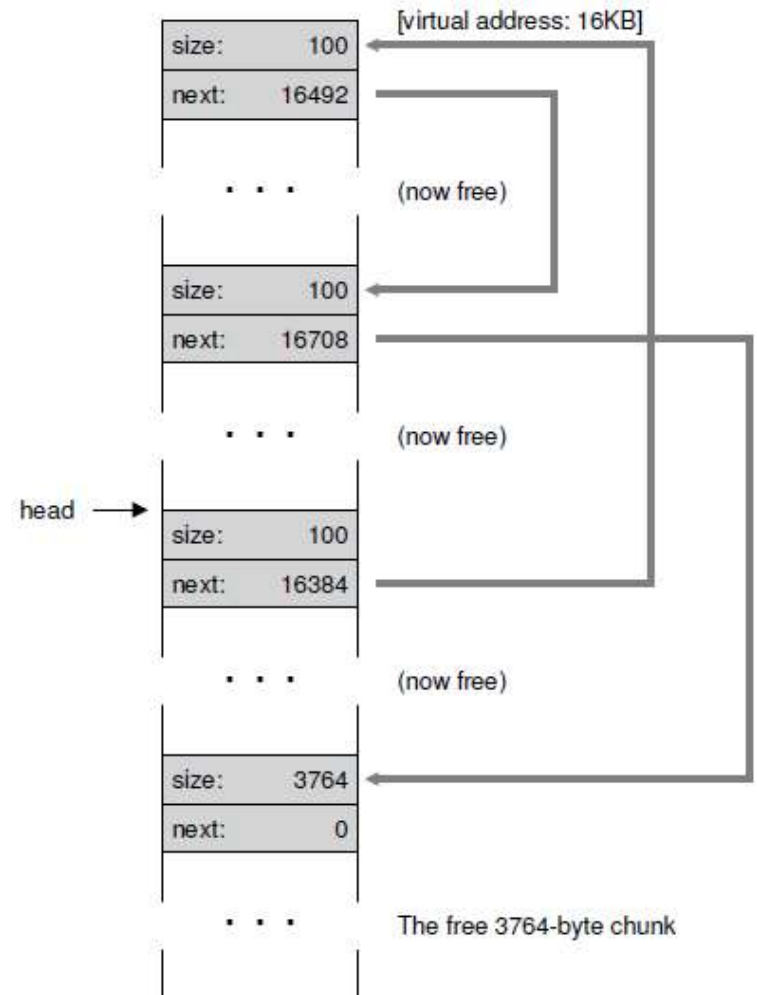


Figure 17.7: A Non-Coalesced Free List

Segregated free lists and slab allocators

- One idea for efficiency: maintain separate free lists for free chunks of different sizes
 - Useful when application calls malloc with popular chunk sizes
 - Requests that cannot be satisfied by segregated lists are handled by variable sized allocator
 - Less worry about external fragmentation, no need to split/coalesce
- Slab allocator: slab of memory with chunks of a particular size pre-allocated, easily available for use
 - Even more efficient than segregated free list, avoids alloc/free overheads

Buddy allocation for easy coalescing

- Allocate memory in size of power of 2
 - E.g., for a request of 7000 bytes, allocate 8 KB chunk
- Why? 2 adjacent power-of-2 chunks can be merged to form a bigger power-of-2 chunk
 - E.g., if 8KB block and its “buddy” are free, they can form a 16KB chunk

